

Universitatea Națională de Știință și Tehnologie POLITEHNICA
București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

*Sistem de Supraveghere și Analiză a Activității în Restaurante
Bazat pe Viziune Computerizată*

Proiect de diplomă

prezentat ca cerință parțială pentru obținerea titlului de
Inginer Inginerie Electronică, Telecomunicații și Tehnologia
Informației
Programul de studii de licență Electronică Aplicată

Conducător științific

Ș.L. dr. ing. Vasile-Mădălin MOISE

Absolvent

George SAUCIUC

2025

TEMA PROIECTULUI DE DIPLOMĂ
a studentului SAUCIUC T.O. George, 441B

1. Titlul temei: Sistem de Supraveghere și Analiză a Activității în Restaurante Bazat pe Viziune Computerizată

2. Descrierea temei:

Sistemul propus integrează hardware (cameră, microcontroler, PCB) și software (AI, OpenCV, YOLO) pentru a oferi o soluție completă de monitorizare a restaurantelor. Prin detectarea obiectelor în timp real și analiza imaginilor, sistemul poate determina starea meselor, genera rapoarte personalizate și optimiza procesele de servire. Sistemul include o cameră web montată pe un suport motorizat rotativ, controlat de un microcontroler integrat pe un PCB, care gestionează alimentarea motoarelor prin intermediul unui stabilizator de tensiune și primește comenzi de la un PC. Acest hardware este configurat pentru stabilizarea și poziționarea inițială a camerei, atât manual, cât și automat, la începutul aplicației. Sistemul este capabil să identifice și să monitorizeze starea fiecărei mese (disponibilă, necesită curățare, gata pentru comandă, ocupată), iar rezultatele sunt afișate într-o interfață grafică interactivă.

3. Contribuția originală:

Integrarea unui sistem hardware inovativ: Implementarea unui suport motorizat pentru camera controlată de microcontroler asigură poziționarea automată precisă, facilitând acoperirea completă a zonei monitorizate.

Adaptarea algoritmului de detecție AI: Optimizarea modelului YOLO pentru detectarea în timp real a meselor și persoanelor, cu ajustări pentru condiții variabile de iluminare și aglomerări de obiecte.

Automatizarea raportării: Crearea unui sistem automatizat pentru generarea rapoartelor zilnice, cu înregistrarea și monitorizarea duratei fiecărui status al meselor.

4. Resurse și bibliografie:

5. Data înregistrării temei: 2024-12-04 10:57:26

Conducător(i) lucrare,
Ș.L. dr. ing. Vasile-Mădălin MOISE

Student,
SAUCIUC T.O. George

Director departament,
Ș.L. dr. ing Bogdan FLOREA

Decan,
Prof. dr. ing. Mihnea UDREA

Cod Validare: **bc4ac98d77**

Cuprins

Lista Acronimelor	9
Introducere	11
Obiectul lucrării.....	11
Motivație alegerii temei	12
Contribuții originale și avantajele soluției dezvoltate	12
Avantajele soluției dezvoltate	13
Capitolul 2: Fundamente teoretice	15
2.1 Bazele teoretice ale viziunii computerizate.....	15
2.1.1 Definiția și scopul viziunii computerizate.....	15
2.1.2 Istoricul și evoluția domeniului.....	15
2.1.3 Procese fundamentale în viziunea computerizată	15
2.1.4. Tehnici și algoritmi de bază	16
2.1.5. Aplicații ale viziunii computerizate	16
2.1.6. Provocări și direcții viitoare	16
2.2 Algoritmul YOLO și detecția obiectelor în timp real	17
2.2.1. Introducere în detecția obiectelor.....	17
2.2.2. Apariția și principiul de funcționare al YOLO.....	17
2.2.3. Evoluția algoritmului YOLO	17
2.2.4. Avantajele și limitările YOLO	18
2.2.5. Aplicații ale YOLO în detecția obiectelor în timp real	18
2.3 Generarea și gestionarea automată a rapoartelor.....	18
2.3.1 Automatizarea generării și gestionării rapoartelor prin Python: biblioteci și abordări	18
2.3.2 Introducerea automatizării în procesul de raportare.....	18
2.3.4 Sinergia între biblioteci: un flux integrat	19
Capitolul 3: Analiza soluțiilor existente.....	21
3.1 Sisteme tradiționale și comerciale de monitorizare în restaurante.....	21
3.1.1 Metode tradiționale de supraveghere operațională.....	21
3.1.2 Soluții comerciale moderne pentru monitorizarea restaurantelor	21
3.1.3 Avantajele integrării sistemelor moderne	22
3.1.4 Considerații pentru adoptarea unei soluții comerciale	22
3.2 Sisteme actuale bazate pe modele AI preantrenate	22
3.2.1 Fundamentul modelelor AI preantrenate.....	22
3.2.2 Evoluția și diversitatea arhitecturilor AI preantrenate	23
3.2.3 Domenii de aplicare actuale	23
3.2.4 Avantajele utilizării modelelor AI preantrenate.....	23
3.2.5 Provocări și limitări.....	24
Capitolul 4: Proiectarea și implementarea hardware.....	25
4.1 Arhitectura generală a sistemului.....	25

4.1.1	Descrierea arhitecturii generale.....	25
4.1.2	Schema logică funcțională	25
4.1.3	Relația dintre componentele hardware	26
4.2	Componente hardware.....	27
4.2.1	Microcontrolerul ESP32-S3-MINI.....	27
4.2.2	Camera Intel RealSense D435i	29
4.2.3	ServoMotoarele MG996R	30
4.2.4	Alimentatorul universal 12V 2A	31
4.2.5	Stabilizatorul de tensiune LM1085ISX-5.0/NOPB.....	32
4.2.6	Stabilizatorul de tensiune TL1963A-33DCYR.....	33
4.2.7	Convertor USB–UART FT231XQ-R.....	34
4.2.8	Conector USB-C Molex 217179-0001.....	35
4.3	Schema electrică și implementarea hardware detaliată.....	36
4.3.1	Introducere generală și rolul plăcii PCB	36
4.3.2	Prezentarea generală a schemei electrice	37
4.3.3	Blocul de alimentare principală.....	38
4.3.4	Blocul de conversie 12V la 5V	39
4.3.5	Blocul de conversie 5V la 3.3V	40
4.3.6	Blocul de interfață USB-C	41
4.3.7	Blocul de conversie USB–UART	42
4.3.8	Blocul de control al semnalelor seriale (Serial Signals Handling)	43
4.3.9	Blocul microcontrolerului ESP32-S3-MINI	44
4.3.10	Blocul conectorilor de extensie (Headers)	45
4.3.11	Blocul butoanelor de control manual (RESET și BOOT).....	46
4.3.12	Blocul LED-urilor de stare (Power și User).....	47
4.4	Proiectarea, realizarea și fabricarea plăcii de circuit imprimat dedicată modului hardware.....	48
4.4.1	Obiectivele proiectării plăcii PCB.....	49
4.4.2	Specificații tehnice și parametri electrici	50
4.4.3	Descrierea stratificată a plăcii (stack-up).....	50
4.4.4	Straturile auxiliare ale plăcii de circuit imprimat.....	51
4.4.5	Procesul de proiectare CAD și verificare	51
4.4.6	Fabricarea și asamblarea plăcii	52
4.4.7	Testarea plăcii și verificarea funcționalității	54
4.4.8	Concluzii asupra procesului de realizare.....	55
Capitolul 5:	Implementarea software	57
5.1	Arhitectura generală a aplicației.....	57
5.1.1	Organizarea fișierelor și structura modulară a proiectului	57
5.1.2	Fluxul funcțional al aplicației.....	57
5.1.3	Tehnologii utilizate	58

5.2 Implementarea detecției obiectelor folosind YOLO	58
5.2.1 Motivația alegerii algoritmului YOLO	58
5.2.2 Structura detectorului YOLO (fișierul <code>yolo_detector.py</code>).....	59
5.2.3 Prelucrarea imaginii – <code>correct_illumination.py</code>	59
5.2.4 Fluxul detecției – de la imagine la etichetare	60
5.2.5 Filtrarea rezultatelor și clasificarea stării meselor.....	60
5.2.6 Integrarea rezultatelor în interfața grafică	61
5.2.7 Evaluarea performanței – viteză și acuratețe.....	61
5.2.8 Concluzii și beneficii ale implementării.....	61
5.3 Analiza și clasificarea stării meselor prin geometrie computațională.....	61
5.3.1 Introducere	61
5.3.2 Algoritmi pentru determinarea stării mesei pe baza poziției obiectelor.....	62
5.3.3 Calculul ariilor și al suprapunerii (IoU)	62
5.3.4 Praguri de relevanță și adaptabilitate	63
5.3.5 Rolul culorilor în procesul de clasificare	63
5.3.6 Optimizări pentru funcționare în medii reale	63
5.3.7 Funcția clasei <code>TableStatus</code> în ansamblul aplicației.....	63
5.4 Integrarea pragurilor temporale și generarea rapoartelor de stare.....	64
5.4.1 Introducere	64
5.4.2 Implementarea monitorizării temporale a fiecărei mese	64
5.4.3 Stocarea înregistrărilor în fișiere JSON	64
5.4.4 Analiza statistică și generarea rapoartelor în Excel	64
5.4.5 Interfața grafică pentru selecția rapoartelor	65
5.4.6 Utilizarea bibliotecilor Python pentru analiză și generare de rapoarte	65
5.4.7 Concluzii privind integrarea analizei temporale	65
5.5 Interfața grafică și rolul utilizatorului în configurarea sistemului.....	65
5.5.1 Introducere	65
5.5.2 Structura interfeței grafice.....	66
5.5.3 Controlul detecției și ajustarea parametrilor	66
5.5.4 Selectarea camerei și a rezoluției	67
5.5.5 Configurarea timpilor maximi și a numărului de persoane.....	67
5.5.6 Salvarea configurației.....	67
5.5.7 Interfața User GUI.....	68
5.5.8 Concluzii	68
5.6 Codul sursă al microcontrolerului ESP32-S3-MINI și integrarea acestuia.....	68
5.6.1 Rolul microcontrolerului în arhitectura sistemului	69
5.6.2 Descrierea generală a programului implementat.....	69
5.6.3 Inițializarea comunicației seriale.....	69
5.6.4 Recepționarea și interpretarea comenzilor	70

5.6.5 Testarea sistemului prin interfața grafică	70
5.6.6 Motivația utilizării unui baudrate ridicat.....	70
5.6.7 Posibilități de extindere	71
5.6.8 Concluzii	71
5.7 Generarea automată și manuală a rapoartelor statistice	71
5.7.1 Prezentare generală	71
5.7.2 Generarea automată a rapoartelor zilnice.....	71
5.7.3 Indicatorii calculați pentru analiza zilnică	72
5.7.4 Vizualizarea automată a performanței zilnice	72
5.7.5 Generarea manuală a rapoartelor comparative.....	72
5.7.6 Interpretarea statistică și evaluarea performanței.....	73
5.7.7 Structura finală a fișierelor generate	73
5.8 – Metodologia de testare software și depanare	73
5.8.1 Obiectivele testării.....	73
5.8.2 Testarea modulară a componentelor AI	74
5.8.3 Simularea comenzilor transmise către ESP32.....	74
5.8.4 Teste de stabilitate și stres	74
5.8.5 Testarea exportului și vizualizării datelor	74
5.8.6 Testarea interfeței grafice (GUI)	75
5.8.7 Debugging și logging	75
5.9 Direcții de extindere viitoare a aplicației	75
5.9.1 Control wireless al camerei video	75
5.9.2 Salvarea datelor într-o bază de date	76
5.9.4 Monitorizare în timp real prin browser	76
5.9.5 Suport multi-cameră și segmentare pe zone.....	76
Capitolul 6: Testarea, optimizarea și evaluarea sistemului integrat.....	77
6.1 Justificarea metodologiei de testare și platforma utilizată	77
6.2 Preprocesarea iluminării și impactul asupra detecției	77
6.3 Filtrarea detecțiilor suprapuse și stabilizarea logicii de interpretare.....	78
6.4 Adaptarea rezoluției video și comutarea inteligentă între modele	78
6.5 Concluzii generale ale testării și evaluării.....	78
Capitolul 7: Concluzii și perspective viitoare	79
Anexe	85
Anexa 1 – Schema Electrica.....	85
Anexa 2 – Layout-ul PCB-ului	86
Anexa 3 – Fișiere Gerber	89
Anexa 4 – Realizarea fizică a PCB-ului.....	91
Anexa 5 – Codul sursă al aplicației software și al modulului hardware	92

Lista Acronimelor

ADC = Analog to Digital Converter
AI = Artificial Intelligence
API = Application Programming Interface
CNN = Convolutional Neural Network
CPU = Central Processing Unit
CSV = Comma-Separated Values
DC = Direct Current
DNN = Deep Neural Network
ESP32 = Embedded Serial Processor 32-bit
FPS = Frames Per Second
GND = Ground
GPIO = General Purpose Input/Output
GUI = Graphical User Interface
HD / Full HD = High Definition / Full High Definition
I/O = Input/Output
IDE = Integrated Development Environment
JSON = JavaScript Object Notation
LCD = Liquid Crystal Display
LED = Light Emitting Diode
ML = Machine Learning
PCB = Printed Circuit Board
PWM = Pulse Width Modulation
RAM = Random Access Memory
RGB = Red Green Blue
UART = Universal Asynchronous Receiver-Transmitter
USB = Universal Serial Bus
TTL = Transistor–Transistor Logic
Wi-Fi = Wireless Fidelity
YOLO = You Only Look Once

Introducere

Obiectul lucrării

În contextul unei economii aflate într-un proces continuu de transformare tehnologică, digitalizarea și automatizarea proceselor din sectorul HoReCa devin factori cheie pentru creșterea performanței operaționale și îmbunătățirea serviciilor oferite clienților. Această lucrare urmărește realizarea unui sistem inteligent destinat monitorizării și analizei activităților dintr-un restaurant, prin utilizarea unor tehnologii avansate de viziune computerizată și inteligență artificială. Scopul principal constă în dezvoltarea unei soluții automatizate capabile să identifice și să evalueze în timp real starea meselor, furnizând informații esențiale pentru optimizarea fluxurilor de lucru. Prin integrarea coerentă a elementelor hardware și software într-o arhitectură funcțională, se urmărește obținerea unui instrument de suport decizional pentru managementul unităților de alimentație publică.

Sistemul proiectat are la bază o cameră video amplasată pe un suport motorizat, care permite supravegherea meselor din incintă. Întregul mecanism este coordonat de un microcontroler, iar imaginile captate sunt transmise către o platformă de procesare bazată pe inteligență artificială. Detecția obiectelor este realizată prin intermediul algoritmului YOLO (You Only Look Once), o soluție performantă de recunoaștere vizuală, optimizată pentru operare în timp real. Acest model a fost antrenat pentru a identifica obiecte relevante precum mese, farfurii, pahare, tăvi sau prezența clienților, facilitând determinarea exactă a stării fiecărei mese.

Clasificarea meselor este structurată pe mai multe categorii, precum: mese libere, mese ocupate, mese în așteptarea comenzii și mese care necesită curățare. Această abordare permite personalului să acționeze rapid și eficient, sprijinind rotirea optimă a meselor și oferind clienților o experiență îmbunătățită. Prin reducerea timpului de așteptare și prin organizarea mai eficientă a activităților, sistemul contribuie la o mai bună gestionare a resurselor.

Un alt obiectiv esențial al acestei lucrări constă în generarea automată de rapoarte privind utilizarea meselor. Analiza datelor istorice referitoare la durata fiecărei stări și la frecvența ocupării meselor permite evidențierea unor modele utile pentru optimizarea managementului resurselor. Aceste rapoarte pot reprezenta un instrument valoros pentru administratorii restaurantelor, sprijinindu-i în luarea unor decizii strategice, precum adaptarea programului personalului sau reorganizarea spațiului de servire pentru maximizarea eficienței operaționale.

Pe lângă aspectele tehnice, lucrarea explorează provocările asociate implementării unui astfel de sistem într-un cadru real. Factorii precum variațiile de iluminare, gradul de aglomerare sau alegerea poziției optime a camerei sunt elemente critice care pot influența performanța sistemului. Optimizarea algoritmilor de procesare a imaginilor și adoptarea unor tehnici avansate de prelucrare sunt esențiale pentru obținerea unei acurateți ridicate în detecție.

Prin combinarea componentelor hardware și software într-o soluție integrată, această lucrare demonstrează avantajele pe care le pot aduce viziunea computerizată și inteligența artificială în industria HoReCa. Învățarea profundă permite modelelor de calcul care sunt compuse din mai multe straturi de procesare să învețe reprezentări ale datelor cu mai multe niveluri de abstractizare[1], fapt ce subliniază potențialul acestor tehnologii de a analiza și interpreta automat date complexe. Sistemul realizat oferă o alternativă modernă față de metodele tradiționale de gestionare a resurselor și poate contribui semnificativ la eficientizarea proceselor operaționale. De asemenea, proiectul investighează modalități inovatoare de automatizare a activităților, ilustrând modul în care tehnologia poate transforma industria HoReCa.

În concluzie, lucrarea de față are ca obiectiv dezvoltarea unui sistem inteligent capabil să monitorizeze, să analizeze și să optimizeze activitățile desfășurate într-un restaurant, prin utilizarea tehnologiilor moderne de recunoaștere vizuală și procesare automată a datelor. Această abordare sprijină atât eficientizarea operațiunilor interne, cât și crearea unei experiențe superioare pentru clienți, contribuind astfel la procesul de digitalizare și modernizare a sectorului HoReCa.

Motivatia alegerii temei

Într-un peisaj economic marcat de transformări digitale accelerate, sectorul HoReCa se confruntă cu provocări legate de eficiența operațională și satisfacția clienților. Deși unele tehnologii au fost adoptate, nivelul de automatizare rămâne limitat. Această lucrare propune dezvoltarea unui sistem inteligent de monitorizare și analiză a activităților din restaurante, utilizând viziune computerizată și inteligență artificială, pentru a răspunde acestor nevoi.

Un factor determinant în alegerea acestei teme este absența soluțiilor accesibile și eficiente pe piață. Sistemele existente fie sunt inexacte, fie implică costuri ridicate, ceea ce le face inaccesibile pentru multe unități. De exemplu, tehnologiile bazate pe senzori de presiune sau RFID nu oferă o imagine completă și în timp real a activităților din restaurant. În acest context, integrarea unui sistem de viziune computerizată, capabil să detecteze și să analizeze automat starea meselor, poate aduce beneficii semnificative.

Adoptarea inteligenței artificiale în industria restaurantelor este în creștere, oferind oportunități pentru îmbunătățirea serviciilor și reducerea costurilor. Inteligența artificială îmbunătățește relațiile cu clienții, eficientizează operațiunile din bucătărie și modernizează managementul afacerilor în restaurantele de astăzi [2]. Acest lucru evidențiază potențialul AI de a transforma fundamental modul în care restaurantele își desfășoară activitatea.

În plus, utilizarea algoritmilor avansați de detecție a obiectelor, precum YOLO (You Only Look Once), permite identificarea rapidă și precisă a elementelor relevante din imagini. YOLO (You Only Look Once) este un model popular de detecție a obiectelor, cunoscut pentru viteza și precizia sa [3]. Această capacitate este esențială pentru monitorizarea eficientă a meselor și pentru luarea deciziilor în timp real.

Prin urmare, dezvoltarea unui sistem care combină viziunea computerizată cu inteligența artificială nu doar că răspunde unei nevoi actuale din industrie, dar oferă și o platformă pentru inovații viitoare în domeniul automatizării proceselor din restaurante.

Contribuții originale și avantajele soluției dezvoltate

În cadrul acestui proiect, au fost dezvoltate și implementate o serie de soluții inovatoare, menite să optimizeze procesele de monitorizare și gestionare a activităților într-un mediu de tip restaurant. Aceste contribuții originale includ:

1. Integrarea unui sistem hardware inovativ:

Implementarea unui suport motorizat pentru cameră, controlat de un microcontroler, permite poziționarea automată și precisă a dispozitivului de captură video. Această soluție asigură acoperirea completă a zonei monitorizate, eliminând necesitatea ajustărilor manuale și reducând astfel timpul și efortul necesar pentru supraveghere. Utilizarea unui astfel de sistem motorizat este esențială în aplicațiile de supraveghere, permițând capturarea unor imagini clare și stabile din diverse unghiuri.

2. GUI interactiv:

Dezvoltarea unei interfețe grafice interactive cu două componente majore – una destinată administratorilor și alta utilizatorilor – facilitează interacțiunea eficientă cu sistemul. Interfața de administrator permite testarea modelelor YOLO, configurarea parametrilor și controlul hardware-ului pentru setarea poziției inițiale a camerei, precum și exportarea fișierelor de configurare utilizate ulterior de către utilizatori. Această interfață oferă atât o analiză live printr-un sistem de detecție pe cadre, alarme și notificări, cât și un sistem de generare a rapoartelor periodice (zilnice, săptămânale, lunare). Implementarea unei astfel de interfețe grafice facilitează gestionarea și monitorizarea eficientă a sistemului de detecție, oferind utilizatorilor o experiență îmbunătățită și acces facil la funcționalitățile esențiale.

3. Adaptarea algoritmului de detecție AI:

Optimizarea modelului YOLO pentru detectarea în timp real a meselor și persoanelor a implicat ajustări specifice pentru a face față condițiilor variabile de iluminare și aglomerărilor de obiecte. Această adaptare asigură o detecție precisă și rapidă, esențială pentru monitorizarea eficientă a activităților din restaurant.

4. Automatizarea raportării:

Crearea unui sistem automatizat pentru generarea rapoartelor zilnice permite înregistrarea și monitorizarea duratei fiecărui status al meselor. Această funcționalitate oferă o imagine clară asupra utilizării meselor și a fluxului de clienți, facilitând luarea deciziilor informate pentru optimizarea serviciilor. Automatizarea raportării reduce erorile umane și economisește timp, oferind totodată acces rapid la date relevante pentru analiza performanței.

5. Geometrie computațională:

Utilizarea tehnicilor de geometrie computațională, cum ar fi calculul ariei, calculul Intersecției peste Uniune (IoU) și filtrarea obiectelor care se suprapun, permite o clasificare eficientă a statusului meselor în categorii precum "disponibilă", "pregătită pentru comandă", "în utilizare" și "necesită curățare". Această abordare asigură o gestionare optimă a meselor și îmbunătățește experiența clienților prin reducerea timpilor de așteptare și asigurarea unui mediu curat și organizat. Aplicarea geometriei computaționale în viziunea computerizată permite o analiză precisă și eficientă a obiectelor și a relațiilor spațiale dintre acestea, fiind esențială pentru dezvoltarea unor sisteme de detecție performante.

Avantajele soluției dezvoltate

Implementarea acestor contribuții originale aduce multiple beneficii:

- **Eficiență operațională:** Automatizarea proceselor de monitorizare și raportare reduce timpul și resursele necesare pentru gestionarea activităților zilnice.
- **Precizie îmbunătățită:** Adaptarea algoritmilor de detecție și utilizarea tehnicilor de geometrie computațională asigură o identificare exactă a statusului meselor și a prezenței persoanelor.
- **Flexibilitate și scalabilitate:** Sistemul poate fi ajustat și extins cu ușurință pentru a răspunde nevoilor specifice ale diferitelor restaurante, indiferent de dimensiune sau tipologie.

Îmbunătățirea experienței clienților: Monitorizarea eficientă și gestionarea optimă a meselor contribuie la reducerea timpilor de așteptare și la crearea unui mediu plăcut pentru clienți.

Capitolul 2: Fundamente teoretice

2.1 Bazele teoretice ale viziunii computerizate

Viziunea computerizată constituie un domeniu interdisciplinar al inteligenței artificiale, orientat spre dezvoltarea de metode și algoritmi care permit sistemelor informatice să interpreteze și să proceseze informații vizuale din lumea reală, precum imagini și videoclipuri. Scopul principal al acestui domeniu este de a conferi calculatoarelor capacitatea de a înțelege conținutul vizual într-un mod analog percepție umane.

2.1.1 Definiția și scopul viziunii computerizate

Viziunea computerizată se axează pe elaborarea de algoritmi și modele care permit extragerea de informații semnificative din date vizuale. Aceasta implică procese precum achiziția, procesarea, analiza și interpretarea imaginilor digitale, având ca obiectiv final reproducerea capacității umane de a percepe și înțelege mediul înconjurător. Astfel, viziunea computerizată urmărește automatizarea sarcinilor vizuale umane, cum ar fi recunoașterea obiectelor, interpretarea scenelor și înțelegerea contextului vizual.

Viziunea computerizată este un domeniu al informaticii care se ocupă cu dezvoltarea de metode pentru ca computerele să poată “vedea” și înțelege conținutul imaginilor digitale [4].

2.1.2 Istoricul și evoluția domeniului

Originile viziunii computerizate pot fi urmărite până în anii 1950 și 1960, când cercetătorii au început să exploreze modalități prin care computerele ar putea fi învățate să interpreteze imagini. Un moment semnificativ a fost dezvoltarea „Perceptronului” de către Frank Rosenblatt, un model de rețea neuronală artificială care a pus bazele sistemelor de viziune computerizată bazate pe rețele neuronale [5].

În anii 1970 și 1980, accentul s-a mutat către detectarea marginilor și extragerea caracteristicilor, iar progresele în tehnologia senzorilor, creșterea puterii de calcul și dezvoltarea algoritmilor de învățare automată au condus la avansuri semnificative în acest domeniu. Astăzi, viziunea computerizată este o componentă esențială a numeroase aplicații, de la vehicule autonome la sisteme de supraveghere și realitate augmentată.

2.1.3 Procese fundamentale în viziunea computerizată

Pentru a înțelege și interpreta imagini, sistemele de viziune computerizată parcurg mai multe etape esențiale:

- Achiziția imaginilor: Capturarea datelor vizuale utilizând diverse tipuri de senzori, cum ar fi camere digitale, scanere sau dispozitive LiDAR.
- Preprocesarea: Îmbunătățirea calității imaginilor și pregătirea acestora pentru analiza ulterioară prin tehnici precum filtrarea zgomotului, corecția iluminării și normalizarea contrastului.
- Extragerea caracteristicilor: Identificarea și izolarea elementelor relevante din imagini, cum ar fi margini, colțuri, texturi sau forme specifice, care pot fi utilizate pentru recunoașterea și clasificarea obiectelor.

- Analiza și interpretarea: Utilizarea caracteristicilor extrase pentru a identifica obiecte, detecta evenimente sau înțelege scene complexe, adesea prin aplicarea algoritmilor de învățare automată și rețele neuronale artificiale.

2.1.4. Tehnici și algoritmi de bază

În viziunea computerizată, diverse tehnici și algoritmi sunt utilizați pentru a îndeplini sarcini specifice:

- Detectarea marginilor: Identificarea conturilor obiectelor dintr-o imagine pentru a facilita segmentarea și recunoașterea acestora. Algoritmul Canny este un exemplu notabil în acest sens.
- Segmentarea imaginii: Împărțirea unei imagini în regiuni sau obiecte omogene pentru a simplifica analiza ulterioară. Metode precum segmentarea bazată pe regiuni sau pe contururi sunt frecvent utilizate.
- Recunoașterea obiectelor: Identificarea și clasificarea obiectelor prezente într-o imagine pe baza caracteristicilor extrase. Tehnici precum Histogramele de Orientări Gradiente (HOG) și Transformata Fourier sunt adesea aplicate.
- Reconstrucția 3D: Crearea unei reprezentări tridimensionale a unei scene sau a unui obiect utilizând imagini bidimensionale capturate din perspective diferite.

După cum se menționează în literatura de specialitate, segmentarea imaginii este procesul de împărțire a unei imagini în mai multe părți sau regiuni pentru a simplifica analiza ulterioară [6].

2.1.5. Aplicații ale viziunii computerizate

Datorită versatilității sale, viziunea computerizată are numeroase aplicații practice:

- Vehicule autonome: Sistemele de viziune computerizată permit vehiculelor să detecteze și să interpreteze semnele de circulație, pietonii și alte vehicule, contribuind la navigarea sigură și eficientă.
- Sisteme de supraveghere: Analiza automată a fluxurilor video pentru detectarea comportamentelor suspecte sau a evenimentelor neobișnuite în timp real.
- Asistență medicală: Interpretarea imaginilor medicale, cum ar fi radiografiile sau scanările RMN, pentru diagnosticarea și monitorizarea afecțiunilor.
- Realitate augmentată: Suprapunerea informațiilor digitale peste lumea reală, îmbunătățind experiența utilizatorului în domenii precum jocurile video, educația și designul industrial.

2.1.6. Provocări și direcții viitoare

Deși s-au realizat progrese semnificative, viziunea computerizată continuă să se confrunte cu provocări, cum ar fi recunoașterea obiectelor în condiții de iluminare variabilă, gestionarea ocluziilor și interpretarea scenelor complexe. Cercetările viitoare se concentrează pe dezvoltarea algoritmilor mai robusti, integrarea cu alte modalități senzoriale și îmbunătățirea generală a performanței sistemelor.

După cum se afirmă într-un articol de specialitate, provocările în viziunea computerizată includ calitatea datelor brute, procesarea în timp real, etichetarea datelor, scalabilitatea și preocupările etice și de confidențialitate [7].

2.2 Algoritmul YOLO și detecția obiectelor în timp real

Algoritmul YOLO (You Only Look Once) a adus o transformare semnificativă în domeniul viziunii computerizate, datorită capacității sale de a detecta obiecte în timp real cu o eficiență remarcabilă. De la introducerea sa în 2015 de către Joseph Redmon și colaboratorii săi, YOLO a evoluat continuu, devenind un instrument esențial în numeroase aplicații, de la vehicule autonome la sisteme de supraveghere. Această secțiune își propune să exploreze în detaliu principiile de funcționare ale YOLO, evoluția sa, avantajele și limitările, precum și aplicațiile practice ale acestui algoritm în detecția obiectelor în timp real.

2.2.1. Introducere în detecția obiectelor

Detecția obiectelor este un domeniu fundamental al viziunii computerizate, având ca scop identificarea și localizarea precisă a obiectelor într-o imagine sau într-un flux video. Metodele tradiționale, precum R-CNN (Region-based Convolutional Neural Networks), utilizează o abordare în două etape: generarea de regiuni de interes, urmată de clasificarea acestora. Deși aceste metode au oferit rezultate precise, ele sunt adesea lente și consumatoare de resurse computaționale, ceea ce le face mai puțin potrivite pentru aplicații în timp real.

2.2.2. Apariția și principiul de funcționare al YOLO

Algoritmul YOLO a fost introdus în 2015 de Joseph Redmon și colaboratorii săi, propunând o abordare unificată pentru detecția obiectelor. Spre deosebire de metodele anterioare, YOLO tratează detecția ca pe o problemă de regresie unică, analizând întreaga imagine într-o singură trecere a rețelei neuronale. Astfel, imaginea este împărțită într-o grilă de dimensiuni $S \times S$, iar fiecare celulă a grilei este responsabilă pentru prezicerea unor cutii de delimitare și a scorurilor de încredere asociate. Această abordare permite modelului să proceseze imagini la viteze ridicate, fiind ideal pentru aplicații care necesită detecție în timp real.

YOLO aplică o singură rețea neuronală asupra întregii imagini, care împarte imaginea în regiuni și prezice cutii de delimitare și probabilități pentru fiecare regiune [8]

2.2.3. Evoluția algoritmului YOLO

De la lansarea sa inițială, YOLO a trecut prin multiple îmbunătățiri și versiuni:

- YOLOv2 (YOLO9000): Lansat în 2016, a introdus normalizarea pe loturi (batch normalization), utilizarea unor imagini de rezoluție mai mare și ancore pentru predicția cutiilor de delimitare. Aceste îmbunătățiri au permis detectarea a peste 9000 de categorii de obiecte.
- YOLOv3: Apărut în 2018, a adus îmbunătățiri incrementale, inclusiv utilizarea unei arhitecturi mai complexe pentru rețeaua de bază și detectarea la multiple scale. De asemenea, a introdus o funcție de pierdere mai sofisticată pentru a îmbunătăți precizia detecției.
- YOLOv4 și versiunile ulterioare: Dezvoltate de diferiți cercetători, aceste versiuni au continuat să îmbunătățească performanța și să introducă noi caracteristici, consolidând poziția YOLO ca standard în detecția obiectelor în timp real.

Conform unei analize recente, YOLOv4 a îmbunătățit atât acuratețea, cât și viteza față de predecesorul său, utilizând o arhitectură CSPDarknet53 ca bază [9]

2.2.4. Avantajele și limitările YOLO

Unul dintre principalele avantaje ale YOLO este viteza sa remarcabilă. De exemplu, versiunea inițială putea procesa imagini la 45 de cadre pe secundă, iar variantele ulterioare au îmbunătățit și mai mult această performanță. Această rapiditate îl face potrivit pentru aplicații care necesită detecție în timp real, cum ar fi vehiculele autonome și sistemele de supraveghere.

Totuși, YOLO prezintă și anumite limitări. De exemplu, poate avea dificultăți în detectarea obiectelor mici sau a celor care se suprapun, deoarece fiecare celulă a grilei poate prezice un număr fix de obiecte. De asemenea, în versiunile anterioare, modelul putea produce mai multe erori de localizare comparativ cu alte metode.

După cum se afirmă într-un articol de specialitate, provocările în viziunea computerizată includ calitatea datelor brute, procesarea în timp real, etichetarea datelor, scalabilitatea și preocupările etice și de confidențialitate [10].

2.2.5. Aplicații ale YOLO în detecția obiectelor în timp real

Datorită vitezei și preciziei sale, YOLO a fost adoptat într-o gamă largă de aplicații:

- Vehicule autonome: YOLO este utilizat pentru a detecta pietoni, alte vehicule și obstacole, permițând vehiculului să ia decizii rapide și informate.
- Sisteme de supraveghere: În domeniul securității, YOLO este folosit pentru a identifica și urmări persoane sau obiecte suspecte în timp real.
- Asistență medicală: YOLO este aplicat în interpretarea imaginilor medicale, cum ar fi radiografiile sau scanările RMN, pentru diagnosticarea și monitorizarea afecțiunilor.
- Realitate augmentată: YOLO este utilizat pentru suprapunerea informațiilor digitale peste lumea reală, îmbunătățind experiența utilizatorului în domenii precum jocurile video, educația și designul industrial.

După cum se menționează într-o analiză recentă, YOLO este utilizat pe scară largă în diverse aplicații, cum ar fi mașinile autonome și sistemele de supraveghere [11].

2.3 Generarea și gestionarea automată a rapoartelor

2.3.1 Automatizarea generării și gestionării rapoartelor prin Python: biblioteci și abordări

În contextul digitalizării accelerate, implementarea unor soluții automate de generare și gestionare a rapoartelor reprezintă o direcție indispensabilă pentru optimizarea fluxurilor de lucru în multiple arii de activitate. Python, prin ecosistemul său extins de biblioteci specializate, constituie o platformă versatilă și eficientă pentru dezvoltarea unor astfel de soluții. Printre instrumentele esențiale utilizate se numără pandas, openpyxl, matplotlib, json și datetime, fiecare contribuind cu funcționalități specifice ce permit o procesare și prezentare a datelor rapidă și coerentă. Acest subcapitol explorează modul în care aceste biblioteci pot fi combinate într-un flux unitar de generare automată a rapoartelor, subliniind avantajele și aplicațiile practice asociate.

2.3.2 Introducerea automatizării în procesul de raportare

Automatizarea procesului de raportare presupune eliminarea intervenției manuale prin utilizarea unor algoritmi capabili să colecteze, prelucreze și afișeze date într-un format standardizat. Python, prin sintaxa sa intuitivă și suportul vast din comunitate, a devenit o opțiune predilectă în acest

domeniu. Python este adesea ales pentru proiecte de automatizare datorită capacității sale de a combina lizibilitatea codului cu puterea bibliotecilor externe [12]. Bibliotecile pandas, openpyxl, matplotlib, json și datetime se integrează armonios în acest proces, fiecare având un rol bine definit în lanțul de generare a raportului.

2.3.3 Biblioteci cheie utilizate în automatizarea rapoartelor

Biblioteca pandas joacă un rol central în manipularea și analiza seturilor de date complexe. Structurile sale fundamentale, precum DataFrame, permit organizarea datelor într-o formă tabelară ușor de gestionat. Pandas oferă un cadru puternic pentru transformarea și agregarea datelor, facilitând analiza exploratorie și pregătirea pentru raportare [13]. Operațiunile de filtrare, sortare, agregare și pivotare sunt implementate eficient, oferind utilizatorului un control granular asupra informațiilor.

Openpyxl este biblioteca dedicată interacțiunii cu fișierele Excel în format .xlsx, permițând atât citirea, cât și scrierea acestora. Prin intermediul său, se pot crea foi de calcul personalizate, se pot aplica stiluri, formule și chiar grafice. Openpyxl simplifică procesul de creare a rapoartelor Excel, eliminând necesitatea intervenției manuale pentru modificări repetitive [14]. Aceasta contribuie la păstrarea unui format profesional al rapoartelor generate.

Pentru vizualizarea datelor, matplotlib oferă suportul necesar construirii unei game variate de grafice: de la simple diagrame liniare, până la reprezentări complexe, animate sau interactive. Vizualizarea datelor este o componentă esențială a raportării, iar matplotlib facilitează această etapă prin flexibilitatea și gradul său ridicat de personalizare [15]. Integrarea graficelor în rapoarte adaugă valoare interpretativă și sprijină luarea deciziilor bazate pe date.

Biblioteca json, inclusă în standardul limbajului Python, facilitează manipularea datelor în format JSON, un format popular pentru schimbul de informații structurale. JSON reprezintă o soluție ușor de utilizat pentru stocarea și transmiterea datelor între aplicații [16]. În contextul raportării automate, această bibliotecă permite extragerea rapidă a datelor din fișiere sau API-uri care utilizează acest format.

Nu în ultimul rând, datetime oferă funcționalități esențiale pentru gestionarea datelor calendaristice și a timpului. Prin intermediul său, pot fi generate timestamp-uri, pot fi calculate diferențe de timp sau pot fi formate datele pentru afișare. Gestionarea corectă a timpului este vitală în rapoarte, fie că este vorba de marcarea evenimentelor, fie de calculul unor durate relevante [17].

2.3.4 Sinergia între biblioteci: un flux integrat

Integrarea acestor biblioteci într-un flux unitar permite construirea unor soluții robuste și automate pentru raportare. Un exemplu practic ar putea consta în următorii pași: datele sunt inițial extrase dintr-un fișier JSON, prelucrate și organizate cu ajutorul pandas, vizualizate printr-un grafic generat de matplotlib, iar rezultatul final exportat într-un fișier Excel utilizând openpyxl. Combinarea acestor instrumente transformă procesul de creare a rapoartelor dintr-o sarcină manuală laborioasă într-o soluție automatizată, reproductibilă și scalabilă [18].

Acest flux demonstrează nu doar funcționalitățile individuale ale fiecărei biblioteci, ci și modul în care acestea pot coopera pentru a livra rezultate complete și adaptabile nevoilor utilizatorului.

Capitolul 3: Analiza soluțiilor existente

3.1 Sisteme tradiționale și comerciale de monitorizare în restaurante

Monitorizarea activităților într-un restaurant constituie un factor esențial în menținerea unui climat operațional eficient, sigur și orientat spre satisfacerea clientului. Evoluția acestor sisteme de monitorizare a cunoscut un parcurs semnificativ, de la metode rudimentare, bazate pe observație directă, la soluții comerciale moderne, caracterizate prin automatizare, integrare și analiză în timp real.

3.1.1 Metode tradiționale de supraveghere operațională

În perioada anterioară digitalizării, unitățile din domeniul ospitalității se bazau în mare măsură pe supraveghere umană și documentare manuală. Aceste metode, deși adecvate în contextul tehnologic al epocii respective, prezentau limitări importante. Supravegherea era realizată, de regulă, prin inspecție vizuală de către personalul de conducere, în timp ce datele privind activitatea zilnică erau consemnate în jurnale fizice sau registre de inventar.

Utilizarea ceasurilor de pontaj mecanice pentru evidența timpului de lucru al angajaților era o practică frecventă. Cu toate acestea, aceste metode erau predispuse la erori umane, consumau mult timp și necesitau spații fizice pentru arhivarea documentelor. În lipsa unor instrumente digitale, monitorizarea manuală era adesea inexactă și dificil de corelat cu indicatori operaționali de performanță [19].

3.1.2 Soluții comerciale moderne pentru monitorizarea restaurantelor

Progresul tehnologic a permis apariția unor soluții specializate, create pentru a automatiza procesele interne și a asigura o supraveghere permanentă și detaliată a operațiunilor. Aceste sisteme comerciale integrează componente hardware și software ce permit analiza datelor în timp real și intervenții rapide.

Un exemplu elocvent îl reprezintă sistemele moderne de supraveghere video, care permit captarea, transmiterea și stocarea imaginilor din diverse zone ale restaurantului. Acestea pot fi corelate cu senzori de mișcare sau cu sisteme de control al accesului, pentru a furniza un tablou complet al securității. Sistemele video inteligente pot recunoaște modele comportamentale și detecta automat anomalii operaționale [20].

În paralel, soluții precum *EnergyManager* optimizează consumul energetic prin automatizarea funcționării echipamentelor, oferind tablouri de bord intuitive și notificări în timp real. Automatizarea consumului de energie nu doar că reduce costurile, dar contribuie semnificativ la sustenabilitate și eficiență energetică [21].

Totodată, sistemele de automatizare a clădirilor permit gestionarea centralizată a climatizării, iluminatului și altor procese esențiale, conducând la reducerea pierderilor și creșterea confortului. Ventilația inteligentă, spre exemplu, utilizează date de la senzori pentru a adapta în mod automat temperatura și fluxul de aer în funcție de gradul de ocupare al spațiului. Clădirile inteligente utilizează senzori pentru a crea un mediu controlat dinamic, reducând intervenția umană și erorile aferente [22].

3.1.3 Avantajele integrării sistemelor moderne

Integrarea acestor componente tehnologice generează o serie de beneficii evidente pentru orice unitate de alimentație publică. Prin automatizarea activităților de supraveghere, se reduce semnificativ timpul necesar pentru colectarea și interpretarea datelor, crescând eficiența managerială.

Optimizarea consumului de resurse – fie că este vorba de energie electrică, personal sau timp – conduce la o scădere a costurilor generale. În plus, sistemele moderne sporesc nivelul de securitate, prin detectarea în timp real a accesului neautorizat sau a comportamentului necorespunzător. Tehnologia video avansată permite o supraveghere continuă și intervenție proactivă în cazul incidentelor [23].

De asemenea, aceste platforme facilitează respectarea reglementărilor din domeniul sănătății și siguranței prin înregistrări automate, arhivare electronică și generarea de rapoarte personalizate la cerere.

3.1.4 Considerații pentru adoptarea unei soluții comerciale

Alegerea unui sistem de monitorizare trebuie realizată în funcție de specificul unității, volumul operațiunilor și nivelul dorit de integrare. Printre criteriile esențiale se numără:

- Scalabilitatea: soluția aleasă trebuie să permită extinderea ușoară odată cu creșterea afacerii.
- Compatibilitatea: sistemele implementate trebuie să se integreze cu infrastructura deja existentă, pentru a evita costurile suplimentare.
- Costurile totale: atât investiția inițială, cât și costurile de întreținere trebuie evaluate în raport cu beneficiile operaționale generate.
- Intuitivitatea interfeței: sistemele moderne trebuie să fie ușor de utilizat, reducând timpul de instruire a personalului.

Un sistem eficient de monitorizare nu trebuie să fie doar tehnologic avansat, ci și adaptat nevoilor reale ale operațiunilor cotidiene [24].

3.2 Sisteme actuale bazate pe modele AI preantrenate

Inteligența artificială (IA) s-a transformat, în ultimele decenii, într-o componentă esențială a progresului tehnologic global, influențând profund modul în care sunt abordate procesele complexe din multiple domenii. Printre inovațiile de referință se numără modelele AI preantrenate, algoritmi capabili să integreze cunoștințe vaste și să ofere soluții adaptabile și scalabile. Acest subcapitol își propune să analizeze sistemele actuale bazate pe astfel de modele, punând accent pe evoluția lor, aplicațiile curente, beneficiile și provocările pe care le implică.

3.2.1 Fundamentul modelelor AI preantrenate

Modelele AI preantrenate sunt algoritmi dezvoltați prin expunerea la seturi mari de date, permițându-le să învețe tipare și relații generale înainte de a fi specializate pentru sarcini particulare. Această metodă reduce semnificativ resursele necesare antrenamentului individualizat și scurtează timpul de implementare. Modelele preantrenate oferă o bază solidă de cunoștințe, eliminând necesitatea construirii unui sistem de la zero pentru fiecare aplicație nouă [25]. Un exemplu ilustrativ este BERT, dezvoltat de Google în 2018, care a redefinit standardele în procesarea limbajului natural.

3.2.2 Evoluția și diversitatea arhitecturilor AI preantrenate

În cursul evoluției inteligenței artificiale, modelele preantrenate au cunoscut o diversificare considerabilă, adaptându-se la nevoi tot mai complexe:

- Modele Generative Preantrenate (GPT): dezvoltate de OpenAI, aceste modele, inclusiv GPT-3 și GPT-4, sunt capabile să genereze texte coerente și contextuale, având aplicații variate în domeniul conversațional și al generării de conținut. Modelele generative transformă interacțiunea om-mașină prin abilitatea lor de a produce răspunsuri fluente și relevante [26].
- Modele bazate pe arhitectura Transformer: această arhitectură a permis o procesare paralelă eficientă a datelor, conducând la progrese remarcabile în traducerea automată, sumarizarea textelor și alte sarcini de procesare a limbajului. Arhitectura Transformer a stat la baza unor salturi semnificative în performanța modelelor lingvistice [27].
- Modele multimodale: acestea integrează procesarea simultană a mai multor tipuri de date – text, imagini, audio – și permit aplicații complexe în domenii precum realitatea augmentată sau analiza conținutului multimedia. Modelele multimodale deschid calea unor aplicații interdisciplinare, capabile să conecteze informații vizuale și lingvistice [28].

3.2.3 Domenii de aplicare actuale

Integrarea modelelor preantrenate a condus la soluții inovatoare în numeroase sectoare de activitate:

- Asistenți virtuali și chatbot-uri: aceste modele permit dezvoltarea unor interfețe conversaționale care pot interpreta și răspunde la solicitări complexe ale utilizatorilor. Asistenții virtuali, alimentați de AI, devin tot mai sofisticați, oferind răspunsuri personalizate și precise [29].
- Medicină și sănătate: modelele AI sunt utilizate pentru diagnosticare asistată, analiză imagistică și personalizarea tratamentelor, contribuind la îmbunătățirea actului medical. Inteligența artificială aduce un plus de acuratețe în diagnostic și o eficiență sporită în analiza volumelor mari de date medicale [30].
- Finanțe: în sectorul financiar, aceste modele sunt aplicate pentru identificarea fraudelor, analiza riscurilor și automatizarea serviciilor de consultanță. Modelele AI contribuie la detecția proactivă a fraudelor și la optimizarea proceselor decizionale în instituțiile financiare [31].
- Industria model: companiile din modă utilizează AI pentru a crea “gemeni virtuali” ai modelelor umane, reducând costurile și accelerând procesele de marketing digital. Modelele AI permit simularea realistă a hainelor pe modele virtuale, scurtând ciclurile de producție și promovare [32].

3.2.4 Avantaje utilizării modelelor AI preantrenate

Adoptarea acestei tehnologii aduce beneficii substanțiale:

- Eficiență și economie de resurse: reutilizarea modelelor preantrenate reduce costurile și timpul necesar dezvoltării de la zero. Modelele preantrenate democratizează accesul la inteligență artificială, permițând dezvoltatorilor să construiască aplicații complexe fără infrastructură masivă [33].
- Performanță îmbunătățită: antrenate pe volume mari de date, aceste modele au o capacitate sporită de generalizare și precizie. Expunerea la date diverse conferă modelelor preantrenate o robustețe superioară în fața variabilității datelor reale [34].

- Accesibilitate crescută: platforme precum Hugging Face sau TensorFlow Hub pun la dispoziția utilizatorilor biblioteci extinse de modele gata de utilizare. Ecosistemele open-source facilitează adoptarea rapidă a inteligenței artificiale prin partajarea resurselor preantrenate [35].

3.2.5 Provocări și limitări

Cu toate avantajele evidente, modelele preantrenate implică și dificultăți:

- Consumul ridicat de resurse: modelele mari necesită capacități hardware semnificative pentru inferență și fine-tuning. Dimensiunea modelelor avansate impune cerințe computaționale greu de susținut în medii limitate [36].
- Bias-uri algoritmice: aceste modele pot reflecta sau amplifica prejudecățile prezente în datele de antrenament, generând rezultate potențial discriminatorii. Bias-urile din seturile de date de antrenament pot afecta echitatea și corectitudinea rezultatelor AI [37].
- Lipsa de interpretabilitate: complexitatea arhitecturală a acestor modele face dificilă explicarea modului în care ajung la o concluzie. Opacitatea modelelor de tip cutie neagră ridică probleme de încredere și transparență, mai ales în domeniile critice [38].

Capitolul 4: Proiectarea și implementarea hardware

4.1 Arhitectura generală a sistemului

4.1.1 Descrierea arhitecturii generale

Arhitectura generală a sistemului propus în cadrul acestei lucrări integrează mai multe module hardware, concepute să colaboreze pentru atingerea obiectivului principal: monitorizarea automată și inteligentă a meselor într-un restaurant. Structura sistemului reflectă o abordare modulară, în care sarcinile de procesare și control sunt împărțite între diferite componente specializate.

În centrul arhitecturii se află patru blocuri funcționale principale: un calculator personal (PC) care acționează ca platformă de procesare software, o cameră video montată pe un sistem motorizat cu două axe, o placă de circuit imprimat (PCB) personalizată ce integrează microcontrolerul ESP32-S3-MINI și o sursă externă de alimentare de 12V/2A. Fiecare dintre aceste blocuri are un rol clar definit în cadrul sistemului, fiind interconectate atât la nivel fizic, cât și la nivel logic.

Camera video, capabilă să furnizeze imagini de înaltă calitate, este montată pe un mecanism motorizat cu două grade de libertate, acționat prin intermediul a două servomotoare. Controlul poziției camerei este realizat prin comenzi transmise de PC către ESP32, care generează semnalele necesare pentru mișcarea pe axele Ox și Oy. Platforma software de pe PC rulează algoritmul de detecție bazat pe rețeaua YOLO, având rolul de a analiza fluxul video și de a determina statusul meselor monitorizate.

Această arhitectură distribuie clar responsabilitățile între partea software și partea hardware, favorizând modularitatea și facilitând extinderea ulterioară a sistemului. Comunicarea dintre module se realizează printr-o conexiune USB-C, iar sursa de alimentare asigură tensiunile necesare funcționării atât a microcontrolerului, cât și a servomotoarelor.

4.1.2 Schema logică funcțională

Schema logică funcțională a sistemului este ilustrată în Figura 4.1 și prezintă modul în care principalele componente hardware interacționează pentru a susține funcționarea generală a proiectului. Această schemă evidențiază conexiunile esențiale între modulele hardware, precum și traseele de comunicație și alimentare care asigură integrarea funcțională a întregului ansamblu.

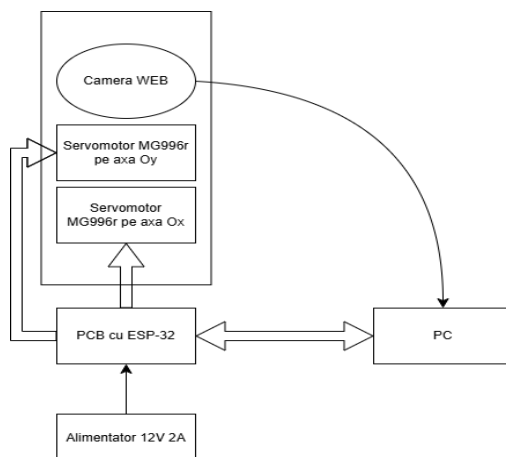


Figura 4.1 – Schema bloc

Conform Figura 4.1, sistemul este organizat în patru blocuri majore: camera Intel RealSense D435i montată pe un sistem motorizat cu două servomotoare MG996R (controlate pe axele Ox și Oy), o placă PCB echipată cu microcontroler ESP32, un calculator personal (PC) și o sursă de alimentare de 12V/2A. Fiecare dintre aceste componente este reprezentată schematic și conectată logic în fluxul de informații și energie.

Fluxul de date începe de la camera Intel RealSense D435i, care captează imagini din mediul monitorizat și le transmite direct către PC pentru procesare. PC-ul analizează fluxul video prin intermediul unui algoritm de detecție bazat pe YOLO și decide, în funcție de rezultatele procesării, dacă este necesară re poziționarea camerei. În cazul în care se impune ajustarea unghiului de vizualizare, PC-ul transmite comenzi către placa PCB cu ESP32 prin interfața USB-C, reprezentată în schema logică printr-o săgeată bidirecțională între PC și PCB.

Placa PCB primește aceste comenzi în format serial (convertite intern de la USB la UART) și, prin intermediul bibliotecii ESP32Servo, generează semnalele PWM necesare pentru controlul celor două servomotoare MG996R. Acestea sunt responsabile pentru orientarea camerei pe axele Ox și Oy, conform indicațiilor din schema funcțională. Traseul semnalelor de control este realizat prin conexiuni directe între pini specifici ai microcontrolerului și intrările de semnal ale celor două servomotoare, așa cum este reprezentat în schema logică.

De asemenea, schema logică prezintă circuitul de alimentare: sursa de 12V/2A furnizează tensiunea necesară întregului sistem, iar aceasta este distribuită către placa PCB, care reglează local tensiunile necesare pentru alimentarea servomotoarelor și a microcontrolerului. Legătura dintre sursa de alimentare și PCB este evidențiată printr-o conexiune directă în schema ilustrată.

În ansamblu, schema logică funcțională prezentată în Figura 4.1 sintetizează relațiile dintre componente, traseele de semnale și alimentare, precum și fluxul operațional al informațiilor, constituind o bază esențială pentru implementarea fizică și funcțională a sistemului.

4.1.3 Relația dintre componentele hardware

Relația dintre componentele hardware este definită atât de legăturile fizice dintre ele, cât și de rolurile complementare pe care le îndeplinesc în cadrul sistemului. Fiecare componentă este esențială pentru funcționarea coerentă a ansamblului, contribuind cu o funcție specifică într-un lanț operațional integrat.

Camera video are rolul de a capta imagini din mediul monitorizat, fiind montată pe un suport motorizat care permite orientarea în plan orizontal (Ox) și vertical (Oy). Mișcarea este realizată cu ajutorul a două servomotoare de tip MG996R, fiecare fiind alocat controlului unei singure axe. Aceste servomotoare sunt conectate direct la placa PCB prevăzută cu un microcontroler ESP32-S3-MINI, care generează semnalele PWM necesare pentru stabilirea precisă a unghiurilor de rotație, prin intermediul bibliotecii software ESP32Servo.

Placa PCB joacă rolul unui centru de control hardware, interfațând camera motorizată cu platforma de procesare software. Aceasta integrează microcontrolerul ESP32-S3-MINI, circuitele de alimentare și protecție, precum și interfețele de conectare necesare. Prin portul USB-C, placa PCB realizează comunicarea bidirecțională cu PC-ul, permițând recepția comenzilor în format serial și generarea corespunzătoare a semnalelor PWM necesare pentru controlul servo-motoarelor, prin intermediul codului firmware. Totodată, portul USB oferă un punct de acces facil pentru programare și depanare.

Sursa de alimentare externă de 12V/2A este conectată la PCB, asigurând energia necesară atât pentru funcționarea servomotoarelor, cât și pentru alimentarea microcontrolerului. Conversia locală a tensiunilor pe PCB permite distribuirea controlată a energiei către fiecare subsistem, evitând supratensiunile sau instabilitatea.

Prin această relație strânsă și coordonată între componente, sistemul asigură o funcționare sincronizată, în care fiecare element contribuie la realizarea obiectivului global de monitorizare automată și dinamică a meselor.

4.2 Componente hardware

4.2.1 Microcontrolerul ESP32-S3-MINI

ESP32-S3-MINI reprezintă un modul de ultimă generație dezvoltat de compania Espressif Systems, bazat pe microcontrolerul ESP32-S3. Este conceput pentru aplicații care impun un echilibru optim între putere de procesare, conectivitate wireless și dimensiuni reduse. Acest modul integrează atât funcționalități Wi-Fi, cât și Bluetooth Low Energy (LE), oferind o soluție completă pentru sisteme embedded, dispozitive IoT și aplicații portabile.



ESP32-S3-MINI-1

Figura 4.2 - ESP32-S3-MINI, sursa: [39]

În Figura 4.2 este prezentată o vedere tridimensională a modului ESP32-S3-MINI, ce evidențiază designul său compact, precum și distribuția componentelor pe suprafața plăcii. Designul urmărește maximizarea densității funcționale într-un spațiu redus, facilitând integrarea modului în plăci de circuit imprimate personalizate, unde spațiul disponibil este adesea limitat.

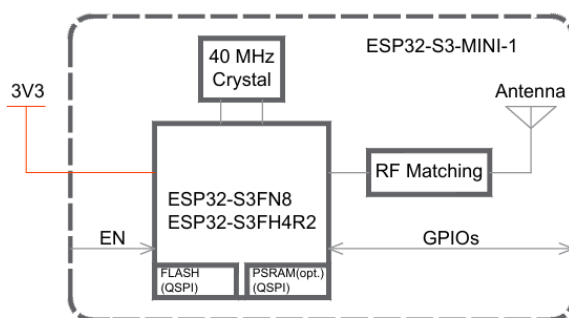


Figura 4.3 - diagramă bloc ESP32-S3-MINI, sursa: [39]

Arhitectura internă a modului este ilustrată în Figura 4.3, printr-o diagramă bloc ce pune în evidență conexiunile dintre principalele componente hardware: procesorul dual-core Xtensa® LX7, memoria flash externă, PSRAM-ul opțional, perifericele digitale și analogice, precum și interfețele de comunicație. Această arhitectură modulară permite procesarea paralelă a sarcinilor, gestionarea eficientă a datelor și suportul pentru mai multe protocoale de comunicație simultan.

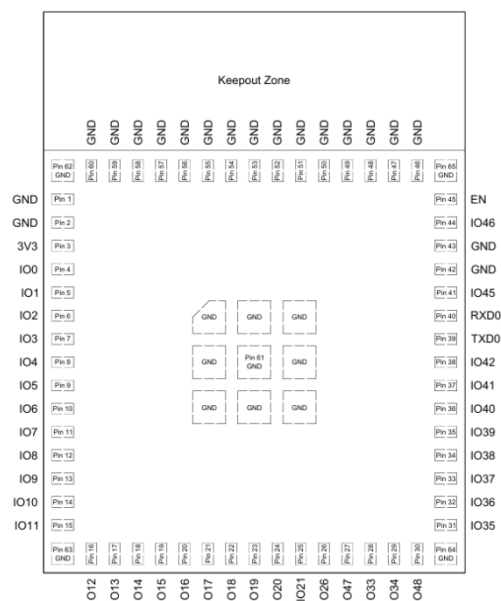


Figura 4.4 - pinout-ului modulului ESP32-S3-MINI, sursa: [39]

Pentru o mai bună înțelegere a resurselor disponibile, Figura 4.4 prezintă schema pinout-ului modulului ESP32-S3-MINI. Această schemă indică poziționarea și funcționalitatea fiecărui pin de intrare/ieșire, oferind informații esențiale pentru conectarea modulelor periferice, a senzorilor și a altor componente externe.

Specificarea completă a caracteristicilor tehnice ale modulului ESP32-S3-MINI este detaliată în Tabelul 4.1, conform documentației oficiale furnizate de Espressif Systems [39].

Caracteristică	Specificație
Procesor	Xtensa® 32-bit LX7 dual-core, până la 240 MHz
Memorie internă	512 KB SRAM, 384 KB ROM, 16 KB SRAM în RTC
Memorie externă	4 MB flash și 2 MB PSRAM sau 8 MB flash fără PSRAM
Conectivitate wireless	Wi-Fi 802.11b/g/n, Bluetooth 5 (LE)
Interfețe periferice	GPIO, ADC, DAC, I2C, SPI, UART, USB OTG
Tensiune de alimentare	3.0 – 3.6 V
Dimensiuni fizice	15.4 mm × 20.5 mm × 2.4 mm (cu antenă PCB integrată)
Temperatura de operare	-40°C până la +85°C

Tabelul 4.1 – Specificații tehnice ale modulului ESP32-S3-MINI [39]

Aceste specificații confirmă capacitatea modulului ESP32-S3-MINI de a susține aplicații complexe, care implică procesare locală rapidă, conectivitate multiplă și interacțiune cu periferice variate. Dimensiunile sale reduse, combinate cu suportul pentru interfețe multiple (GPIO-uri, ADC, DAC, I2C, SPI, UART, USB OTG), fac din acest modul o soluție extrem de flexibilă și adaptabilă.

În plus, integrarea nativă a funcționalităților Wi-Fi și Bluetooth LE contribuie la extinderea potențialului de conectivitate al sistemelor în care este utilizat, eliminând necesitatea modulelor externe suplimentare. Această abordare reduce atât costurile de implementare, cât și complexitatea generală a designului hardware.

Documentația oficială evidențiază și eficiența energetică a modulului, precum și suportul extins din partea comunității de dezvoltatori și a resurselor open-source disponibile online, aspecte esențiale pentru accelerarea procesului de dezvoltare și prototipare [39].

4.2.2 Camera Intel RealSense D435i



Figura 4.5 - camera Intel RealSense D435i, sursa: [40]

În cadrul sistemului de monitorizare, un rol esențial este deținut de camera Intel RealSense D435i, ilustrată în Figura 4.5. Aceasta face parte din seria D400 și oferă funcționalități avansate de captare a adâncimii și a imaginilor RGB. Camera D435i combină un senzor stereo activ, un proiector infraroșu și un IMU integrat pentru a furniza simultan imagini 3D și date de mișcare [39].

S-a utilizat exclusiv fluxul camerei RGB integrate, fără a exploata momentan capabilitățile de adâncime sau datele IMU. Alegerea s-a bazat pe necesitatea obținerii unor imagini clare în timp real, esențiale pentru procesarea prin YOLO. Camera D435i poate furniza imagini la 1920×1080 pixeli, cu până la 30 cadre pe secundă, permițând capturi de înaltă calitate într-un cadru dinamic [41].

În perspectivă, se dorește extinderea funcționalității prin integrarea datelor de adâncime în algoritmul de detecție a statusului meselor, pentru a îmbunătăți precizia în situații cu suprapuneri sau obstacole. Camera D435i permite măsurători de adâncime pe distanțe între 0,1 m și 10 m, oferind flexibilitate sporită în analiza tridimensională a scenelor [42]. Mai jos este prezentat Tabelul 4.2:

Specificație	Valoare
Tehnologie adâncime	Stereo activ cu proiector IR
Câmp vizual (FOV)	$86^\circ \times 57^\circ \times 95^\circ$
Rezoluție maximă RGB	1920×1080 pixeli
Cadre pe secundă (FPS)	Până la 90 FPS (la rezoluții mici)
Interval măsurare adâncime	0,1 m – 10 m
IMU integrat	Da
Conectivitate	USB 3.1 Type-C
Dimensiuni	$90 \text{ mm} \times 25 \text{ mm} \times 25 \text{ mm}$
Greutate	72 grame
Temperatura operare	$0^\circ\text{C} - 35^\circ\text{C}$

Tabel 4.2 – Date tehnice ale camerei Intel RealSense D435i

Integrarea camerei, prezentată în Figura 4.5, a oferit o soluție optimă pentru captarea imaginilor necesare detecției automate a meselor. Utilizarea exclusivă a fluxului RGB a permis o

integrare rapidă în aplicația AdminGUI, iar pe viitor, utilizarea simultană a canalelor RGB și de adâncime ar putea contribui la creșterea acurateții sistemului.

D435i oferă suport software extins pentru OpenCV și RealSense SDK, facilitând integrarea datelor vizuale și de adâncime în aplicații avansate de viziune computerizată [43]. Prin aceste capabilități, camera rămâne un element cheie în perspectiva extinderii funcționalităților sistemului.

4.2.3 ServoMotoarele MG996R

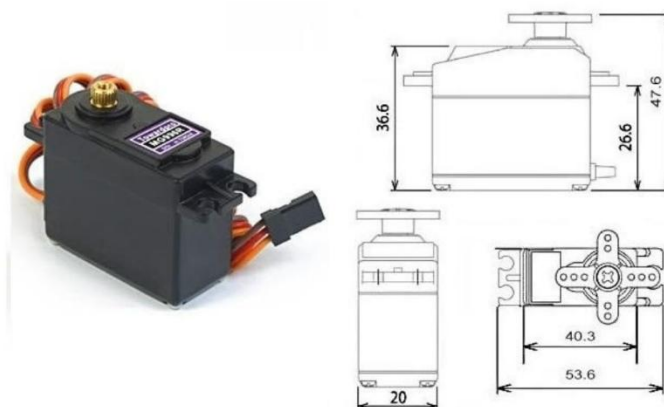


Figura 4.6 – servomotorul MG996R, sursa: [44]

Un alt element esențial în componența sistemului hardware al proiectului este servomotorul MG996R, ilustrat în Figura 4.6. Acesta a fost utilizat pentru acționarea mecanismului de orientare a camerei Intel RealSense D435i, permițând astfel ajustarea unghiului de vizualizare al zonei monitorizate în funcție de necesități.

MG996R face parte din categoria servomotoarelor digitale de cuplu mare, fiind o soluție accesibilă și fiabilă în aplicații de robotică și automatizare. Potrivit documentației oficiale, „MG996R este un servomotor digital de înaltă performanță, cu un cuplu maxim de 11 kg·cm la 6 V și o viteză de rotație de 0,17 secunde per 60°” [44]. Această caracteristică îl face potrivit pentru aplicații ce necesită control precis al poziției și o forță mecanică semnificativă, cum este cazul manipulării unei camere montate pe un braț sau suport.

Motorul este echipat cu angrenaje metalice, ceea ce îi conferă o rezistență sporită la uzură și o durată de viață mai mare în comparație cu motoarele similare cu angrenaje din plastic. „Construcția robustă cu angrenaje metalice asigură fiabilitatea motorului în condiții de sarcină mecanică ridicată, reducând riscul de deteriorare a transmisiei interne” [45].

Din punct de vedere al specificațiilor electrice, MG996R funcționează la o tensiune de alimentare standard de 4,8–7,2 V, având un consum de curent în gol de aproximativ 500 mA și un curent maxim ce poate atinge 2,5 A sub sarcină maximă [46]. Aceste valori impun utilizarea unui circuit de alimentare capabil să furnizeze curenți mari pe perioade scurte, pentru a evita pierderi de cuplu sau resetări accidentale.

Mai jos este prezentat Tabelul 4.3, realizat pe baza informațiilor furnizate în fișa tehnică oficială [44]:

Specificație	Valoare
Cuplu maxim	11 kg·cm la 6 V
Viteză rotație	0,17 s/60° la 6 V
Tensiune alimentare	4,8 – 7,2 V
Tip control	PWM
Unghi maxim rotație	120° – 180°
Material angrenaje	Metal
Dimensiuni	40,7 × 19,7 × 42,9 mm
Greutate	55 grame

Tabel 4.3 – Date tehnice ale servomotorului MG996R, sursa: [44]

Integrarea servomotorului MG996R în sistemul hardware a permis obținerea unui mecanism de orientare fiabil, capabil să susțină greutatea camerei Intel RealSense D435i și să asigure o mișcare precisă în plan orizontal. „MG996R suportă un unghi de rotație între 120° și 180°, în funcție de impulsurile PWM aplicate, oferind flexibilitate în controlul poziției” [47].

Prin caracteristicile sale tehnice și prin raportul cost-performanță avantajos, servomotorul MG996R s-a dovedit o alegere potrivită pentru sistemul dezvoltat, asigurând un echilibru între putere, precizie și fiabilitate.

4.2.4. Alimentatorul universal 12V 2A



Figura 4.7 - Alimentatorul universal 12V 2A, sursa:[48]

În cadrul sistemului hardware dezvoltat, alimentatorul universal 12V 2A, prezentat în Figura 4.7, joacă un rol esențial în asigurarea tensiunii și curentului necesar pentru funcționarea ansamblului electronic, în special a servomotoarelor MG996R și a plăcii de control ESP32-S3-MINI.

Acest alimentator furnizează o tensiune stabilizată de 12V continuu, având o capacitate de curent de până la 2A, suficientă pentru a susține simultan consumatorii principali ai sistemului. Potrivit documentației oficiale, „alimentatorul universal 12V 2A este proiectat pentru a oferi o sursă de energie fiabilă și stabilă pentru echipamente electronice de mică și medie putere” [48].

Un aspect important al utilizării acestui alimentator este legat de protecțiile integrate. „Dispozitivul este echipat cu protecție la supratensiune, supracurent și scurtcircuit, prevenind deteriorarea componentelor alimentate în cazul unor defecțiuni electrice” [48]. Aceste caracteristici

de siguranță sunt esențiale mai ales în aplicațiile ce implică motoare, unde fluctuațiile de curent sunt frecvente.

Alimentatorul beneficiază de un conector tip jack de 5,5 mm, compatibil cu majoritatea modulelor electronice standard. În cadrul proiectului, s-a utilizat un adaptor jack la terminal cu șuruburi, pentru a conecta direct cablurile de alimentare ale motoarelor și ale plăcii ESP32-S3-MINI. „Conectorul standardizat de 5,5 mm permite integrarea facilă a alimentatorului în diverse circuite fără modificări mecanice suplimentare” [48].

Din punct de vedere constructiv, carcasa este realizată din plastic ignifug, având dimensiuni reduse și greutate mică, aspecte ce contribuie la ușurința instalării în spații restrânse. Potrivit specificațiilor, „alimentatorul prezintă o eficiență energetică ridicată, de aproximativ 85%, reducând pierderile de căldură în regim continuu de funcționare” [48].

Mai jos este prezentat Tabelul 4.4, pe baza informațiilor din documentația oficială [48]:

Specificație	Valoare
Tensiune de ieșire	12V DC
Curent maxim	2A
Putere maximă	24W
Protecții integrate	Supratensiune, supracurent, scurtcircuit
Eficiență energetică	85%
Conector ieșire	Jack 5,5 mm
Dimensiuni	80 × 50 × 30 mm
Greutate	120 grame

Tabel 4.4 – Date tehnice ale alimentatorului universal 12V 2A, sursa: [48]

Integrarea acestui alimentator în sistemul hardware a permis alimentarea sigură și stabilă a componentelor, contribuind la menținerea unei funcționări optime a servomotoarelor și a plăcii de control. Alegerea sa s-a bazat pe raportul cost-eficiență și pe compatibilitatea cu cerințele de curent și tensiune ale echipamentelor utilizate.

4.2.5 Stabilizatorul de tensiune LM1085ISX-5.0/NOPB

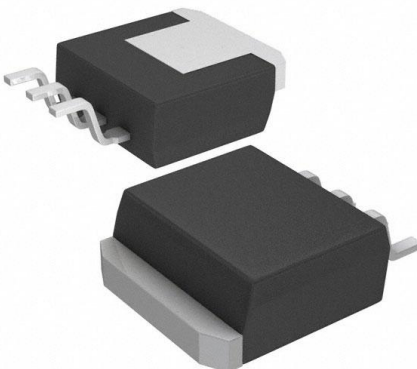


Figura 4.8 - LM1085ISX-5.0/NOPB, sursa:[49]

Pentru a asigura o alimentare stabilă și fiabilă a componentelor electronice sensibile din cadrul sistemului, s-a utilizat stabilizatorul de tensiune LM1085ISX-5.0/NOPB, prezentat în Figura 4.8. Acest regulator liniar de tensiune, fabricat de Texas Instruments, oferă o ieșire fixă de 5V și un curent maxim de 3A, fiind ideal pentru aplicații care necesită o sursă de tensiune constantă și curent ridicat.

Conform fișei tehnice oficiale, „LM1085 este un regulator cu un dropout maxim de 1,5 V la un curent de sarcină de 3 A” [50]. Această caracteristică îl face potrivit pentru aplicații unde diferența dintre tensiunea de intrare și cea de ieșire trebuie să fie minimă, reducând astfel disiparea de putere și îmbunătățind eficiența energetică.

Dispozitivul este echipat cu protecții interne împotriva supracurentului și supraîncălzirii, asigurând o funcționare sigură în condiții variate de sarcină. „Circuitul LM1085 include o referință bandgap ajustată cu zener, limitare de curent și oprire termică” [50]. Aceste funcții de protecție sunt esențiale pentru prevenirea deteriorării componentelor în cazul unor condiții de operare nefavorabile.

Mai jos este prezentat Tabelul 4.5, conform documentației oficiale:

Specificație	Valoare
Tensiune de ieșire	5V (fixă)
Curent maxim de ieșire	3A
Tensiune maximă de intrare	25V
Dropout maxim	1,5V la 3A
Protecții integrate	Supracurent, supraîncălzire
Tip pachet	TO-263 (DDPAK-3)
Interval temperatură operare	-40°C până la +125°C
PSRR (120Hz)	68 dB

Tabel 4.5 – Date tehnice ale stabilizatorului LM1085ISX-5.0/NOPB, sursa: [50]

Integrarea stabilizatorului LM1085ISX-5.0/NOPB în sistemul hardware a permis obținerea unei alimentări stabile de 5V pentru componentele critice, contribuind la fiabilitatea și performanța generală a proiectului.

4.2.6 Stabilizatorul de tensiune TL1963A-33DCYR

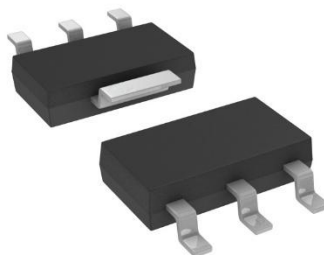


Figura 4.9 - TL1963A-33DCYR, sursa: [51]

Pentru a asigura o alimentare stabilă și fiabilă a componentelor electronice sensibile din cadrul sistemului, s-a utilizat stabilizatorul de tensiune TL1963A-33DCYR, prezentat în Figura 4.9. Acest

regulator liniar de tensiune, fabricat de Texas Instruments, oferă o ieșire fixă de 3,3V și un curent maxim de 1,5A, fiind ideal pentru aplicații care necesită o sursă de tensiune constantă și curent moderat.

Conform fișei tehnice oficiale, „TL1963A-33 este un regulator cu un dropout maxim de 340 mV la un curent de sarcină de 1,5 A” [52]. Această caracteristică îl face potrivit pentru aplicații unde diferența dintre tensiunea de intrare și cea de ieșire trebuie să fie minimă, reducând astfel disiparea de putere și îmbunătățind eficiența energetică [52].

Dispozitivul este echipat cu protecții interne împotriva supracurentului, supraîncălzirii, tensiunii inverse și curentului invers, asigurând o funcționare sigură în condiții variate de sarcină. „Circuitul TL1963A-33 include protecție la supracurent, limitare termică, protecție la tensiune inversă și protecție la curent invers” [52]. Aceste funcții de protecție sunt esențiale pentru prevenirea deteriorării componentelor în cazul unor condiții de operare nefavorabile [52].

Mai jos este prezentat Tabelul 4.6, conform documentației oficiale [52]:

Specificație	Valoare
Tensiune de ieșire	3,3V (fixă)
Curent maxim de ieșire	1,5A
Tensiune maximă de intrare	20V
Dropout maxim	340 mV la 1,5A
Protecții integrate	Supracurent, supraîncălzire, tensiune inversă, curent invers
Tip pachet	SOT-223-4
Interval temperatură operare	-40°C până la +125°C
PSRR (120Hz)	63 dB

Tabel 4.6 – Date tehnice ale stabilizatorului TL1963A-33DCYR, sursa: [52]

Integrarea stabilizatorului TL1963A-33DCYR în sistemul hardware a permis obținerea unei alimentări stabile de 3,3V pentru componentele critice, contribuind la fiabilitatea și performanța generală a proiectului.

4.2.7 Convertor USB–UART FT231XQ-R

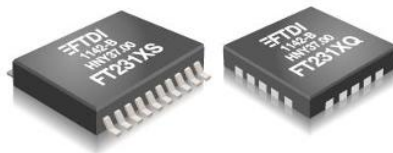


Figura 4.10 - USB–UART FT231XQ-R

Pentru a facilita comunicarea între microcontrolerul ESP32-S3-MINI și un computer personal prin interfață USB, în cadrul sistemului hardware a fost integrat circuitul FT231XQ-R, prezentat în Figura 4.10. Acest circuit integrat, fabricat de FTDI, oferă o soluție compactă pentru conversia

semnalelor USB 2.0 Full Speed în semnale UART, eliminând necesitatea unei programări suplimentare a firmware-ului USB.

Conform fișei tehnice oficiale, „FT231XQ-R este un convertor USB 2.0 Full Speed către interfață UART cu handshake complet, capabil să opereze până la 3 Mbps” [53]. Dispozitivul include o memorie MTP integrată și un circuit de ceas intern, eliminând necesitatea unui cristal extern și reducând astfel numărul de componente externe necesare. De asemenea, consumul de curent este redus, având un consum tipic de 8 mA în regim activ și 125 μ A în modul suspendat.

Mai jos este prezentat Tabelul 4.7, conform documentației oficiale [53]:

Specificație	Valoare
Interfață USB	USB 2.0 Full Speed
Viteză de transfer UART	Până la 3 Mbps
Tensiune de alimentare	3,3V – 5,5V
Curent de consum tipic	8 mA
Curent în modul suspendat	125 μ A
Pachet	QFN-20 (4x4 mm)
Temperatura de operare	-40°C până la +85°C
Funcții suplimentare	EEPROM integrat, ceas intern

Tabel 4.7 – Date tehnice ale convertorului FT231XQ-R, sursa: [53]

Integrarea convertorului FT231XQ-R în sistemul hardware a permis o comunicare eficientă și fiabilă între microcontroler și computer, contribuind la ușurința în programare și diagnosticare a sistemului.

4.2.8 Conector USB-C Molex 217179-0001



Figura 4.11 - USB-C Molex 217179-0001, sursa: [54]

Pentru a asigura o interfață de alimentare și comunicație modernă și robustă în cadrul sistemului hardware, a fost integrat conectorul Molex 217179-0001, prezentat în Figura 4.11. Acest conector USB 2.0 de tip C, cu montare la unghi drept și fixare prin tehnologie SMD, oferă o soluție compactă și fiabilă pentru aplicații electronice moderne.

Conform fișei tehnice oficiale, „conectorul 217179-0001 este un receptacol USB 2.0 de tip C, cu 24 de poziții (16 active și 8 neconectate), montare la unghi drept și fixare prin tehnologie SMD”

[54]. Acesta suportă un curent maxim de 5A și o tensiune de până la 30V AC/DC, fiind ideal pentru aplicații care necesită o alimentare eficientă și transfer de date fiabil.

Mai jos este prezentat Tabelul 4.8, conform documentației oficiale [54]:

Specificație	Valoare
Tip conector	USB 2.0 Type-C
Număr de contacte	24 (16 active + 8 neconectate)
Montare	SMD, unghi drept
Curent maxim	5A
Tensiune maximă	30V AC/DC
Durabilitate	10.000 cicluri de cuplare
Temperatura de operare	-40°C până la +85°C
Material contacte	Aliaj de cupru, placat cu aur
Material carcasă	Oțel inoxidabil

Tabel 4.8 – Date tehnice ale conectorului Molex 217179-0001, sursa: [54]

Integrarea conectorului Molex 217179-0001 în sistemul hardware a permis o conectivitate fiabilă și durabilă, asigurând compatibilitatea cu standardele moderne USB și contribuind la eficiența generală a proiectului.

4.3 Schema electrică și implementarea hardware detaliată

4.3.1 Introducere generală și rolul plăcii PCB

Realizarea unei plăci PCB personalizate a constituit un element fundamental în arhitectura hardware a sistemului de monitorizare automată dezvoltat în cadrul proiectului. Această placă nu servește doar ca suport fizic pentru componentele electronice, ci și ca un hub central de alimentare, interfațare și conectivitate pentru întreg sistemul bazat pe microcontrolerul ESP32-S3-MINI-1-N8. În acest subcapitol se descrie în detaliu rolul funcțional al plăcii și structura sa electrică, respectând schema reală implementată.

Placa a fost proiectată utilizând software-ul Altium Designer și este realizată pe patru straturi. Această configurație multi-layer a permis separarea eficientă a traseelor electrice și a planurilor de alimentare și masă. Structura straturilor este organizată astfel:

- Layer 1 (roșu): trasee pentru liniile de +12V, +5V, semnale de comunicație USB și conexiuni diferențiale între portul USB-C și convertorul FT231XQ-R.
- Layer 2 (galben): trasee dedicate semnalelor de +3,3V și rutărilor suplimentare neacoperite pe primul strat.
- Layer 3 (mov): plan complet de masă, utilizat pentru întreg PCB-ul, asigurând stabilitatea electrică și reducerea interferențelor.
- Layer 4 (albastru): trasee pentru semnalele GPIO de la microcontroler către conectorii de extensie, alături de alte semnale auxiliare.

Un aspect important al designului este integrarea zonelor de cupru suplimentare pe straturile de semnal, utilizate ca heatsink pentru disiparea eficientă a căldurii generate de stabilizatoarele de tensiune.

Microcontrolerul ESP32 este alimentat din două surse alternative: fie dintr-o intrare externă de +12V, redusă succesiv la +5V prin stabilizatorul LM1085ISX-5.0/NOPB și apoi la +3,3V prin stabilizatorul TL1963A-33DCYR, fie prin portul USB-C, în cazul activării jumperului 5VBUS→5V. Activarea jumperului conectează linia VBUS de la USB la intrarea stabilizatorului de 5V, permițând alimentarea completă a plăcii prin USB. În lipsa jumperului, alimentarea prin USB este utilizată exclusiv pentru comunicație și programare.

Blocul de comunicație USB–UART este realizat cu integratul FT231XQ-R, conectat la portul USB-C. Semnalele diferențiale USB_D+ și USB_D- sunt filtrate și protejate de diodele AQ3045-01ETG și trase prin rezistențe de 27 Ω (R5, R6), asigurând potrivirea de impedanță și reducerea zgomotului. Circuitul generează semnalele TXD_FTDI și RXD_FTDI, transmise către ESP32 prin rezistențe 0 Ω (R14, R15), utilizate ca legături directe sau jumperi de configurare.

Pentru controlul inițializării automate și a resetării, semnalele RTS_FTDI și DTR_FTDI sunt rutate prin tranzistori SS8050 (Q2, Q3) și rezistențe 10k Ω (R1, R4), conform schemei, permițând activarea pinilor CHIP_PU și GPIO0 fără intervenție manuală. Opțional, placa include două butoane tactile RESET (SW1) și BOOT (SW2), conectate la aceiași pini, pentru control manual.

Pentru feedback vizual, sunt integrate două LED-uri de stare (D6 – verde și D7 – roșu), fiecare conectat la +3,3V prin rezistențe de 1k Ω (R12, R13), utile pentru diagnosticarea rapidă a funcționării plăcii.

Placa expune o serie de pini GPIO prin conectorii J3 și J4, oferind acces direct la majoritatea pinilor microcontrolerului. În plus, conectorii J5 și J6 asigură acces suplimentar la GPIO6 și GPIO7, alături de alimentare +5V și GND, destinați extinderilor hardware.

În zona de protecție, schema include un tranzistor IRF5305PBF (Q1), o diodă Zener 10V (D1) și o diodă TVS AVR-M1608C120MT6AB (D2), pentru protecția liniilor de alimentare la supratensiuni și comutare controlată.

4.3.2 Prezentarea generală a schemei electrice

Schema electrică a plăcii PCB personalizate din cadrul acestui proiect integrează o serie de blocuri funcționale interconectate, fiecare având un rol bine definit în asigurarea funcționării optime a sistemului de monitorizare automată. În această secțiune este prezentată o imagine de ansamblu asupra acestor blocuri, evidențiind structura logică și relațiile dintre componentele principale. Detalierea fiecărui bloc va fi realizată în subcapitolele următoare.

Primul bloc esențial al schemei este blocul de alimentare, responsabil pentru gestionarea și stabilizarea tensiunilor necesare operării circuitelor. Acesta integrează două stabilizatoare liniare: unul pentru reducerea tensiunii de intrare de +12V la +5V și al doilea pentru conversia de la +5V la +3,3V, tensiune utilizată de microcontroler și de restul componentelor logice. Blocul de alimentare include și componente de protecție, precum siguranțe, diode TVS și zener, precum și condensatori de filtrare pentru stabilizarea tensiunilor.

Un alt bloc important este blocul de conversie USB–UART, realizat prin integratul FT231XQ-R, care permite comunicarea și programarea microcontrolerului prin intermediul unui port USB-C.

Acest bloc asigură transformarea semnalelor USB în semnale UART compatibile cu ESP32 și integrează circuite auxiliare pentru controlul resetării și modului de boot al microcontrolerului.

În centrul schemei se află blocul de microcontroler, reprezentat de modulul ESP32-S3-MINI-1-N8, conectat la o serie extinsă de pini GPIO expuși prin conectori externi. Acest bloc este completat de circuite auxiliare de inițializare, control și protecție, inclusiv butoanele RESET și BOOT, precum și rezistențele de pull-up și pull-down necesare configurării inițiale.

Schema mai include și un bloc de semnalizare vizuală, alcătuit din două LED-uri conectate prin rezistențe de limitare a curentului, utilizate pentru indicarea stării plăcii sau a altor evenimente definite software.

Pentru interfațarea cu componente externe, placa oferă un bloc de extensie GPIO, realizat prin conectori suplimentari care expun pinii GPIO6 și GPIO7, împreună cu linii de alimentare +5V și GND, destinate extinderilor hardware viitoare sau conectării unor periferice suplimentare.

În completarea acestor blocuri, schema integrează un bloc de protecție și comutare, care include tranzistorul IRF5305PBF, o diodă zener și circuite asociate, menite să protejeze și să controleze liniile de alimentare și semnal în diverse condiții de operare.

Toate aceste blocuri sunt interconectate prin trasee plasate pe o arhitectură cu 4 straturi, fiecare strat având un rol funcțional bine definit: primul strat alocând traseelor de alimentare și comunicație USB, al doilea strat semnalelor de +3,3V, al treilea strat servind ca plan complet de masă, iar al patrulea strat fiind destinat semnalelor GPIO și altor trasee suplimentare.

Astfel, schema electrică prezintă o structură modulară și clar organizată, fiecare bloc având o funcție bine delimitată, contribuind la realizarea unui sistem hardware fiabil, protejat și ușor de extins. În subcapitolele următoare vor fi prezentate detaliat fiecare dintre aceste blocuri, incluzând explicații privind componentele utilizate, traseele implicate și interacțiunile interne.

4.3.3 Blocul de alimentare principală

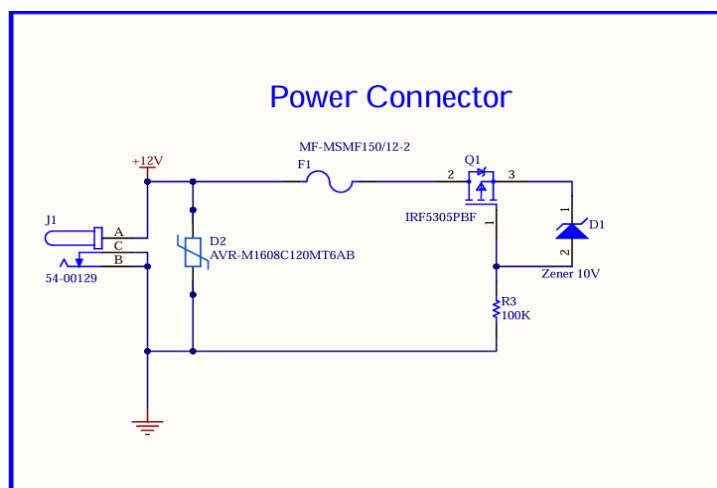


Figura 4.12 – Blocul de alimentare principală

Blocul de alimentare principală are rolul de a prelua tensiunea externă de +12V și de a asigura protecția sistemului împotriva supracurenților și supratensiunilor, după cum este ilustrat în Figura 4.12. Acesta este prezentat în Figura 4.11 și include următoarele componente esențiale: conectorul de alimentare J1, siguranța F1, dioda TVS D2, tranzistorul Q1, dioda Zener D1 și rezistența R3.

Conectorul J1 este un port de alimentare tip baril care acceptă un adaptor de +12V, tensiune ce reprezintă sursa principală de energie pentru întreaga placă. Imediat după conector, tensiunea trece prin siguranța resetabilă F1 (tip PTC), care întrerupe automat curentul în cazul unei suprasarcini și revine la starea inițială după răcire.

Pentru protecția împotriva supratensiunilor tranzitorii (ESD, vârfuri de tensiune), pe linia de +12V este conectată o diodă TVS (D2). Aceasta are rolul de a limita tensiunea prin descărcarea rapidă a energiilor dăunătoare direct către masă.

Tensiunea protejată ajunge apoi la un tranzistor MOSFET canal P (IRF5305PBF, Q1), care funcționează ca întrerupător controlat. Comanda porții tranzistorului este realizată cu ajutorul unei diodă Zener de 10V (D1) și a unei rezistențe de 100k Ω (R3). Acest circuit stabilizează tensiunea pe poartă și menține tranzistorul în regim de conducție cât timp tensiunea de intrare este sigură. Dacă tensiunea depășește pragul de funcționare al diodei zener, tranzistorul se va opri, blocând alimentarea către restul circuitului.

Prin această arhitectură, blocul de alimentare asigură:

- Protecție la supracurent (siguranță F1),
- Protecție la supratensiune tranzitorie (diodă TVS),
- Filtrare și control activ al alimentării (MOSFET + Zener).

Acest bloc pregătește tensiunea de intrare pentru a fi transmisă către stabilizatorul de 5V, oferind astfel o bază solidă și protejată pentru întregul sistem electronic al plăcii.

4.3.4 Blocul de conversie 12V la 5V

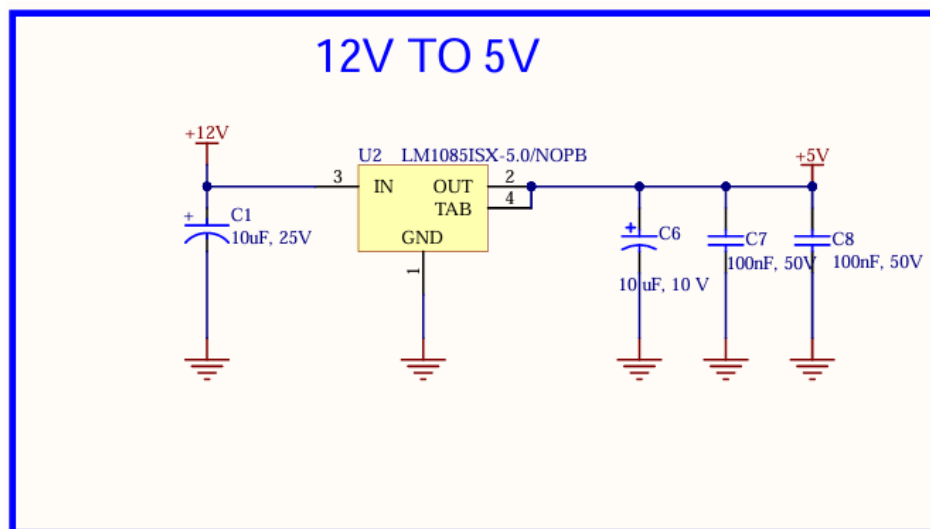


Figura 4.13 - Blocul de conversie 12V la 5V

După ce tensiunea de +12V a fost filtrată și protejată de blocul anterior, următorul pas în arhitectura electrică a plăcii îl reprezintă conversia eficientă la +5V, necesară pentru alimentarea intermediară a microcontrolerului și a altor componente auxiliare. Acest proces este realizat cu ajutorul unui regulator de tensiune liniar, LM1085ISX-5.0/NOPB, ilustrat în Figura 4.13.

Stabilizatorul U2 este un regulator liniar de joasă tensiune, capabil să furnizeze până la 3A la ieșire, fiind astfel ideal pentru aplicații care necesită o alimentare curată și stabilă. Acesta este conectat cu trei terminale: IN (intrare), OUT (ieșire) și GND (masă), iar carcasa sa de tip DDPAK/TO-

263-3 permite montarea pe suprafață și disiparea eficientă a căldurii prin zona de cupru plasată dedesubt.

Pe linia de intrare, tensiunea de +12V este aplicată prin pinul IN și filtrată de condensatorul C1 (10μF, 25V), care are rolul de a amortiza eventualele fluctuații sau zgomote de alimentare. În paralel, pe ieșirea pinului OUT este conectat un grup de condensatori: C6 (10μF, 10V) și doi condensatori ceramici C7, C8 (100nF, 50V). Aceștia asigură o filtrare corespunzătoare a tensiunii de +5V, reducând ripple-ul și îmbunătățind stabilitatea în regim de sarcină variabilă.

Valoarea tensiunii de ieșire este fixă, stabilizatorul LM1085ISX-5.0 fiind un model preconfigurat pentru 5V, fără a necesita rezistențe de ajustare. Acest avantaj simplifică implementarea și reduce erorile de proiectare în comparație cu versiunile ajustabile. De asemenea, regulatorul oferă protecție internă la supracurent și supratemperatură, contribuind la creșterea fiabilității întregului sistem.

Tensiunea de +5V obținută la ieșirea acestui bloc este apoi direcționată către următorul stabilizator, care va furniza +3,3V către microcontrolerul ESP32 și alte componente logice. Astfel, acest bloc reprezintă o etapă intermediară esențială în lanțul de alimentare al plăcii PCB.

Prin utilizarea unui regulator liniar în acest punct al lanțului, se asigură o tensiune curată și lipsită de zgomot, ideală pentru aplicații sensibile din punct de vedere electric, cum ar fi circuitele de comunicație și componentele logice.

4.3.5 Blocul de conversie 5V la 3.3V

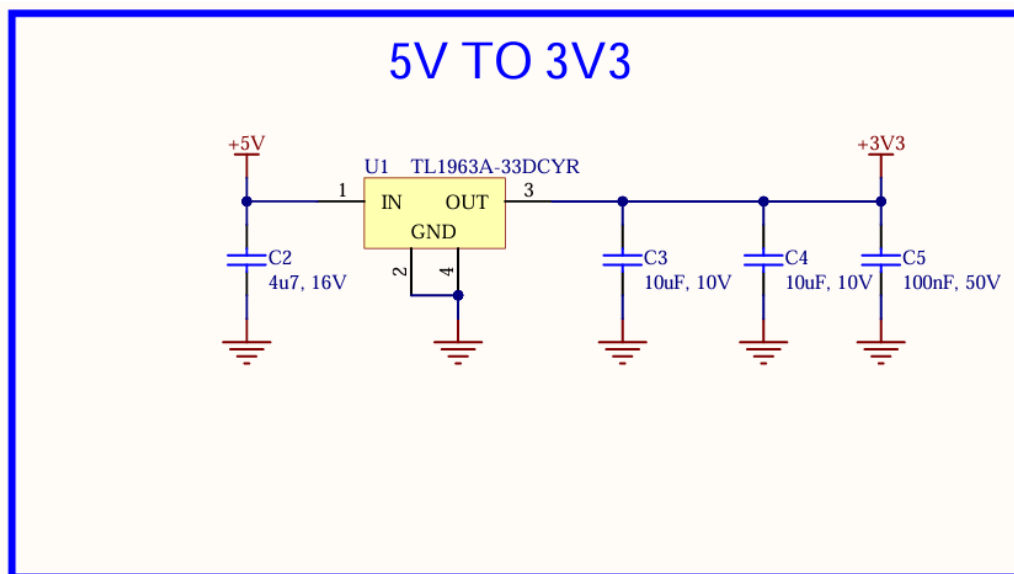


Figura 4.14 - Blocul de conversie 5V la 3.3V

După stabilizarea tensiunii de +5V în blocul anterior, următoarea etapă în lanțul de alimentare o constituie conversia la +3,3V, tensiunea necesară microcontrolerului ESP32 și altor componente logice de joasă tensiune. Acest rol este îndeplinit de un regulator liniar de înaltă performanță, TL1963A-33DCYR, a cărui schemă este ilustrată în Figura 4.14.

Stabilizatorul U1 este un LDO (Low Dropout Regulator), capabil să furnizeze până la 1,5A curent de ieșire, fiind ideal pentru aplicații embedded unde stabilitatea tensiunii este esențială. În această configurație, regulatorul are o tensiune de ieșire fixă de +3,3V, fiind alimentat la intrare cu +5V proveniți de la stabilizatorul precedent (LM1085).

Pentru asigurarea unei alimentări stabile, regulatorul este însoțit de un set de condensatori de filtrare:

- C2 (4.7 μ F, 16V) este plasat la intrare pentru a elimina eventualele fluctuații din linia de 5V;
- C3 și C4 (10 μ F, 10V) sunt conectați în paralel la ieșire și ajută la reducerea ripple-ului și menținerea unei tensiuni constante chiar și în condiții de sarcină variabilă;
- C5 (100nF, 50V) este un condensator ceramic care filtrează componentele de frecvență înaltă și îmbunătățește răspunsul la tranziții.

Prin această combinație de capacitive, se obține o alimentare extrem de curată și stabilă, esențială pentru funcționarea corectă a microcontrolerului și a comunicațiilor digitale sensibile.

Stabilizatorul TL1963A include și protecții interne la scurtcircuit, supracurent și supratemperatură, oferind un plus de fiabilitate în utilizare. Ieșirea sa este conectată la toate componentele digitale alimentate la 3.3V, inclusiv ESP32-S3-MINI-1-N8 și LED-urile de stare.

Astfel, acest bloc de conversie joacă un rol critic în arhitectura de alimentare a plăcii PCB, asigurând o tensiune precisă, stabilă și sigură pentru funcționarea întregii logicii digitale.

4.3.6 Blocul de interfață USB-C

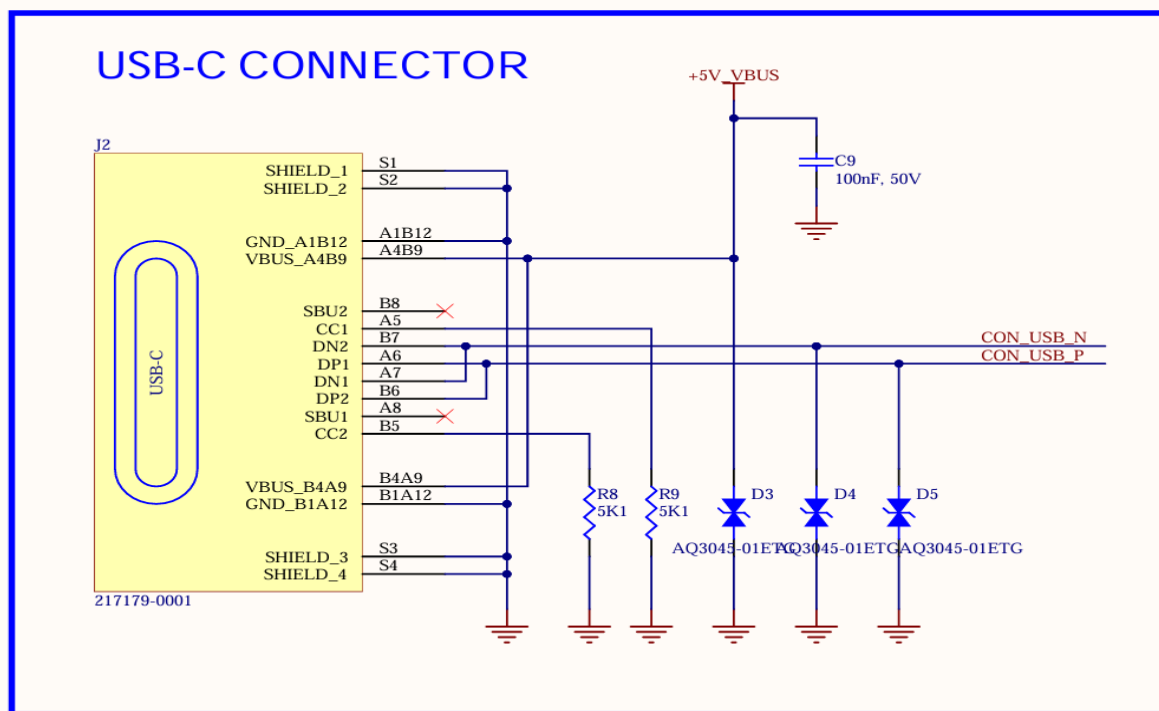


Figura 4.15 - Blocul de interfață USB-C

Blocul de interfață USB-C, ilustrat în Figura 4.15, reprezintă poarta principală de comunicație și alimentare între placa PCB și dispozitivele externe, precum un PC. Acesta are rolul de a prelua semnalele USB și tensiunea VBUS de la conectorul USB-C și de a le transmite către restul circuitelor, protejând în același timp liniile sensibile.

Conectorul utilizat este J2 (217179-0001), un model USB-C standard, care permite conectarea cablurilor USB tip C reversibile. Pinii de date diferențiale D+ (A6) și D- (A7) sunt conectați către

ieșirile CON_USB_N și CON_USB_P, ce duc mai departe semnalul către convertorul USB–UART (FT231XQ-R) din blocul următor.

Pentru protecția liniilor de date, schema integrează trei diode TVS AQ3045-01ETG (D3, D4, D5), conectate la masă. Aceste diode protejează liniile diferențiale USB împotriva descărcărilor electrostatice și supratensiunilor tranzitorii, acționând ca o barieră de protecție împotriva deteriorării circuitelor downstream.

Liniile de date sunt trase prin două rezistențe de 27Ω (R5, R6), montate în serie cu D+ și D-, având rol de potrivire de impedanță și reducere a reflexiilor semnalelor la viteze mari de transfer.

Pe linia de alimentare +5V_VBUS, tensiunea de alimentare provenită din portul USB este filtrată de un condensator ceramic C9 (100nF, 50V), care elimină zgomotele de înaltă frecvență, stabilizând semnalul de alimentare.

Schema include și liniile de control CC1 și CC2, folosite intern de conectorul USB-C pentru detectarea direcției de conectare și negocierea curentului de încărcare, care sunt conectate la masă prin rezistențe interne din conector.

4.3.7 Blocul de conversie USB–UART

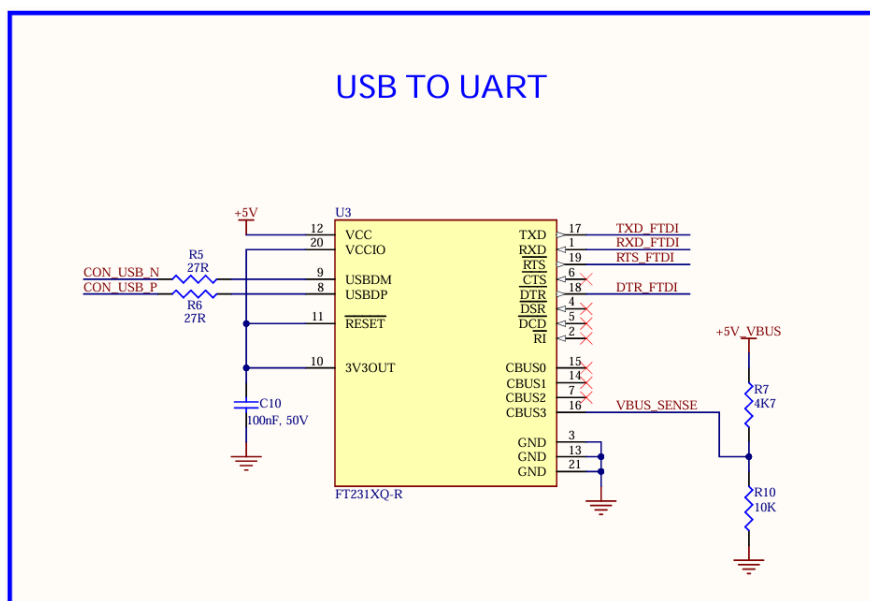


Figura 4.16 - Blocul de conversie USB–UART

Blocul de conversie USB–UART, prezentat în Figura 4.16, are rolul esențial de a transforma semnalele USB primite prin conectorul USB-C în semnale UART compatibile cu microcontrolerul ESP32. Acest bloc facilitează astfel comunicarea serială între placa PCB și un computer sau alt dispozitiv host USB, fiind esențial pentru programare și depanare.

Componenta centrală a acestui bloc este integratul FT231XQ-R (U3), un convertor USB–UART cu montaj pe suprafață, configurat în schema de bază recomandată de producător. Alimentarea acestuia se face cu +5V, aplicată pe pinul VCC (12) și pe pinul VCCIO (20), asigurând compatibilitatea logică la nivel de semnal.

Pe partea de intrare USB, semnalele diferențiale CON_USB_N și CON_USB_P sosesc de la blocul USB-C anterior, fiind trase prin rezistențe de 27Ω (R5, R6), care oferă potrivirea de impedanță

necesară pentru comunicațiile USB. Semnalele sunt conectate la pinii USBDM (9) și USBDP (8) ai FT231XQ-R. Un condensator ceramic C10 (100nF, 50V) este conectat între pinul 3V3OUT (10) și masă, stabilizând tensiunea de ieșire internă a regulatorului încorporat al circuitului.

La ieșirea blocului sunt generate semnalele UART:

- TXD_FTDI (17): transmisia de date de la FTDI către microcontroler;
- RXD_FTDI (1): recepția de date de la microcontroler către FTDI;
- RTS_FTDI (19) și DTR_FTDI (18): semnale de control pentru gestionarea resetării și intrării în modul boot al ESP32.

Linia de detectare a alimentării USB, VBUS_SENSE (16), este conectată la un divizor de tensiune realizat din rezistențele R7 (4.7kΩ) și R10 (10kΩ), permițând integratului FT231XQ-R să detecteze prezența tensiunii VBUS și să activeze comunicația doar când placa este conectată la un dispozitiv host USB.

Pinul RESET (11) rămâne neutilizat, fiind lăsat flotant conform specificațiilor, iar liniile de semnal de control suplimentare (CTS, DSR, DCD, RI, CBUS0–CBUS3) sunt de asemenea neconectate în această aplicație, pentru simplificare.

4.3.8 Blocul de control al semnalelor seriale (Serial Signals Handling)

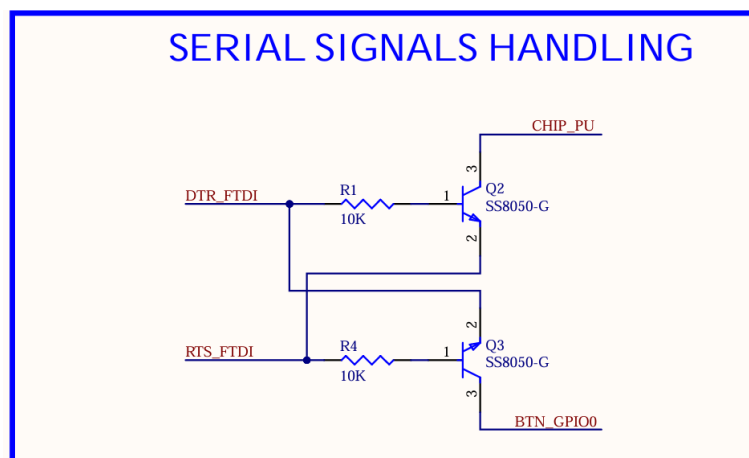


Figura 4.17 - Blocul de control al semnalelor seriale

Blocul de control al semnalelor seriale, prezentat în Figura 4.17, are rolul de a automatiza procesul de inițializare a microcontrolerului ESP32, în special în timpul încărcării firmware-ului prin USB. Acest bloc permite resetarea și intrarea în modul de boot fără intervenție manuală, utilizând semnalele de control provenite de la convertorul USB–UART.

Semnalele DTR_FTDI și RTS_FTDI, generate de integratul FT231XQ-R, sunt utilizate pentru a controla doi tranzistori NPN de tip SS8050-G (Q2 și Q3). Acești tranzistori sunt configurați ca întrerupătoare comandate electronic, iar bazele lor sunt conectate la semnalele FTDI prin intermediul rezistențelor R1 și R4, ambele având valoarea de 10kΩ.

- Tranzistorul Q2 controlează linia CHIP_PU, responsabilă cu resetarea generală a microcontrolerului ESP32. Când semnalul DTR_FTDI este activ (la nivel logic scăzut), tranzistorul Q2 se deschide, trăgând linia CHIP_PU la masă și declanșând o resetare a microcontrolerului.

- Tranzistorul Q3 acționează asupra liniei BTN_GPIO0, conectată la pinul GPIO0 al ESP32. Activarea semnalului RTS_FTDI determină deschiderea lui Q3, care trage pinul GPIO0 la masă, condiție necesară pentru ca ESP32 să intre în modul de boot (UART download mode).

Această combinație de semnale și comenzi permite un control complet software asupra procesului de programare. Placa devine astfel capabilă să intre automat în modul de upload firmware și să revină apoi în modul normal de execuție, fără a necesita apăsarea manuală a butoanelor RESET și BOOT.

4.3.9 Blocul microcontrolerului ESP32-S3-MINI

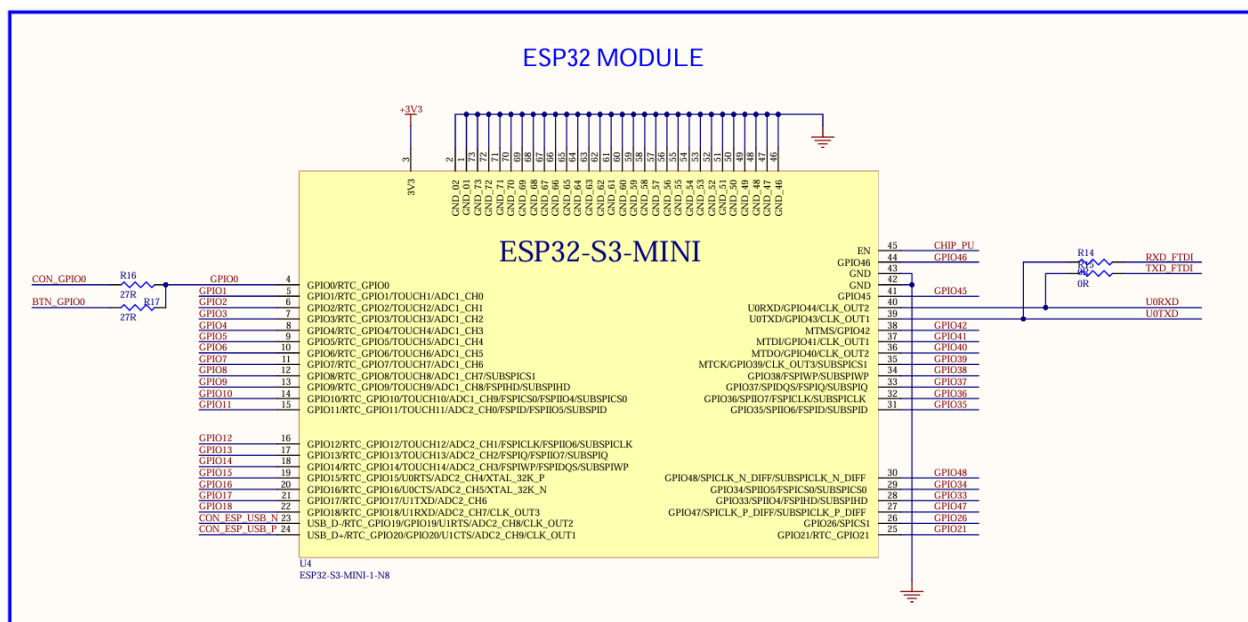


Figura 4.18 - Blocul microcontrolerului ESP32-S3-MINI

Blocul central al plăcii, prezentat în Figura 4.18, este dedicat microcontrolerului ESP32-S3-MINI-1-N8, componenta principală responsabilă de procesarea și gestionarea tuturor funcțiilor sistemului. Acest modul integrează nucleul de procesare, memoria și perifericele necesare aplicației de monitorizare automată.

Microcontrolerul este alimentat cu +3,3V aplicat pe pinul 3V3, provenit de la stabilizatorul anterior. Pinii de masă sunt conectați direct la planul de masă al plăcii pentru reducerea zgomotului electric și asigurarea unei referințe stabile.

Pentru comunicarea serială, pinii U0RXD (39) și U0TXD (40) sunt conectați la semnalele RXD_FTDI și TXD_FTDI provenite din convertorul USB-UART. Între TXD_FTDI și U0RXD a fost inserată o rezistență de 0Ω (R14), care permite eventuale modificări sau deconectări rapide la nivel de traseu.

Pinul CHIP_PU (45) controlează pornirea microcontrolerului, fiind conectat la blocul de control serial (prin tranzistorul Q2), astfel încât să poată fi resetat automat în timpul programării.

Linia GPIO0 (4) este conectată atât la butonul de boot BTN_GPIO0 (prin rezistență R17, 27Ω), cât și la conectorul extern CON_GPIO0 (prin rezistență R16, 0Ω), oferind utilizatorului flexibilitate în activarea modului de programare sau în utilizarea pinului ca I/O general.

Restul pinilor GPIO sunt expuși prin conectori suplimentari (CON_ESP_USB_N, CON_ESP_USB_P) sau utilizați intern, în funcție de necesitățile aplicației. Fiecare pin este etichetat conform funcțiilor multiple disponibile pe ESP32, inclusiv ADC, DAC, SPI, I2C sau UART suplimentare.

Schema include și conectarea pinilor critici EN (45) și GPIO46 (44) la semnale de control și la masa sistemului, asigurând funcționarea corectă la pornire și inițializare.

4.3.10 Blocul conectorilor de extensie (Headers)

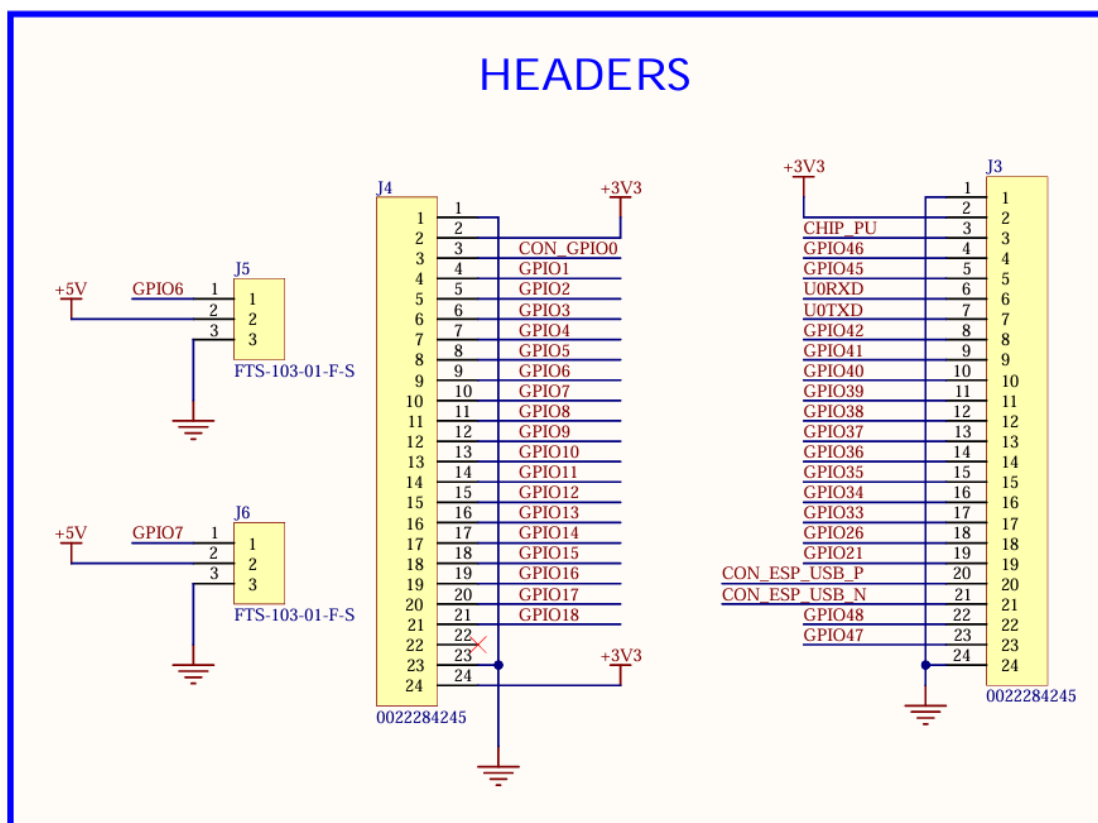


Figura 4.19 – Blocul conectorilor de extensie

Blocul conectorilor externi, ilustrat în Figura 4.19, oferă acces direct la liniile GPIO și la semnalele critice ale microcontrolerului, permițând astfel extinderea funcționalității plăcii PCB prin conectarea unor module periferice suplimentare, senzori sau actuatori.

Schema include trei conectori principali:

- J4 – un header de 24 de pini, care expune majoritatea pinilor GPIO ai microcontrolerului;
- J3 – un al doilea header de 24 de pini, destinat semnalelor speciale, inclusiv liniile de comunicație serială, semnalele de boot și reset, precum și GPIO suplimentare;
- J5 și J6 – doi conectori cu 3 pini fiecare, dedicați semnalelor GPIO6 și GPIO7, fiecare furnizând și o linie de +5V și masă, utilă pentru conectarea unor dispozitive cu consum redus.

Conectorul J4 este conceput ca o interfață generală de extensie, punând la dispoziție pinii GPIO0 – GPIO18, precum și semnalul CON_GPIO0 și o linie de alimentare +3,3V. Acest header

permite accesul rapid la funcțiile principale de intrare/ieșire digitală ale ESP32, inclusiv canale ADC, PWM, I2C sau SPI, în funcție de configurarea firmware-ului.

Conectorul J3 oferă acces la semnalele de control esențiale ale microcontrolerului, precum CHIP_PU (reset), GPIO46, GPIO45, liniile de comunicație U0RXD/U0TXD, dar și GPIO suplimentare (GPIO42 – GPIO48). În plus, acest header expune liniile diferențiale CON_ESP_USB_P și CON_ESP_USB_N, utilizabile pentru comunicații USB suplimentare sau debug.

Conectorii J5 și J6 sunt prevăzuți cu câte un pin de semnal (GPIO6, respectiv GPIO7), o linie de alimentare +5V și o linie de masă, fiind gândiți pentru conectarea rapidă a unor module simple, cum ar fi senzori digitali sau drivere de motoare care necesită alimentare directă din sursa de 5V.

4.3.11 Blocul butoanelor de control manual (RESET și BOOT)

Placa PCB integrează două butoane dedicate controlului manual al microcontrolerului ESP32: unul pentru resetare (RESET) și unul pentru activarea modului de programare (BOOT). Aceste circuite sunt ilustrate în Figura 4.3.9 (buton RESET) și Figura 4.3.10 (buton BOOT), având un rol esențial în procesul de inițializare și depanare.

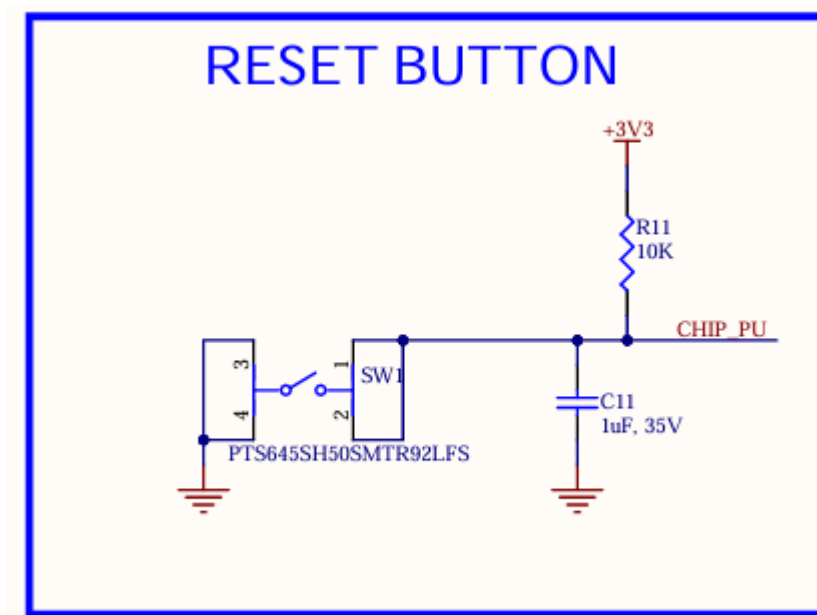


Figura 4.20 - buton RESET

Circuitul pentru resetare, prezentat în Figura 4.20, utilizează un buton tactil SW1 (PTS645SH50SMTR92LFS), conectat între pinul CHIP_PU al ESP32 și masă. În stare normală, linia CHIP_PU este menținută la nivel logic ridicat prin rezistența de 10kΩ (R11) legată la +3,3V. Un condensator de 1μF (C11) este montat între linie și masă pentru a filtra zgomotul și a preveni declanșările false.

La apăsarea butonului, linia CHIP_PU este trasă temporar la masă, provocând resetarea microcontrolerului. După eliberarea butonului, rezistența R11 readuce linia la +3,3V, permițând repornirea normală a ESP32. Acest mecanism oferă utilizatorului posibilitatea de a efectua manual o resetare hardware a microcontrolerului, independent de comanda serială.

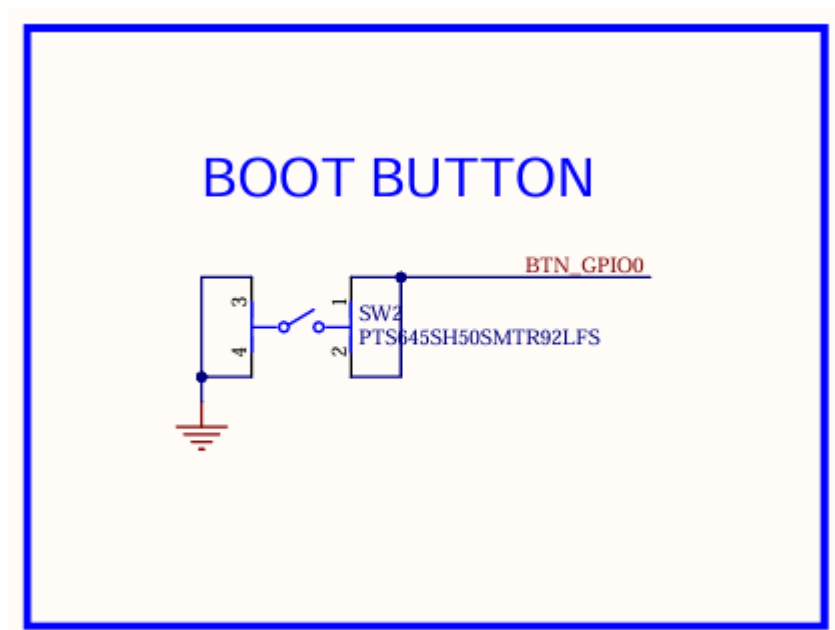


Figura 4.21- buton BOOT

Butonul de boot, ilustrat în Figura 4.21, folosește un comutator identic SW3 (PTS645SH50SMTR92LFS), conectat între linia BTN_GPIO0 și masă. Prin apăsarea acestui buton, pinul GPIO0 al ESP32 este tras la nivel logic scăzut, condiție necesară pentru intrarea în modul de descărcare UART (programare firmware).

Atunci când butonul este apăsat simultan cu resetarea, microcontrolerul intră în modul de bootloader, permițând încărcarea firmware-ului prin portul USB–UART. În lipsa apăsării, pinul GPIO0 rămâne ridicat printr-o rezistență de pull-up externă, iar microcontrolerul pornește normal în aplicația instalată.

4.3.12 Blocul LED-urilor de stare (Power și User)

Placa PCB integrează două LED-uri de stare, cu funcții distincte, ce oferă indicii vizuale privind starea de funcționare a sistemului. Acestea sunt prezentate în Figura 4.22 (LED-ul Power) și Figura 4.23 (LED-ul User).

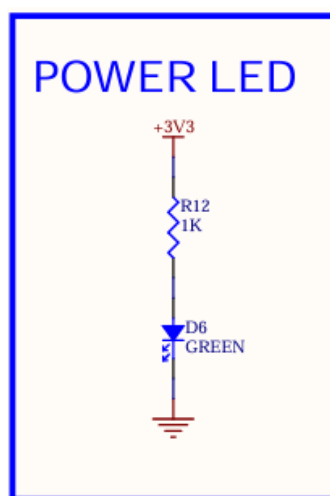


Figura 4.22 - LED-ul Power

Circuitul LED-ului de alimentare, ilustrat în Figura 4.22, este alcătuit dintr-un LED verde D6, conectat în serie cu o rezistență de limitare a curentului R12 (1k Ω). Ansamblul este alimentat direct din linia +3,3V, iar catodul LED-ului este conectat la masă.

Scopul principal al acestui LED este de a semnala prezența tensiunii de alimentare pe placa PCB. În momentul în care placa este alimentată corespunzător, LED-ul se aprinde, oferind utilizatorului o confirmare vizuală că stabilizatorul de 3,3V funcționează corect și că sistemul este activ.

Valoarea rezistenței R12 a fost aleasă pentru a limita curentul prin LED la un nivel sigur și suficient pentru o vizibilitate adecvată, prevenind în același timp suprasolicitarea sau deteriorarea LED-ului.

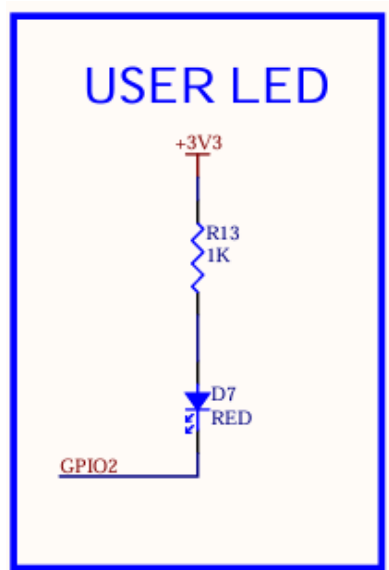


Figura 4.23 - LED-ul User

Circuitul LED-ului de utilizator, prezentat în Figura 4.23, include un LED roșu D7, conectat în serie cu o rezistență de 1k Ω (R13). Spre deosebire de LED-ul Power, anodul acestuia este alimentat din +3,3V, iar catodul este conectat la pinul GPIO2 al microcontrolerului ESP32.

Acest LED este destinat utilizării programabile, permițând aprinderea și stingerea sa din software, prin comenzi trimise către pinul GPIO2. Astfel, dezvoltatorul poate folosi LED-ul User pentru semnalizarea unor evenimente interne, cum ar fi stări de eroare, finalizarea unei sarcini sau indicarea modului activ al aplicației.

În stare inițială, pinul GPIO2 trebuie configurat ca ieșire digitală de tip open-drain sau ca ieșire standard, iar nivelul său logic controlează aprinderea LED-ului. Rezistența R13 are același rol ca în cazul LED-ului Power: limitează curentul prin LED pentru a proteja atât componenta, cât și microcontrolerul.

4.4 Proiectarea, realizarea și fabricarea plăcii de circuit imprimat dedicată modului hardware

4.4.1 Obiectivele proiectării plăcii PCB

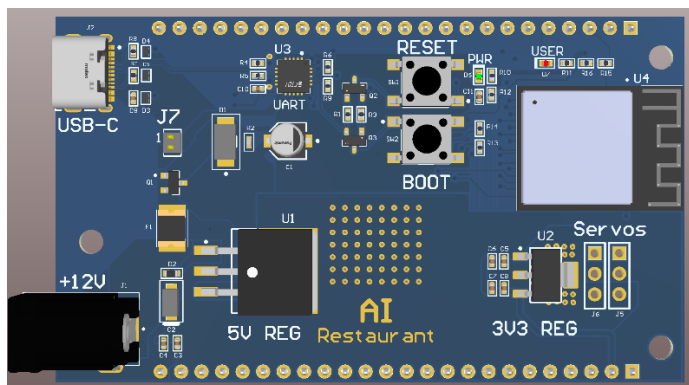


Figura 4.24 - Randare 3D top view PCB

Proiectarea plăcii de circuit imprimat a avut ca obiectiv principal integrarea într-o singură platformă hardware a tuturor funcționalităților necesare sistemului de monitorizare bazat pe inteligență artificială. Această placă a fost concepută pentru a asigura alimentarea corespunzătoare a modulelor electronice, conectivitatea necesară comunicației de date și controlul precis al motorului servomecanic utilizat pentru orientarea camerei.

Un alt obiectiv esențial al proiectării a fost optimizarea traseelor și a stratificării, astfel încât să se minimizeze interferențele electromagnetice și să se asigure stabilitatea semnalelor, mai ales în condițiile unei alimentări mixte la 12V, 5V și 3.3V. În plus, s-a urmărit implementarea unui plan de masă generos, pentru a reduce zgomotul electric și pentru a asigura un potențial de referință uniform pe întreaga suprafață a plăcii.

Placa a fost proiectată pentru a integra un microcontroler ESP32-S3-MINI ca element central, un port de comunicație USB-C cu convertor USB-Serial, circuite de reglare a tensiunii la 5V și 3.3V, butoane de control (RESET și BOOT), LED-uri de stare și conectori dedicați controlului servomotoarelor.

Pe lângă integrarea funcțională, un obiectiv suplimentar a constat în respectarea unui factor de formă redus, astfel încât placa să fie compatibilă cu dimensiunile sistemului mecanic de suport al camerei. În acest sens, s-au utilizat dimensiuni de 3064 mil $\times$ 1780 mil, optimizate pentru a încadra toate componentele necesare și a menține accesibilitatea interfețelor de conectare.

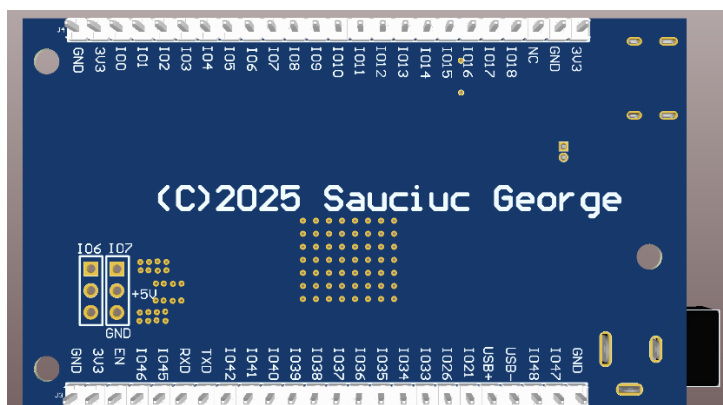


Figura 4.25 - Randare 3D bottom view

4.4.2 Specificații tehnice și parametri electrici

Placa de circuit imprimat realizată în cadrul acestui proiect prezintă o serie de caracteristici tehnice atent definite, astfel încât să răspundă cerințelor funcționale și structurale impuse de sistemul hardware. Dimensiunile acesteia sunt de 3064 mil $\times$ 1780 mil, rezultând o placă compactă, dar suficient de încăpătoare pentru a găzdui toate componentele necesare. Conform fișei tehnice generate în mediul de proiectare, pe placă au fost montate 51 de componente electronice, dintre care unele în tehnologie SMD, iar altele în montaj THT, în funcție de specificațiile fiecărui element.

Placa este alcătuită din patru straturi conductive, fiecare având un rol bine stabilit în schema electrică și topologia circuitului. Structura stratificată va fi detaliată în subcapitolul următor. În ceea ce privește rutarea, aceasta a fost complet realizată, obținându-se o rată de 100% a conexiunilor rutate, respectiv 188 de conexiuni finalizate, fără legături rămase neconectate.

Analiza traseelor arată o distribuție variată a lățimilor, în funcție de curentul nominal și de funcția fiecărei linii. Cele mai frecvent utilizate lățimi de traseu au fost de 6 mil (pentru semnale de date și GPIO), 10 mil și 20 mil (pentru trasee de putere secundară), respectiv 30 mil și 50 mil pentru traseele principale de alimentare la +12V și +5V, după cum se evidențiază în Anexa 2, care ilustrează Layer 1 al plăcii. Această selecție de lățimi a fost realizată pentru a respecta criteriile de siguranță electrică și pentru a limita căderile de tensiune pe traseele de curent ridicat.

În ceea ce privește distribuția cuprului pe fiecare strat, valorile obținute din analiza layout-ului indică o acoperire de 62% pe Layer 1, 84% pe Layer 2, 89% pe Layer 3, respectiv 26% pe Layer 4. Această distribuție reflectă strategia utilizată în proiectare, unde Layer 3, reprezentând planul de masă, deține cea mai mare suprafață de cupru continuu, așa cum se observă în Anexa 2, contribuind astfel la stabilitatea electrică generală a plăcii și la reducerea zgomotului electromagnetic.

Placa conține 188 de via-uri între straturi, asigurând interconectarea semnalelor verticale între traseele amplasate pe diferite nivele. Aceste via-uri au un diametru standard de 12 mil, corespunzând parametrilor de fabricație acceptați de furnizorul JLCPCB. De asemenea, s-au utilizat 267 de pad-uri pentru componente, incluzând atât pad-uri pentru montaj superficial, cât și pad-uri THT, evidențiate în Figura 4.24 și Figura 4.25, care prezintă randările 3D ale plăcii pe partea superioară și inferioară.

Mai mult, placa integrează 4 sloturi placate și 3 găuri pătrate placate, acestea fiind incluse pentru fixarea mecanică și pentru compatibilitatea cu ansamblul mecanic de prindere. Din punct de vedere al densității traseelor, pe întregul circuit au fost trasate 1582 de piste electrice, cu lățimi variate, iar numărul total de texte și inscripții serigrafice se ridică la 177, incluzând marcaje de identificare a componentelor și simboluri funcționale.

Aceste specificații tehnice confirmă complexitatea moderată a plăcii și demonstrează respectarea standardelor de proiectare pentru un sistem embedded cu multiple surse de alimentare și funcționalități integrate.

4.4.3 Descrierea stratificată a plăcii (stack-up)

Placa de circuit imprimat concepută pentru acest proiect este construită pe o arhitectură cu patru straturi conductive, fiecare având o funcție bine definită în distribuția traseelor electrice. Alegerea unei structuri multilayer a fost determinată de nevoia de a separa logic și fizic traseele de alimentare, semnalele de mare viteză și conexiunile de control, asigurând o performanță electrică superioară și o stabilitate funcțională optimă.

Primul strat, Layer 1, ilustrat în Anexa 2, este utilizat pentru rutarea liniilor principale de alimentare la +12V și +5V, precum și a semnalelor de comunicație USB. Traseele diferențiale dintre portul USB-C și convertorul FT231XQ-R au fost plasate strategic pentru a respecta impedanțele controlate, fiind rutate paralel și echidistant. Lățimile traseelor variază între 6 mil și 50 mil, în funcție de sarcina electrică transportată.

Layer 2, de culoare galbenă în reprezentările din Anexa 2, a fost rezervat rutărilor suplimentare, inclusiv traseelor alimentate la +3,3V, și semnalelor auxiliare. Acest strat a completat rutările neacoperite de Layer 1, menținând o distanțare optimă față de planul de masă.

Layer 3, colorat în verde, constituie planul complet de masă al plăcii și oferă o referință electrică comună pentru toate semnalele. Având un grad de acoperire de 89%, acest strat contribuie semnificativ la reducerea zgomotului electromagnetic, fapt evidențiat în imaginile din Anexa 2.

Stratul inferior, Layer 4, reprezentat în albastru, găzduiește traseele GPIO ale microcontrolerului și semnalele de comandă către periferice. Distribuirea acestor trasee în partea inferioară a permis degajarea zonelor superioare pentru alimentare și comunicație. Valorile detaliate de acoperire și trasee sunt analizate și în Anexa 3, prin fișierele Gerber corespunzătoare fiecărui strat.

4.4.4. Straturile auxiliare ale plăcii de circuit imprimat

Pe lângă cele patru straturi conductive, designul plăcii include o serie de straturi auxiliare care nu transportă curent, dar au un rol esențial în procesul de fabricație și montaj. Acestea sunt evidențiate în Anexa 2, unde pot fi observate individual în vizualizările CAD, și detaliate tehnic în Anexa 3, prin fișierele Gerber exportate.

Conturul mecanic al plăcii (Board Shape), desenat cu violet, definește limitele fizice ale plăcii și locațiile sloturilor mecanice. Acesta servește ca ghid pentru procesul de frezare industrială și asigură compatibilitatea cu sistemul mecanic de susținere al camerei. În aceeași anexă se regăsesc și straturile de serigrafie (Legend), atât pentru fața superioară cât și pentru cea inferioară, în care sunt marcate pozițiile componentelor și identificatorii utili în etapa de montaj.

Straturile de pastă de lipit (Paste) sunt dedicate procesului industrial SMD și definesc zonele unde va fi aplicată pasta de lipit prin șablon. Deși în cadrul acestui proiect montajul a fost realizat manual, aceste straturi asigură compatibilitate cu metodele de producție automatizate și pot fi consultate în Anexa 3.

Un alt strat important este masca de lipire (Soldermask), aplicată pentru protejarea cuprului expus și pentru a preveni scurtcircuiturile accidentale. Acesta lasă libere doar pad-urile și via-urile, și este vizibil în reprezentările din Anexa 2, unde se observă și un element decorativ – inscripționarea textului „AI Restaurant”.

Fișierele de foraj (Drill files) definesc locațiile găurilor metalizate și nemetalizate, fiind indispensabile pentru montajul componentelor THT și al conectorilor. Ele sunt incluse în Anexa 3, unde fiecare tip de strat auxiliar este prezentat separat și complet.

4.4.5 Procesul de proiectare CAD și verificare

Proiectarea fizică a plăcii a fost realizată integral în mediul Altium Designer, folosind facilitățile avansate ale acestuia pentru schematizare, rutare și verificare. Procesul a debutat cu definirea conexiunilor funcționale în schema electrică, incluzând microcontrolerul ESP32-S3-MINI, portul USB-C, convertorul USB-UART, stabilizatoarele de tensiune și conectorii de extensie.

După finalizarea schematicii, componentele au fost plasate pe layout în funcție de criterii electrice și mecanice. Amplasarea convertorului USB în imediata vecinătate a conectorului USB-C și apropierea reguletoarelor față de consumatorii locali sunt aspecte care pot fi observate în randările 3D din capitolele anterioare și corelate cu Anexa 2, unde Layer 1 și Layer 2 evidențiază traseele principale.

Rutarea a fost realizată cu respectarea cerințelor de impedanță, lățime minimă și distanțare între trasee. Traseele diferențiale pentru USB sunt rutate paralel, iar liniile de alimentare utilizează lățimi între 20 și 50 mil. Distribuția cuprului între straturi indică o acoperire majoritară pe Layer 3, consolidând planul de masă.

Procesul de validare a inclus verificări automate de tip Design Rule Check (DRC), pentru a asigura respectarea regulilor de fabricație. Ulterior, au fost generate fișierele de producție, inclusiv cele Gerber, ilustrate în Anexa 3. Acestea includ toate straturile necesare pentru fabricație, de la cele conductive până la cele auxiliare, oferind o imagine completă asupra designului final.

4.4.6 Fabricarea și asamblarea plăcii

După încheierea procesului de proiectare în Altium Designer și generarea completă a fișierelor necesare, fabricarea fizică a plăcii de circuit imprimat a fost realizată prin serviciul furnizorului JLCPCB. Fișierele Gerber exportate au inclus toate straturile relevante pentru producție – conductoare, mască de lipire, serigrafie, foraj și contur mecanic – respectând specificațiile tehnice al procesului industrial.



Figura 4.26 – PCB livrat de JLCPCB, înainte de asamblare

Alegerea JLCPCB a fost motivată de fiabilitatea sa, de suportul pentru plăci multi-strat și de rapiditatea execuției. Datele transmise au inclus dimensiunea exactă a plăcii (3064 mil × 1780 mil), arhitectura cu patru straturi conductive, grosimea standard a cuprului, precum și specificațiile de finisare pentru pad-uri. Placa livrată s-a remarcat printr-o execuție precisă a găurilor și sloturilor mecanice, calitate ridicată a serigrafiei și aliniere perfectă a straturilor, conform observațiilor din Figura 4.26.

Ulterior, asamblarea componentelor electronice a fost realizată manual în laborator, începând cu montarea celor în tehnologie SMD (Surface-Mount Device). Întregul proces a fost documentat fotografic în Anexa 4 și a cuprins următoarele etape:

1. Poziționarea plăcii pe un suport fix, pentru a preveni deplasările accidentale în timpul aplicării pastei și montajului (Anexa 4, figura 1);
2. Alinierea și fixarea stencilului metalic, realizat din oțel inoxidabil și comandat de la JLCPCB, utilizat pentru aplicarea precisă a pastei de lipit (Anexa 4, figura 2);
3. Aplicarea pastei de fludor, distribuită uniform cu o racletă peste stencil, pentru a acoperi doar pad-urile expuse (Anexa 4, figura 3);
4. Verificarea vizuală a pastei aplicate, pentru a confirma uniformitatea și absența scurgerilor nedorite;
5. Plasarea componentelor SMD, cu ajutorul unei unelte de tip pick-and-place manual, pentru a asigura alinierea corectă pe pad-uri (Anexa 4, figura 4);
6. Introducerea plăcii în cuptorul reflow, unde a fost supusă unui profil termic de aproximativ 3 minute, activând procesul de lipire uniformă prin topirea controlată a pastei (Anexa 4, figura 5);
7. Inspecția finală sub microscop, pentru verificarea calității lipiturilor și alinierea precisă a componentelor (Anexa 4, figura 6).

După finalizarea montajului SMD, au fost asamblate componentele în tehnologie through-hole (THT), prin inserarea și lipirea manuală a acestora:

- Mufa de alimentare de tip jack, destinată alimentării externe a sistemului;
- Pinii de prindere ai portului USB-C;
- Două rânduri de pini male (2×3), folosiți pentru conectarea a două servomotoare;
- Cele două butoane tactile, pentru reset și comandă locală.

Lipirea acestor componente THT a fost efectuată cu letcon reglabil, utilizând sârmă de fludor cu flux integrat, asigurând o fixare mecanică și electrică durabilă.



Figura 4.27 - PCB complet asamblat, cu componente SMD și THT

Întregul proces de asamblare a fost realizat cu atenție și precizie, conducând la obținerea unei plăci complet funcționale, pregătită pentru integrarea în sistemul AI de monitorizare. Rezultatul final

este redat în Figura 4.27, unde poate fi observat ansamblul complet al plăcii, cu toate componentele montate.

4.4.7 Testarea plăcii și verificarea funcționalității

După finalizarea procesului de asamblare, placa de circuit imprimat a fost supusă unei serii de teste funcționale și electrice, menite să confirme integritatea conexiunilor, corectitudinea rutării și funcționalitatea componentelor integrate. Testarea a debutat cu o verificare de continuitate a traseelor electrice, utilizând un multimetru digital setat pe modul de detecție a rezistenței. Această etapă a avut ca scop identificarea eventualelor întreruperi sau scurtcircuite apărute accidental în procesul de lipire.

Ulterior, s-a realizat alimentarea treptată a plăcii, inițial aplicând tensiune doar pe liniile de +5V și +3.3V, fără a conecta microcontrolerul la sursa principală, pentru a verifica stabilitatea și corectitudinea tensiunilor de ieșire ale reguletoarelor. Valorile măsurate au confirmat funcționarea adecvată a circuitelor de reglare, în limitele specificate de producătorii acestora.

Un test important a vizat funcționalitatea portului USB-C și a convertorului FT231XQ-R, prin conectarea plăcii la un calculator și verificarea apariției portului serial virtual în managerul de dispozitive al sistemului de operare. Această verificare a demonstrat corectitudinea traseelor diferențiale USB și funcționarea completă a liniei de comunicație.

Funcționarea butoanelor RESET și BOOT a fost testată prin observarea răspunsului microcontrolerului ESP32-S3-MINI la apăsarea acestora, confirmând intrarea corectă în modurile de programare sau resetare. În paralel, LED-urile de semnalizare au fost verificate prin comenzi software trimise către liniile GPIO, iar aprinderea acestora a confirmat atât funcționalitatea componentelor, cât și a traseelor aferente, vizibile pe Layer 4 din Anexa 2.

Un test esențial în validarea funcțională a plăcii PCB a constat în evaluarea ieșirilor dedicate controlului servomotoarelor. Acestea au fost conectate la un servomotor MG996R real, iar răspunsul acestuia a fost analizat în raport cu semnalele PWM generate de microcontrolerul ESP32-S3-MINI, prin intermediul bibliotecii software ESP32Servo. Comportamentul corespunzător al servomotorului a confirmat nu doar acuratețea generării semnalelor de control, ci și calitatea rutării traseelor pe PCB și fiabilitatea interconectărilor dintre microcontroler și conectorii de extensie.

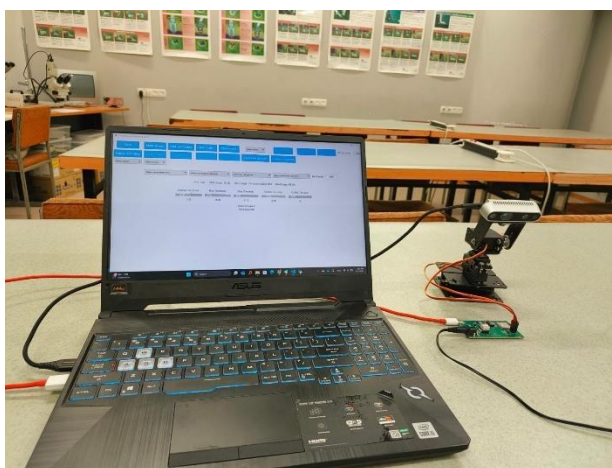


Figura 4.28 – Testul final al plăcii în configurația completă a sistemului

La final, placa a fost supusă unui test integrat, conectând toate modulele hardware în configurația finală a sistemului de monitorizare. Acest test a permis evaluarea funcționării simultane

a tuturor componentelor, inclusiv alimentarea multi-tensiune, comunicația USB, controlul servomotorului și funcționarea LED-urilor de semnalizare, după cum se poate observa în Figura 4.28.

Rezultatele testării au confirmat că placa respectă specificațiile proiectului, oferind stabilitate, funcționalitate completă și compatibilitate cu ansamblul mecanic al sistemului. Nu au fost identificate defecțiuni majore, iar eventualele corecții minore au fost realizate prin retribaj manual, utilizând letconul pentru îmbunătățirea unor îmbinări de lipire.

4.4.8 Concluzii asupra procesului de realizare

Procesul de proiectare, fabricare și asamblare a plăcii de circuit imprimat a reprezentat o etapă esențială în realizarea modului hardware al sistemului de monitorizare. Prin aplicarea unui flux riguros de proiectare, utilizând platforma Altium Designer, s-a obținut un design optimizat din punct de vedere electric, mecanic și funcțional, capabil să integreze toate modulele necesare într-o arhitectură compactă și fiabilă.

Fabricarea plăcii la furnizorul JLCPCB a demonstrat compatibilitatea fișierelor generate cu cerințele de producție, iar calitatea execuției a permis asamblarea manuală fără dificultăți majore. Componentele selectate, provenite de la furnizori consacrați precum DigiKey și LCSC, au asigurat fiabilitatea necesară funcționării pe termen lung, iar procesul de montaj, realizat manual în laborator, a permis un control direct al calității fiecărei etape.

Testele funcționale au confirmat că placa respectă toate specificațiile proiectului, incluzând stabilitatea tensiunilor de alimentare, funcționalitatea comunicației USB, răspunsul corect al liniilor GPIO și controlul precis al servomotoarelor. Nu au fost identificate defecțiuni structurale sau electrice majore, iar eventualele mici corecții au putut fi remediate prin intervenții punctuale, fără a afecta funcționarea globală a plăcii.

Capitolul 5: Implementarea software

5.1 Arhitectura generală a aplicației

5.1.1 Organizarea fișierelor și structura modulară a proiectului

Aplicația software dezvoltată în cadrul acestui proiect este structurată pe o arhitectură modulară clar definită, construită pe principiile separării responsabilităților și scalabilității. Proiectul este organizat în directorul principal „AI_restaurant”, care cuprinde directoare esențiale pentru funcționarea completă a sistemului: `data`, `docs`, `src`, `static` și `templates`, alături de fișierul principal `main.py`, care acționează ca punct de pornire al aplicației.

Directorul `src` reprezintă nucleul logic al aplicației, incluzând toate modulele software majore. Acesta este împărțit în următoarele componente:

- **detectors/** – conține fișierul `yolo_detector.py`, responsabil pentru implementarea logicii de detecție AI utilizând modelele YOLO.
- **gui/** – gestionează interfața grafică a aplicației, incluzând subfoldere precum `frames` (pentru componente vizuale reutilizabile), `modes` (pentru modurile de operare: captură live, video, imagine) și fișierele principale `admin_gui.py` și `user_gui.py`.
- **managers/** – administrează logica asociată entităților principale (mese, persoane, alarme) prin fișiere precum `table_manager.py`, `people_manager.py` și `alarm_manager.py`.
- **reports/** – include modulele `automatic_report_generator.py` și `manual_report_generator.py`, dedicate generării automate sau manuale a rapoartelor, integrând informațiile colectate în cadrul aplicației într-un format accesibil și structurat.
- **utils/** – reunește funcții auxiliare precum `correct_illumination.py`, `detection_utils.py`, `time_utils.py` și `excel_utils.py`, care sprijină procesarea imaginilor, gestionarea timpilor și crearea rapoartelor.
- **web/** – oferă o bază pentru extinderea aplicației către o interfață web viitoare.
- **libs.py** – centralizează importurile esențiale utilizate la nivel global în aplicație.

Această organizare modulară permite dezvoltarea, testarea și extinderea eficientă a aplicației, facilitând în același timp înțelegerea clară a fluxurilor interne și a interdependențelor dintre module.

5.1.2 Fluxul funcțional al aplicației

Fluxul logic al aplicației urmează o succesiune bine definită de etape:

1. **Capturarea imaginii** – utilizatorul selectează sursa video (cameră web, fișier video sau imagine). Acest pas este gestionat prin interfața grafică și modulele din `gui/modes`.
2. **Preprocesarea imaginii** – cadrul video este transmis către funcția `display_frame`, unde, dacă este activată, se aplică funcția `correct_illumination` pentru ajustarea iluminării pe baza analizei histogramei și a luminozității. Această etapă sporește precizia detecțiilor în condiții de lumină insuficientă sau excesivă.

3. **Deteția obiectelor** – cadrul preprocesat este introdus în modelul YOLO (v8, v9 sau v10), care returnează o listă cu obiectele detectate, inclusiv persoane (clasa 0), mese (clasa 60) și obiecte roșii (ex. sticle, farfurii).

4. **Filtrare și analiză geometrică** – obiectele detectate sunt procesate prin `filter_detections` pentru a elimina suprapunerile nedorite, iar ulterior sunt analizate prin `table_status.py` pentru determinarea stării fiecărei mese, pe baza clasei, poziției și ariei de suprapunere (IoU).

5. **Afișarea rezultatelor** – imaginea rezultată, cu detecțiile etichetate, este afișată în GUI, împreună cu scorurile de centrare și starea actuală a fiecărei mese (exemplu: „Table 2: eating for 00:03:45”).

6. **Acțiuni hardware și export date** – în cazul în care este detectată o stare critică (de exemplu, timp excesiv la masă, număr prea mare de persoane), se declanșează alarme sonore. Datele sunt salvate în fișiere `.json`, iar programul poate genera rapoarte `.xlsx` utilizând modulele din `reports/`, respectiv `automatic_report_generator.py` sau `manual_report_generator.py`.

7. **Controlul camerei video** – comenzile de mișcare a camerei sunt transmise prin USB-C (protocol UART logic) către microcontrollerul ESP32, care controlează două servomotoare. Această funcționalitate este integrată în interfața `move_camera_frame.py`.

5.1.3 Tehnologii utilizate

Proiectul este implementat în principal în Python 3.12 și integrează următoarele biblioteci și tehnologii:

- **YOLOv8/v9/v10 (Ultralytics)** – utilizate pentru detecția rapidă și precisă a obiectelor, cu modele preantrenate.
- **OpenCV** – pentru manipularea cadrelor video și prelucrarea imaginilor.
- **Tkinter și ttk** – pentru crearea interfeței grafice desktop.
- **NumPy și Pandas** – pentru operații numerice și manipulare de date.
- **Matplotlib** – pentru vizualizarea statisticilor.
- **openpyxl** – pentru crearea și manipularea fișierelor Excel (`.xlsx`).
- **pyserial** – pentru comunicarea UART cu microcontrollerul ESP32.
- **json și datetime** – pentru gestionarea fișierelor de configurare și a marcajelor temporale.

Această combinație de tehnologii asigură:

- compatibilitate cu sisteme de operare moderne,
- flexibilitate pentru extinderi viitoare (exemplu: trecere la aplicație web),
- suport activ din partea comunității,
- performanță ridicată în procesarea imaginilor și generarea rapoartelor.

De asemenea, pentru microcontrollerul ESP32, codul este dezvoltat în C++ utilizând platforma Arduino IDE, iar comunicarea serială este gestionată asincron, permițând interpretarea comenzilor în timp real – aspect detaliat în subcapitolul 5.5.

5.2 Implementarea detecției obiectelor folosind YOLO

5.2.1 Motivația alegerii algoritmului YOLO

În cadrul acestui proiect, algoritmul YOLO (You Only Look Once) a fost ales pentru realizarea detecției obiectelor, datorită vitezei ridicate și acurateței bune, obținute într-un singur pas

de procesare. Acest algoritm aparține clasei de modele de detecție „single-stage”, ceea ce înseamnă că predicțiile sunt generate direct, fără a necesita pași suplimentari de filtrare sau clasificare, facilitând procesarea în timp real. Aceste caracteristici îl fac potrivit chiar și pentru sisteme hardware cu resurse limitate.

Pe lângă rapiditate, YOLO se remarcă prin adaptabilitate. În aplicația realizată, au fost utilizate modele din generațiile YOLOv8, YOLOv9 și YOLOv10, configurate cu greutatea pre-antrenate, astfel încât să recunoască eficient obiectele relevante: persoane, mese și obiecte de culoare roșie, cum ar fi sticle sau farfurii. Alegerea acestui algoritm a permis integrarea rapidă și precisă a unui sistem de recunoaștere a obiectelor fără necesitatea unui antrenament suplimentar pe date personalizate.

5.2.2 Structura detectorului YOLO (fișierul `yolo_detector.py`)

Fișierul `yolo_detector.py`, aflat în directorul `src/detectors`, definește o clasă numită `YoloDetector`. Aceasta gestionează încărcarea modelului YOLO, prelucrarea imaginilor, realizarea predicțiilor și vizualizarea rezultatelor.

Inițializarea clasei implică identificarea automată a resursei hardware disponibile (CPU sau GPU), stabilirea căii către model și apelarea metodei de încărcare a acestuia:

```
class YoloDetector:
    def __init__(self, model_path="models/yolov10l.pt"):
        self.device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")

        self.model_path = model_path
        self.load_model(self.model_path)
```

Modelul YOLO este încărcat în memoria dispozitivului selectat prin metoda `load_model`, utilizând biblioteca Ultralytics:

```
def load_model(self, model_path):
    self.model = YOLO(model_path, verbose=False).to(self.device)
```

5.2.3 Prelucrarea imaginii – `correct_illumination.py`

Pentru a asigura o detecție eficientă chiar și în condiții de iluminare deficitară sau neuniformă, aplicația integrează o funcție de corectare a iluminării, implementată în fișierul `correct_illumination.py`. Această funcție este apelată din `display_frame.py` atunci când utilizatorul activează opțiunea de ajustare automată.

Funcția combină mai multe tehnici de procesare:

- ajustare adaptivă a contrastului prin algoritmul CLAHE;
- reglarea automată a gamma-ului;
- normalizarea nivelurilor de culoare;
- reducerea selectivă a zgomotului.

Fragmentul de cod ilustrează aplicarea acestor operații:

```
clahe = cv2.createCLAHE(clipLimit=clahe_clip,
tileGridSize=(clahe_tile_size, clahe_tile_size))
l_clahe = clahe.apply(l)
inv_gamma = 1.0 / gamma
```

```
frame_gamma = cv2.LUT(frame_clahe, gamma_table)
```

Această etapă s-a dovedit esențială pentru îmbunătățirea calității imaginilor în spații cu lumină slabă sau reflexii puternice, precum cele întâlnite în mediile de tip restaurant.

5.2.4 Fluxul detecției – de la imagine la etichetare

După procesarea inițială, cadrul este convertit într-un tensor și transmis către modelul YOLO pentru inferență:

```
frame_tensor = torch.from_numpy(frame_normalized).to(device).permute(2, 0, 1).unsqueeze(0)
results = self.model(frame_tensor)
```

Rezultatele includ coordonatele casetelor de delimitare, clasa atribuită și nivelul de încredere. Acestea sunt organizate într-o listă de dicționare, fiecare conținând informațiile asociate unui obiect detectat:

```
detections.append({
    'box': (x1.item(), y1.item(), x2.item(), y2.item()),
    'class': cls,
    'confidence': conf.item()
})
```

5.2.5 Filtrarea rezultatelor și clasificarea stării meselor

După obținerea detecțiilor brute, sistemul aplică o filtrare suplimentară prin funcția `filter_detections`, care elimină suprapunerile redundante utilizând o valoare prag (IoU). Ulterior, informațiile procesate sunt transmise către `table_status.py`, unde este calculată starea fiecărei mese în funcție de obiectele prezente și de poziționarea acestora.

Sistemul clasifică mesele în patru stări posibile:

- available
- ready to order
- eating
- need to clean

Determinarea stării se bazează pe numărul obiectelor roșii și pe prezența persoanelor în zona mesei, utilizând o logică simplă:

```
if red_count > 0 and blue_count == 0:
    return "need to clean"
elif blue_count > 0 and red_count == 0:
    return "ready to order"
```

În această abordare, culorile roșu și albastru sunt folosite ca simboluri semantice:

- **Roșu** – indică prezența obiectelor din clasele 39–55 (ex. farfurii, pahare, tacâmuri, sticle), semnalând resturi pe masă.
- **Albastru** – corespunde clasei 0 (persoană), indicând prezența unui client.

Această metodă permite o evaluare automată și intuitivă a statusului meselor, folosind doar datele vizuale extrase.

5.2.6 Integrarea rezultatelor în interfața grafică

Rezultatele obținute sunt afișate în interfața grafică prin funcțiile `draw_detections`, `draw_auto_detections` și `draw_detection_with_table_id`. Aceste funcții adaugă etichete pe imagine, indicând clasa obiectului și durata stării asociate:

```
cv2.putText(frame, label, (int(x1), int(y1) - 5), cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 1)
```

Funcțiile sunt apelate din `display_frame.py`, în funcție de modul activat de utilizator (captură live, fișier video sau imagine).

5.2.7 Evaluarea performanței – viteză și acuratețe

Performanța aplicației a fost evaluată prin măsurarea numărului de cadre procesate pe secundă (FPS) și prin calcularea scorului de centrare a fiecărei mese. FPS-ul este determinat astfel:

```
current_time = time.time()
delta_time = current_time - self.prev_time
if delta_time > 0:
    fps = 1 / delta_time
```

Pe un sistem echipat cu placă grafică NVIDIA, aplicația a atins viteze între 30 și 60 FPS în modul live, menținând totodată o acuratețe ridicată datorită modelelor YOLO utilizate.

5.2.8 Concluzii și beneficii ale implementării

Integrarea algoritmului YOLO în acest proiect a oferit o serie de avantaje esențiale: procesare rapidă în timp real, clasificare automată a statusului meselor, afișare dinamică în interfața grafică și rezistență la variații de iluminare. Sistemul este ușor de extins pentru a recunoaște și alte tipuri de obiecte, dacă va fi necesar în viitor.

Prin utilizarea funcției de corectare a iluminării, alături de analiza geometrică și integrarea vizuală, aplicația asigură o monitorizare eficientă și precisă a meselor într-un mediu de restaurant. Alegerea algoritmului YOLO s-a dovedit ideală pentru cerințele de performanță și fiabilitate ale proiectului.

În capitolul următor va fi analizat în detaliu modul de gestionare a stării meselor, concentrându-se pe algoritmi geometrice implementați.

5.3 Analiza și clasificarea stării meselor prin geometrie computațională

5.3.1 Introducere

După ce obiectele de interes au fost detectate, un pas esențial în procesul aplicației a fost clasificarea automată a stării fiecărei mese. Această clasificare nu s-a bazat exclusiv pe simpla prezență a obiectelor, ci a integrat concepte din geometria computațională, pentru a evalua poziția relativă a obiectelor detectate față de suprafața fiecărei mese. Astfel, analiza fiecărei mese a fost realizată ținând cont de gradul de suprapunere al obiectelor roșii și al persoanelor cu aria mesei, precum și de proporțiile acestora. În continuare sunt prezentate principiile și algoritmi implementați pentru o clasificare precisă și adaptabilă a stării meselor.

5.3.2 Algoritmi pentru determinarea stării mesei pe baza poziției obiectelor

Clasificarea automată a unei mese se bazează pe trei elemente principale obținute prin procesare geometrică: identificarea obiectelor relevante, calculul ariilor și determinarea gradului de suprapunere (IoU). Această logică este implementată în clasa `TableStatus`, din fișierul `table_status.py`, apelată pentru fiecare masă detectată în timpul funcționării aplicației.

```
class TableStatus:
    def check_table_status(self, table_box, detections):
        red_objects, blue_objects = self.filter_relevant_objects(table_box,
detections)
        table_area = calculate_area(table_box)

        red_count = sum(
            self.is_relevant_object(table_box,    obj['box'],    table_area,
self.red_threshold)
            for obj in red_objects
        )
        blue_count = sum(
            self.is_relevant_object(table_box,    obj['box'],    table_area,
self.blue_threshold)
            for obj in blue_objects
        )

        if red_count > 0 and blue_count == 0:
            return "need to clean"
        elif red_count == 0 and blue_count == 0:
            return "available"
        elif blue_count > 0 and red_count == 0:
            return "ready to order"
        elif blue_count > 0 and red_count > 0:
            return "eating"
        return "unknown"
```

5.3.3 Calculul ariilor și al suprapunerii (IoU)

Un aspect esențial al analizei geometrice constă în calcularea ariei fiecărui obiect detectat și a gradului de suprapunere dintre obiect și masa analizată. Aceste operații sunt efectuate prin funcțiile `calculate_area()` și `calculate_iou()`:

```
def calculate_area(box):
    x1, y1, x2, y2 = box
    return max(0, x2 - x1) * max(0, y2 - y1)

def calculate_iou(box1, box2):
    x1, y1, x2, y2 = box1
    x3, y3, x4, y4 = box2

    x11 = max(x1, x3)
    y11 = max(y1, y3)
    x12 = min(x2, x4)
    y12 = min(y2, y4)

    inter_width = max(0, x12 - x11)
    inter_height = max(0, y12 - y11)
    inter_area = inter_width * inter_height

    box1_area = calculate_area(box1)
    box2_area = calculate_area(box2)

    union_area = box1_area + box2_area - inter_area
```

```
return inter_area / union_area if union_area > 0 else 0
```

5.3.4 Praguri de relevanță și adaptabilitate

Utilizatorul are posibilitatea de a regla valorile pragurilor `red_threshold` și `blue_threshold` direct din interfața grafică, pentru a ajusta sensibilitatea algoritmului în stabilirea relevanței obiectelor. Aceste valori sunt transmise din `AdminGUI` către instanțele clasei `TableStatus` prin intermediul `TableManager`.

```
def set_red_threshold_for_all_tables(self, threshold):
    for table in self.tables.values():
        table.set_red_threshold(threshold)
```

Această funcționalitate asigură adaptarea sistemului la condiții variabile de iluminare sau aglomerare.

5.3.5 Rolul culorilor în procesul de clasificare

În logica algoritmului, culoarea obiectelor joacă un rol central. Obiectele considerate roșii (clasele 39–55) sunt asociate cu semnele unei mese utilizate, în timp ce clasa 0 (persoană) indică prezența unui client activ.

```
def filter_relevant_objects(self, table_box, detections):
    red_objects, blue_objects = [], []
    for det in detections:
        class_id = det['class']
        if class_id in range(39, 56):
            if calculate_iou(table_box, det['box']) > 0:
                red_objects.append(det)
        elif class_id == 0:
            if calculate_iou(table_box, det['box']) > 0:
                blue_objects.append(det)
    return red_objects, blue_objects
```

5.3.6 Optimizări pentru funcționare în medii reale

Pentru a preveni schimbările bruște ale stării mesei din cauza fluctuațiilor de detecție, a fost implementat un filtru temporal care înregistrează doar acele stări ce persistă un timp minim de o secundă:

```
def update_status(self, new_status):
    current_time = datetime.now()
    duration = (current_time - self.start_time).total_seconds()
    if duration >= 1.0:
        self.status_durations.append((self.current_status, duration))
        self.previous_status = self.current_status
        self.current_status = new_status
        self.start_time = current_time
```

5.3.7 Funcția clasei `TableStatus` în ansamblul aplicației

Fiecare masă este monitorizată printr-o instanță a clasei `TableStatus`, care implementează integral logica de analiză și clasificare. Instanțele sunt gestionate colectiv de `TableManager`, responsabil pentru actualizarea continuă a stării meselor detectate.

5.4 Integrarea pragurilor temporale și generarea rapoartelor de stare

5.4.1 Introducere

Pentru a completa procesul de monitorizare vizuală și a oferi un suport decizional relevant pentru managementul restaurantului, sistemul dezvoltat integrează un mecanism de înregistrare și analiză temporală a stărilor detectate. Această funcționalitate permite nu doar afișarea în timp real a stării fiecărei mese, ci și elaborarea automată a unor rapoarte statistice zilnice, săptămânale sau lunare, utile pentru îmbunătățirea proceselor operaționale.

5.4.2 Implementarea monitorizării temporale a fiecărei mese

Fiecare obiect `TableStatus` conține o variabilă `start_time` și o listă `status_durations`, în care sunt stocate toate tranzițiile de stare și duratele aferente. La fiecare schimbare de stare validată (care persistă peste un prag de 1 secundă), o nouă înregistrare este adăugată în listă:

```
def update_status(self, new_status):
    current_time = datetime.now()
    duration = (current_time - self.start_time).total_seconds()
    if duration >= 1.0:
        self.status_durations.append((self.current_status, duration))
        self.previous_status = self.current_status
        self.current_status = new_status
        self.start_time = current_time
```

Această abordare garantează că doar tranzițiile semnificative sunt luate în considerare și incluse în analiza finală.

5.4.3 Stocarea înregistrărilor în fișiere JSON

Pe baza acestor înregistrări, clasele `TableManager` și `PeopleManager` creează periodic fișiere JSON care conțin informații despre stările meselor și numărul de persoane detectate. Salvarea acestor fișiere se face automat la intervale prestabilite, utilizând un mecanism bazat pe fire de execuție (threading):

```
def auto_save_thread(self):
    while True:
        if self.auto_save_enabled:
            self.save_to_file()
            time.sleep(self.save_interval)
```

Fiecărei mese îi este atribuit un identificator unic, iar fiecare fișier JSON reflectă istoricul complet al stărilor pentru masa respectivă pe durata sesiunii de rulare.

5.4.4 Analiza statistică și generarea rapoartelor în Excel

Fișierele JSON generate sunt procesate prin clasa `JsonToExcel`, care centralizează datele și le exportă într-un fișier Excel. Această funcționalitate este accesibilă utilizatorului prin butonul „Analyze Records” din interfața AdminGUI. Procesul implică obținerea mediei, valorii maxime și duratei totale pentru fiecare stare a mesei:

```
def save_average_statistics(self):
    for table_id in self.table_stats:
        for status, durations in self.table_stats[table_id].items():
            avg = round(statistics.mean(durations), 2) if durations else 0
```

```
self.json_data[table_id][status]['average_duration'] = avg
```

Prin această analiză, utilizatorul poate identifica mesele care rămân necurățate o perioadă prea lungă, poate evalua timpii de așteptare și poate determina gradul de ocupare în funcție de intervalele orare.

5.4.5 Interfața grafică pentru selecția rapoartelor

Sistemul pune la dispoziția utilizatorului o interfață grafică intuitivă pentru selectarea fișierelor JSON ce urmează a fi analizate. În plus, aplicația creează automat directoare date pentru salvarea rapoartelor, asigurând o organizare logică și cronologică a fișierelor generate:

```
current_date = datetime.now().strftime("%Y-%m-%d")
excel_output_dir = os.path.join("data", "outputs", "daily_report",
current_date)
os.makedirs(excel_output_dir, exist_ok=True)
```

Această automatizare reduce riscul de erori umane și simplifică utilizarea aplicației chiar și de către persoane fără pregătire tehnică.

5.4.6 Utilizarea bibliotecilor Python pentru analiză și generare de rapoarte

Funcționalitatea de analiză și generare a rapoartelor este susținută de o serie de biblioteci Python, alese pentru fiabilitatea și popularitatea lor:

- pandas pentru manipularea și procesarea datelor;
- matplotlib pentru crearea graficelor și vizualizărilor;
- openpyxl pentru manipularea fișierelor Excel;
- json și datetime pentru gestionarea fișierelor de intrare și a marcajelor temporale.

Aceste instrumente oferă o bază solidă și flexibilă, fiind recunoscute pentru compatibilitatea lor cu aplicații de analiză a datelor.

5.4.7 Concluzii privind integrarea analizei temporale

Această componentă adaugă sistemului o dimensiune analitică suplimentară, transformând aplicația dintr-un simplu instrument de monitorizare vizuală într-o platformă completă de suport decizional. Utilizarea fișierelor JSON ca bază de date intermediară, împreună cu generarea rapoartelor Excel, oferă un echilibru optim între flexibilitate, ușurință în utilizare și posibilități de extindere viitoare.

5.5 Interfața grafică și rolul utilizatorului în configurarea sistemului

5.5.1 Introducere

Interfața grafică (GUI) dezvoltată în cadrul proiectului reprezintă legătura principală dintre utilizator și sistemul automatizat de monitorizare. Prin această interfață, utilizatorul beneficiază de control complet asupra funcțiilor de detecție, configurare a camerei, analiză statistică și generare de rapoarte. În acest subcapitol sunt prezentate structura interfeței și funcționalitățile disponibile, atât pentru utilizatorii de tip administrator, cât și pentru utilizatorii obișnuiți.

5.5.2 Structura interfeței grafice

Aplicația oferă două niveluri de interacțiune: Admin GUI și User GUI. Admin GUI oferă acces extins la funcții de configurare, testare și analiză, în timp ce User GUI este limitată la rularea aplicației cu parametrii predefiniți încărcăți anterior.

Structura Admin GUI include mai multe module și componente vizuale, organizate astfel:

- `move_camera_frame.py` – controlează poziția camerei motorizate;
- `schedule_frame.py` – permite programarea intervalelor orare pentru funcționarea automată;
- `calendar_frame.py` – facilitează selectarea perioadelor pentru analiză statistică;
- `test_servos_frame.py` – oferă posibilitatea de testare individuală a servo-motoarelor MG996R;
- `display_frame.py` – afișează în timp real rezultatele detecției YOLO.

5.5.3 Controlul detecției și ajustarea parametrilor

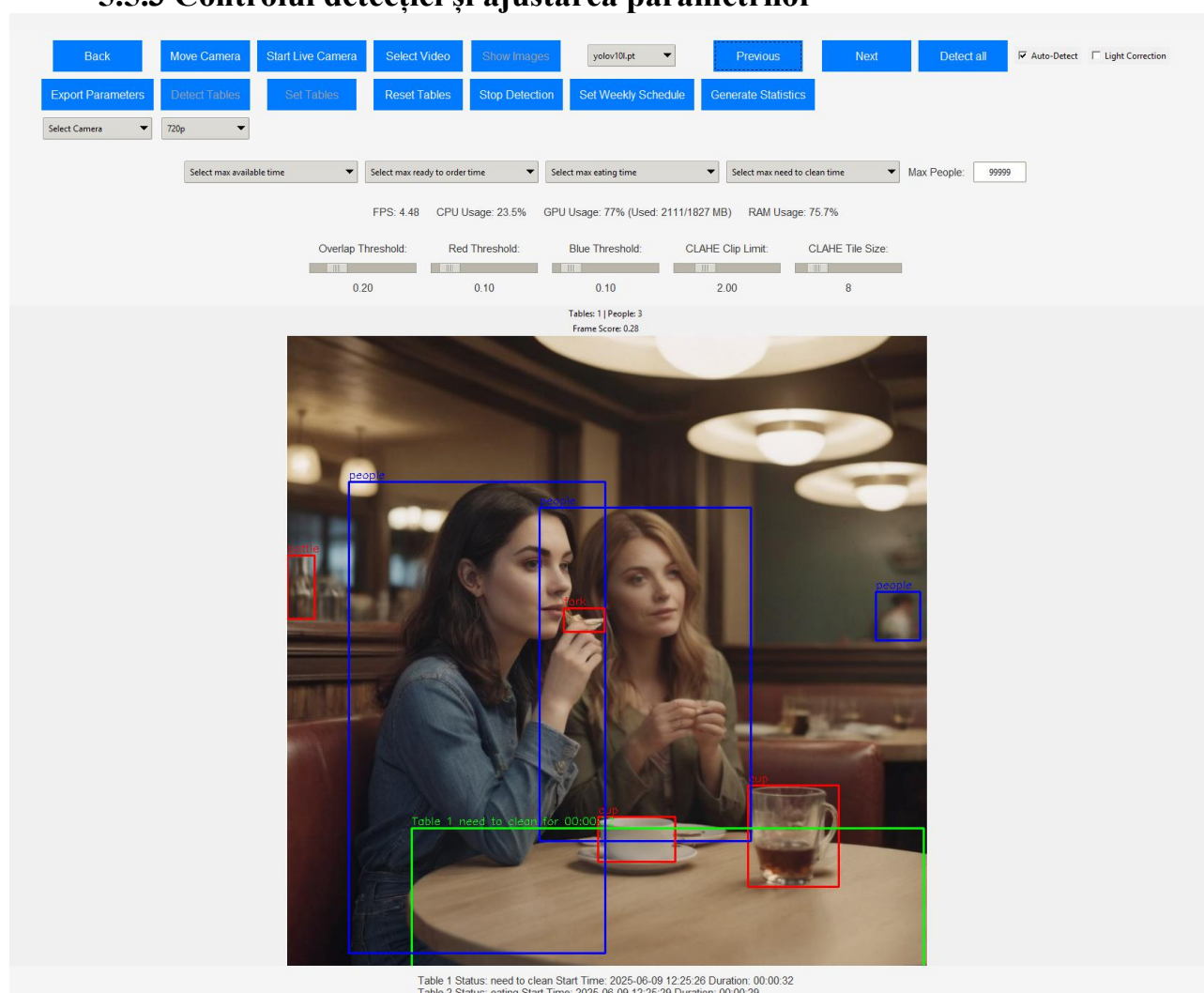


Figura 5.1 – Interfața Admin GUI

În cadrul Admin GUI, după cum este ilustrat în Figura 5.1, utilizatorul are posibilitatea de a selecta modelul YOLO dorit dintr-un meniu derulant, de a alege sursa video (cameră web sau fișier video) și de a activa diferite moduri de detecție, printre care:

- Detectare completă (Detect All);
- Detectare exclusivă a meselor (Detect Tables);
- Setare poziții mese (Set Tables);
- Resetare configurare mese (Reset Tables).

De asemenea, poate fi activat modul Auto-Detect, care rulează automat un algoritm de detecție continuă, actualizând în mod dinamic starea fiecărei mese.

Parametrii de prag, esențiali în clasificarea corectă a stării meselor, pot fi reglați prin glisoare:

- Overlap Threshold – stabilește procentul minim de suprapunere între un obiect și masa analizată pentru ca obiectul să fie considerat relevant;
- Red Threshold și Blue Threshold – ajustează filtrarea obiectelor roșii sau a persoanelor, în funcție de raportul dintre aria obiectului și cea a mesei.

5.5.4 Selectarea camerei și a rezoluției

Prin intermediul interfeței, utilizatorul poate selecta una dintre camerele disponibile (detectate automat de aplicație) și poate alege rezoluția dorită (480p, 720p sau 1080p). Aceste setări permit adaptarea performanței detecției la resursele hardware disponibile.

5.5.5 Configurarea timpilor maximi și a numărului de persoane

Un element important al aplicației este posibilitatea de a seta timpii maximi acceptați pentru fiecare stare posibilă a mesei (available, ready to order, eating, need to clean). Aceste valori pot fi selectate din liste derulante, câte una pentru fiecare stare. În plus, utilizatorul poate introduce un număr maxim admis de persoane într-un cadru, depășirea acestuia declanșând o alertă.

```
self.time_available = tk.StringVar(value='Select Max Time Available')
self.time_ready = tk.StringVar(value='Select Max Time Waiting')
self.time_eating = tk.StringVar(value='Select Max Time Eating')
self.time_clean = tk.StringVar(value='Select Max Time Clean')
```

Aceste setări sunt esențiale pentru logica de declanșare a alertelor și pentru identificarea potențialelor situații anormale.

5.5.6 Salvarea configurației

După finalizarea configurării, aplicația oferă opțiunea de a salva parametrii într-un fișier JSON localizat în directorul `data/config`, identificat printr-un timestamp unic. Acest fișier poate fi ulterior importat în User GUI, facilitând rularea aplicației în regim operațional.

```
config_data = {
    "selected_camera": self.selected_camera,
    "selected_model": self.model_var.get(),
    "video_resolution": self.video_resolution,
    "max_times": {
        "available": self.time_available.get(),
        "ready_to_order": self.time_ready.get(),
        "eating": self.time_eating.get(),
        "need_to_clean": self.time_clean.get()
    },
    "max_people": int(self.max_people_var.get()),
    "overlap_threshold": round(float(self.overlap_threshold_var.get()),
2),
```

```

    "red_threshold": round(float(self.red_threshold_var.get()), 2),
    "blue_threshold": round(float(self.blue_threshold_var.get()), 2),
}

```

5.5.7 Interfața User GUI

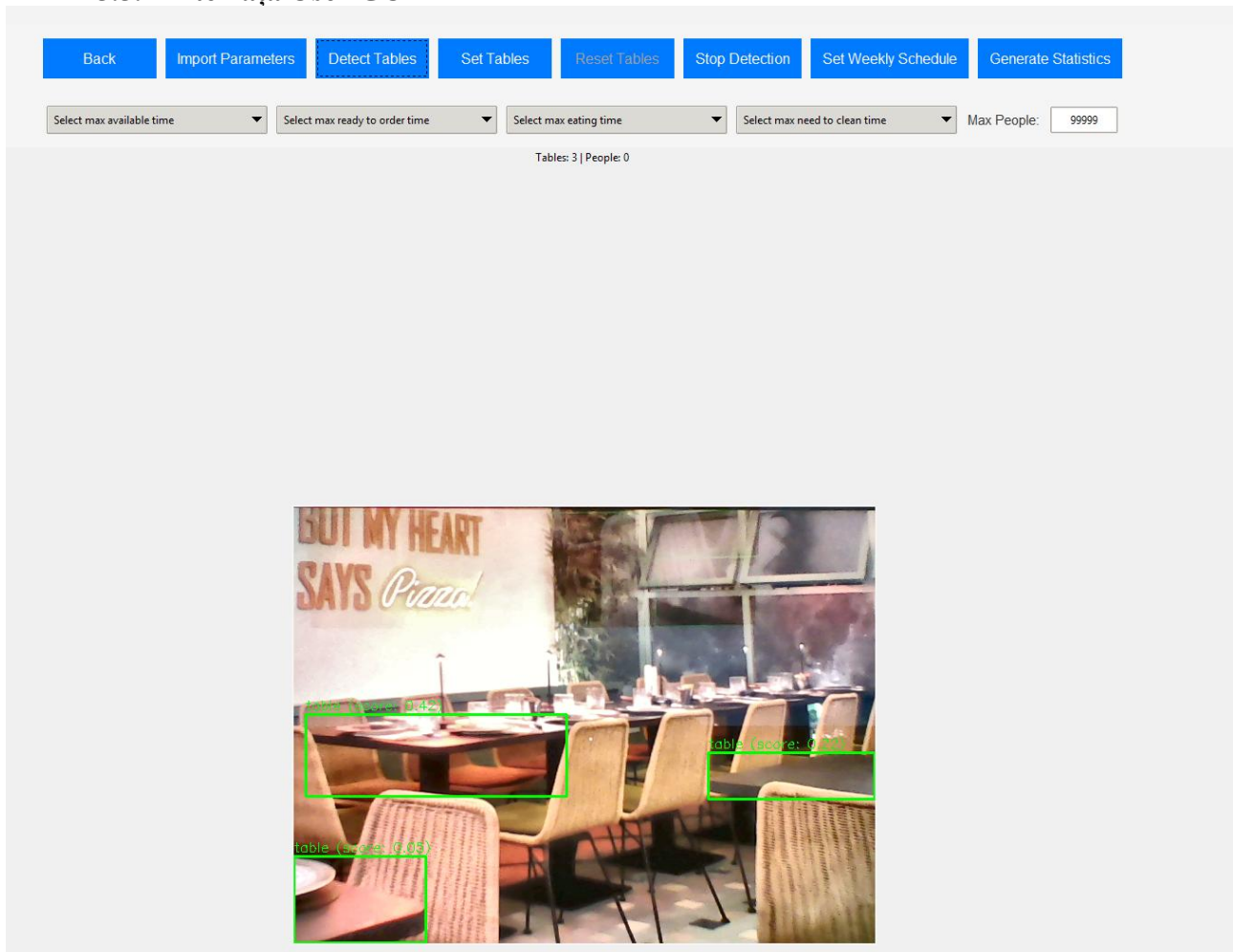


Figura 5.2 – Interfața User GUI

Versiunea destinată utilizatorului final (`user_gui.py`), după cum este ilustrat în Figura 5.2, oferă o interfață simplificată, care încarcă automat fișierul de configurare creat anterior în Admin GUI și inițiază sesiunea de monitorizare. Utilizatorul nu poate modifica parametrii detecției, având rolul de a urmări în timp real rezultatele și de a primi notificări vizuale sau alerte sonore.

5.5.8 Concluzii

Interfața grafică reprezintă un element cheie pentru ușurința utilizării și eficiența sistemului în mediul real. Prin separarea clară a interfeței de administrare față de cea destinată utilizatorului final, aplicația asigură un echilibru optim între flexibilitate, securitate și accesibilitate. Astfel, orice utilizator poate beneficia de funcționalitățile complexe ale sistemului fără a necesita cunoștințe tehnice aprofundate.

5.6 Codul sursă al microcontrolerului ESP32-S3-MINI și integrarea acestuia

5.6.1 Rolul microcontrolerului în arhitectura sistemului

Microcontrolerul ESP32-S3-MINI ocupă un rol central în arhitectura sistemului, asigurând comunicarea dintre aplicația software care rulează pe PC și ansamblul fizic de orientare a camerei video. Programul încărcat pe microcontroler are ca funcție interpretarea comenzilor recepționate prin interfața serială și generarea, prin intermediul bibliotecii ESP32Servo, a semnalelor PWM necesare acționării celor două servomotoare de tip MG996R. Acestea controlează orientarea camerei în planurile orizontal (Ox) și vertical (Oy), contribuind la stabilirea unghiului optim de captare a imaginilor.

Este important de subliniat că acest mecanism de control este activ doar în faza de configurare inițială. Obiectivul său principal este identificarea unei poziții optime a camerei, astfel încât aceasta să capteze un număr maxim de mese în câmpul vizual. Odată ce această poziție a fost stabilită de utilizator, prin intermediul interfeței grafice administrative, camera rămâne fixată, iar microcontrolerul nu mai procesează comenzi noi, cu excepția cazului în care utilizatorul inițiază manual o recalibrare.

Orientarea camerei se poate realiza fie manual, printr-un joystick virtual disponibil în modul `move_camera_frame.py` al interfeței grafice, fie automat, printr-un algoritm de scanare sistematică. În modul automat, camera este deplasată pe o traiectorie spiralată, explorând treptat spațiul disponibil. La fiecare poziție, este calculat un „scor al cadrului” (`frame_score`), care reflectă calitatea unghiului în funcție de numărul și poziționarea meselor detectate în imagine.

Formula de calcul utilizată este următoarea:

```
self.frame_score = (total_score / num_tables) * num_tables if num_tables > 0 else 0
```

Aceasta determină scorul pe baza devierii meselor față de centrul imaginii. La finalul procesului de explorare, cadrul cu scorul maxim este ales drept unghi optim, iar camera este repositionată automat.

5.6.2 Descrierea generală a programului implementat

Codul sursă al microcontrolerului a fost dezvoltat utilizând platforma Arduino IDE, cu ajutorul unui set restrâns de biblioteci dedicate controlului serial și al servomotoarelor. Programul principal se află în directorul `hardware/arduino_servo_camera/arduino_servo_camera.ino`. Structura sa este simplă, fiind împărțită în două secțiuni principale:

- `setup()` – funcția în care sunt inițializate comunicația serială la o viteză de 921600 bauds și sunt configurați pinii asociați celor două servomotoare, prin atașarea acestora la obiectele Servo, ceea ce permite generarea semnalelor PWM necesare poziționării inițiale;
- `loop()` – funcția principală de execuție continuă, în cadrul căreia sunt citite comenzile primite prin interfața serială, interpretate sub forma unghiurilor pentru axele Ox și Oy, și aplicate gradual către cele două servomotoare, asigurând o mișcare lină și controlată a camerei motorizate.

5.6.3 Inițializarea comunicației seriale

Inițializarea comunicației și a controlului servomotoarelor este realizată în funcția `setup()`, astfel:

```
void setup() {  
    Serial.begin(921600); // Configurare serială la viteză mare  
    servol.attach(13);  
    servo2.attach(14);  
}
```

Comunicarea este stabilită la o viteză de 921600 bps, permițând transfer rapid de date între aplicația desktop și microcontroler. Pini 13 și 14 sunt destinați controlului celor două servomotoare.

5.6.4 Recepționarea și interpretarea comenzilor

Comenzile transmise din aplicația desktop respectă formatul `ox_oy`, unde `ox` și `oy` reprezintă unghiurile dorite pentru fiecare servo. Microcontrolerul citește șirul de caractere, îl împarte și transmite valorile către servomotoare:

```
void loop() {  
    if (Serial.available()) {  
        String command = Serial.readStringUntil('\n');  
        int separatorIndex = command.indexOf('_');  
  
        if (separatorIndex > 0 && separatorIndex < command.length() - 1) {  
            int ox = command.substring(0, separatorIndex).toInt();  
            int oy = command.substring(separatorIndex + 1).toInt();  
  
            ox = constrain(ox, 0, 180);  
            oy = constrain(oy, 0, 180);  
  
            servol.write(ox);  
            servo2.write(oy);  
        }  
    }  
}
```

Această abordare simplificată și robustă permite controlul în timp real al camerei, valorile transmise fiind încadrate automat în intervalul valid de 0° – 180° prin funcția `constrain()`.

5.6.5 Testarea sistemului prin interfața grafică

Un modul separat din aplicația desktop, `test_servos_frame.py`, oferă posibilitatea de a trimite manual comenzi către microcontroler, facilitând astfel verificarea funcționării și calibrarea sistemului de control al camerei. Această funcție este utilă pentru ajustarea poziției camerei și pentru validarea comunicației seriale.

5.6.6 Motivația utilizării unui baudrate ridicat

Alegerea unei viteze de transmisie de 921600 bps a fost motivată de necesitatea reducerii latenței. Deoarece procesul de detecție rulează la 30–60 cadre pe secundă, era esențial ca răspunsul camerei să fie sincronizat cu fluxul video. Suportul nativ al ESP32-S3-MINI pentru viteze mari pe UART a permis o integrare fără întârzieri perceptibile între aplicația desktop și microcontroler.

5.6.7 Posibilități de extindere

Microcontrolerul ESP32-S3-MINI dispune de conectivitate Wi-Fi și Bluetooth, ceea ce deschide oportunități pentru extinderea sistemului. În viitor, controlul camerei ar putea fi realizat wireless, prin comunicație TCP/IP sau Bluetooth LE, eliminând nevoia unei conexiuni fizice prin cablu USB-C.

5.6.8 Concluzii

Codul implementat pe ESP32-S3-MINI asigură o legătură eficientă și rapidă între software-ul de detecție de pe PC și sistemul fizic de control al camerei. Prin integrarea acestui modul hardware, sistemul beneficiază de o soluție completă, capabilă să coordoneze atât analiza video avansată, cât și acționarea fizică. În utilizarea obișnuită, microcontrolerul rămâne inactiv după stabilirea poziției inițiale a camerei, fiind reactivat doar în urma unei comenzi explicite de recalibrare din partea utilizatorului.

5.7 Generarea automată și manuală a rapoartelor statistice

5.7.1 Prezentare generală

În cadrul sistemului software implementat, a fost dezvoltat un modul dedicat generării și analizării rapoartelor statistice. Aceste rapoarte se împart în două categorii principale:

- rapoarte generate automat la sfârșitul fiecărei zile, pe baza datelor colectate;
- rapoarte realizate manual de către utilizator, pentru o perioadă selectată, cu scop comparativ între diferite zile, săptămâni sau luni.

Ambele tipuri de rapoarte utilizează informațiile salvate în fișiere JSON pe parcursul funcționării sistemului. După procesare, rezultatele sunt exportate în fișiere Excel și JSON, oferind o imagine detaliată asupra activității restaurantului și comportamentului clienților.

5.7.2 Generarea automată a rapoartelor zilnice

Sistemul creează rapoartele zilnice în mod automat la finalul programului configurat în `ScheduleFrame`. La terminarea zilei, sunt generate două fișiere:

- `table_records.json`, care stochează informațiile despre stările meselor (available, eating, ready to order, need to clean);
- `people_records.json`, care conține date referitoare la numărul de persoane identificate pe intervale orare.

Analiza acestor fișiere este realizată prin clasa `AutomaticReportGenerator`, definită în fișierul `automatic_report_generator.py`. Aceasta pune la dispoziție metode precum:

- `save_to_excel()`, care creează automat un fișier Excel cu date brute, grafice și statistici prelucrate;
- `save_average_statistics()`, care calculează indicatorii zilnici și îi salvează într-un fișier `average_statistics.json`.

Exemplu de salvare a datelor:

```
with open(json_file_path, "w") as json_file:
```

```
json.dump(data_to_save, json_file, indent=4)
```

5.7.3 Indicatorii calculați pentru analiza zilnică

Pe baza datelor colectate, sistemul calculează următorii indicatori de performanță:

- Idle Time Ratio: proporția de timp în care masa nu a fost ocupată;
- Cleaning Ratio: procentul de timp în care masa a necesitat curățare;
- Cycles per Hour: numărul de cicluri complete pe oră (ciclu = available → eating → need to clean);
- Utilization Rate: proporția de timp în care masa este ocupată sau se așteaptă comanda;
- Performance Score: scor general calculat astfel:

```
performance_score = (  
    0.4 * cycles_per_hour +  
    0.3 * utilization_rate -  
    0.2 * cleaning_ratio -  
    0.1 * idle_time_ratio  
)
```

Acești parametri ajută la clasificarea meselor în funcție de eficiență și la identificarea potențialelor surse de întârziere sau utilizare suboptimă.

5.7.4 Vizualizarea automată a performanței zilnice

Metoda `save_to_excel()` creează automat diverse grafice și vizualizări, printre care:

- histograme cu duratele de status ale fiecărei mese;
- grafice circulare cu distribuția medie a fiecărui status;
- box plot-uri care surprind variația duratelor pe status;
- histograme privind durata și frecvența ciclurilor meselor;
- tabele comparative cu scorurile de performanță;
- grafice cu evoluția numărului maxim de persoane detectate pe oră.

Exemplu de generare a unui grafic:

```
ax.bar(people_max_count['time_slot'].astype(str),  
people_max_count['people_count'], color='skyblue')  
ax.set_title('Max Number of People per Hour')
```

5.7.5 Generarea manuală a rapoartelor comparative

Pentru intervale mai extinse, sistemul permite utilizatorului să selecteze o perioadă de interes printr-o interfață calendaristică (clasa `ManualReportGenerator`, fișier `manual_report_generator.py`). Procesul implică:

1. Identificarea fișierelor `average_statistics.json` corespunzătoare perioadei alese;
2. Combinarea acestora într-un fișier `general_stats.json`;
3. Crearea fișierului Excel `general_statistic.xlsx`, care conține tabele și grafice comparative.

Acest fișier include:

- tabele cu indicatorii zilnici agregați;
- grafice comparative pentru fiecare indicator (ex.: utilization rate);
- linii de trend obținute prin regresie liniară:

```
trend = np.polyfit(x, values, 1)
trend_line = np.polyval(trend, x)
```

5.7.6 Interpretarea statistică și evaluarea performanței

Rapoartele generate permit:

- compararea eficienței restaurantului în zile diferite;
- identificarea zilelor cu activitate slabă sau vârfuri de trafic;
- evaluarea performanței fiecărei mese, în funcție de frecvența ciclurilor și gradul de utilizare;
- realizarea unei evaluări globale prin scoruri sintetice, comparabile între intervale.

5.7.7 Structura finală a fișierelor generate

La finalul procesării, fișierele sunt organizate astfel:

- data/outputs/daily_report/YYYY-MM-DD/average_statistics.json – indicatori zilnici;
- table_records.json și people_records.json – date brute zilnice;
- general_stats.json – agregare de statistici pe interval selectat;
- general_statistic.xlsx – analiză sintetică cu tabele și grafice.

Fișierul Excel include următoarele foi:

- Histogram;
- People Statistics;
- Status Analysis;
- Cycle Duration;
- Table Performance.

Această structură oferă utilizatorului și administratorului instrumente concrete și intuitive pentru analiza și optimizarea activității restaurantului, prin utilizarea unor măsurători obiective și vizualizări clare.

5.8 – Metodologia de testare software și depanare

5.8.1 Obiectivele testării

Activitatea de testare software a avut ca scop principal verificarea corectitudinii funcționale, a rezistenței aplicației și a stabilității în condiții de utilizare reală. Procesul de testare a vizat atât componentele software rulate pe PC, cât și firmware-ul implementat pe microcontrolerul ESP32-S3-MINI.

5.8.2 Testarea modulară a componentelor AI

Pentru a asigura buna funcționare a fiecărui modul, testarea s-a desfășurat individual pe următoarele componente:

- Modulul de detecție YOLO: a fost verificată încărcarea corectă a modelului, recunoașterea claselor vizate și acuratețea detecțiilor;
- Funcțiile de corectare a iluminării: au fost testate pe imagini cu iluminare deficitară sau neuniformă;
- Algoritmii de clasificare a meselor: evaluați prin imagini cu suprapuneri parțiale între obiecte și mese;
- Interfața grafică (GUI): a fost testată funcționalitatea tuturor elementelor grafice și validarea comenzilor transmise către ESP32;
- Modulul de export în Excel și analiza statistică: s-au comparat valorile generate automat cu rezultate calculate manual.

5.8.3 Simularea comenzilor transmise către ESP32

Pentru verificarea comunicării seriale dintre aplicație și microcontroler, au fost aplicate următoarele teste:

- Trimiterea manuală a comenzilor prin GUI pentru controlul unghiurilor servomotoarelor;
- Monitorizarea comenzilor recepționate folosind Serial Monitor;
- Observarea poziționării efective a camerei în urma comenzilor transmise.

Testele s-au realizat în două moduri distincte:

- Manual, prin modulul `test_servos_frame.py`;
- Automat, utilizând algoritmul de mișcare în spirală din `move_camera_frame.py`.

5.8.4 Teste de stabilitate și stres

Pentru evaluarea rezistenței aplicației, sistemul a fost rulat continuu timp de 8 ore, urmărindu-se:

- Consumul de memorie și detectarea eventualelor scurgeri de resurse;
- Menținerea sincronizării între cadre, consistența detecțiilor și precizia clasificării stărilor meselor;
- Verificarea integrității fișierelor JSON generate pe durata funcționării.

5.8.5 Testarea exportului și vizualizării datelor

Modulul `json_to_excel.py` a fost supus unor teste pentru a valida:

- Corectitudinea tabelelor generate în fișierele Excel;
- Concordanța între valorile medii și datele brute;
- Calitatea graficelor produse (histograme, box plot, grafice circulare);
- Compatibilitatea fișierelor create cu diferite versiuni de Excel (2016, 2019) și cu Google Sheets.

De asemenea, `calendar_frame.py` a fost testat în situații speciale:

- Alegerea unei date de început mai mari decât data de sfârșit;
- Interval extins de peste 30 de zile;
- Absența fișierelor JSON pentru anumite date din intervalul selectat.

5.8.6 Testarea interfeței grafice (GUI)

Pentru interfața grafică, testarea a inclus:

- Verificarea adaptabilității la diferite dimensiuni de ecran;
- Funcționarea corectă a sliderelor și elementelor de selecție;
- Afișarea corectă a detecțiilor și feedback vizual asociat;
- Confirmarea funcționării alertelor sonore și vizuale;
- Navigarea între modurile de operare (user/admin, cameră/video/imagini);
- Persistența setărilor salvate și încărcate.

5.8.7 Debugging și logging

Pentru facilitarea procesului de depanare, s-au implementat mecanisme de logare și mesaje informative:

- Mesaje de tip `print()` și `logging.info()` în punctele critice ale aplicației;
- Afișarea valorilor FPS și scorurilor direct în GUI;
- Salvarea automată a erorilor într-un fișier `error_log.txt`.

Exemplu de logare a erorilor:

```
try:
    detections = self.model(frame_tensor)
except Exception as e:
    with open("logs/error_log.txt", "a") as f:
        f.write(f"Error on frame {i}: {str(e)}\n")
```

5.9 Direcții de extindere viitoare a aplicației

5.9.1 Control wireless al camerei video

O direcție evidentă de extindere, având în vedere capacitățile native ale microcontrolerului ESP32-S3-MINI, este eliminarea conexiunii prin cablu USB-C și migrarea către un control wireless. Această tranziție ar putea include:

- Implementarea comunicației prin Wi-Fi Direct sau Bluetooth Low Energy (BLE);
- Dezvoltarea unei aplicații mobile sau a unei interfețe web dedicate controlului camerei;
- Utilizarea unui protocol de comunicație precum JSON-over-UDP sau MQTT pentru transmiterea comenzilor către microcontroler.

5.9.2 Salvarea datelor într-o bază de date

Pentru o gestiune mai eficientă a datelor pe termen lung, sistemul ar putea înlocui utilizarea fișierelor JSON cu o bază de date relațională. Această modificare ar putea include:

- Implementarea unei baze de date SQLite sau PostgreSQL;
- Crearea unor tabele dedicate pentru detecții, stările meselor, alarme și utilizatori;
- Posibilitatea realizării unor interogări complexe pentru analiză istorică și generarea de rapoarte personalizate.

5.9.4 Monitorizare în timp real prin browser

Extinderea funcționalității aplicației pentru a permite vizualizarea și controlul în timp real printr-un browser web reprezintă o evoluție naturală. Această implementare ar putea presupune:

- Dezvoltarea unui server Flask care să preia fluxul video și detecțiile asociate;
- Oferirea unui live stream cu suprapunerea vizuală a detecțiilor YOLO direct în interfața web;
- Integrarea unor funcționalități de control al camerei și afișarea indicatorilor de performanță într-un dashboard online.

5.9.5 Suport multi-cameră și segmentare pe zone

Sistemul ar putea fi scalat pentru a permite utilizarea simultană a mai multor camere, beneficiind astfel de o acoperire extinsă. Această extindere ar putea implica:

Identificarea automată și gestionarea mai multor fluxuri video provenite de la camere diferite;
Clasificarea meselor în funcție de zonele restaurantului (de exemplu: terasă, interior);

Optimizarea fluxurilor de monitorizare și analiză statistică pentru restaurante cu suprafețe mari sau mai multe săli.

Capitolul 6: Testarea, optimizarea și evaluarea sistemului integrat

6.1 Justificarea metodologiei de testare și platforma utilizată

Etapă de testare și optimizare a sistemului AI Restaurant Monitoring a fost esențială pentru validarea funcționalității, a robusteții și a capacității de adaptare la condiții reale de operare. Dincolo de verificarea componentelor software și hardware în mod individual, scopul principal a fost analiza integrată a întregului flux de procesare, în contextul unui mediu vizual variabil și imprevizibil, așa cum este cel specific restaurantelor. Pentru desfășurarea testelor, s-a utilizat în mod exclusiv interfața AdminGUI, concepută pentru a oferi control granular asupra tuturor parametrilor relevanți: modele YOLO active, niveluri de rezoluție, activarea preprocesării iluminării, praguri de filtrare, monitorizarea performanței hardware și exportul rezultatelor în format JSON sau Excel.

Testarea a fost efectuată atât pe două videoclipuri filmate în locații reale de tip restaurant, cât și pe imagini sintetice generate cu inteligență artificială, în scopul acoperirii unei game cât mai largi de scenarii. Prin această combinație de surse, s-a putut investiga atât comportamentul sistemului în contexte dinamice (iluminare variabilă, mișcare, aglomerare), cât și în situații limită, simulate artificial (reflexii intense, obiecte parțial vizibile, unghiuri neobișnuite).

6.2 Preprocesarea iluminării și impactul asupra detecției

Una dintre provocările recurente identificate în mediile reale de testare a fost reprezentată de variațiile de iluminare. Scenele captate prezentau de multe ori combinații de surse de lumină artificială, lumină naturală filtrată, umbre și reflexii generate de suprafețe lucioase sau de mișcarea personalului. Astfel de condiții pot afecta negativ performanța modelelor YOLO, care, în absența unui preprocesor adaptiv, pot genera detecții incorecte sau incomplete.

Pentru a contracara aceste efecte, a fost implementat algoritmul `correct_illumination`, activabil opțional din AdminGUI. Acesta parcurge mai multe etape esențiale: conversia imaginii din spațiul BGR în LAB, aplicarea metodei CLAHE (Contrast Limited Adaptive Histogram Equalization) pentru canalul de luminozitate, ajustarea adaptivă a valorii gamma și normalizarea finală a contrastului.

Testele efectuate au evidențiat o îmbunătățire vizibilă a clarității marginilor și a consistenței detecțiilor YOLO. Analiza comparativă a arătat o scădere de până la 20% a numărului de detecții fals pozitive, mai ales în zonele margine ale cadrelor sau în proximitatea surselor directe de lumină. Deși timpul mediu de procesare per cadru a crescut cu 10–15 ms, acest cost a fost considerat acceptabil în faza de inițializare a scenei, unde precizia este prioritară.

6.3 Filtrarea detecțiilor suprapuse și stabilizarea logicii de interpretare

Un al doilea proces esențial analizat în profunzime a fost filtrarea detecțiilor multiple și redundante generate de YOLO în contexte aglomerate. În scenele analizate, mesele erau adesea încărcate cu obiecte (pahare, farfurii, sticle), iar clienții se aflau în proximitate. Această densitate vizuală ridicată determina modelul să emită mai multe box-uri pentru același obiect, cu variații minime în scoruri și poziții.

Algoritmul `filter_detections`, integrat în AdminGUI, are rolul de a detecta suprapunerile nejustificate și de a elimina acele detecții care depășesc un prag definit de utilizator (default 0.2 pentru IoU). Dintre detecțiile suprapuse, se păstrează doar cea cu scorul de încredere cel mai mare, asigurând astfel un echilibru între precizie și concizie.

Testele au arătat că, în medie, numărul detecțiilor redundante a fost redus cu 35–40%, iar randarea vizuală a devenit mult mai clară. Acest efect a fost observat în mod special în afișarea statusurilor meselor, care s-au stabilizat după aplicarea filtrării, reducându-se semnificativ numărul cazurilor de clasificare greșită (ex. mese considerate ocupate din cauza dublării detecțiilor de farfurii). Eficiența acestui algoritm a fost dovedită atât în imagini statice AI, cât și în video-uri în care obiectele se aflau în mișcare.

6.4 Adaptarea rezoluției video și comutarea inteligentă între modele

Pentru a menține un echilibru optim între precizie și viteză de procesare, sistemul a fost testat în AdminGUI la trei niveluri de rezoluție: 480p, 720p și 1080p. Această alegere a permis observarea directă a efectului rezoluției asupra comportamentului fiecărui model YOLO utilizat: `yolov8n.pt`, `yolov8l.pt`, `yolov10l.pt` și `yolov10x.pt`.

La 480p, modelul `yolov8n.pt` a rulat la 45–50 FPS pe un laptop ASUS TUF (i5 Gen10, GTX 1650 Ti), demonstrând un flux stabil și rapid. Totuși, în zonele slab iluminate sau în colțurile imaginii, acuratețea detecțiilor a fost limitată. În contrast, `yolov10l.pt`, deși a funcționat la doar 20–25 FPS în aceeași rezoluție, a reușit să identifice obiecte în condiții dificile. Din acest motiv, rezoluția 480p a fost considerată adecvată pentru monitorizarea continuă, dar nu suficientă pentru inițializarea scenei.

Rezoluția 720p a oferit un compromis eficient: suficientă claritate pentru modele mari și performanță acceptabilă în termeni de FPS. În acest context, `yolov10l.pt` a rulat la 15–20 FPS, iar `yolov8n.pt` la 30–35 FPS, fiind astfel considerată rezoluția optimă pentru etapa de inițializare.

În cazul 1080p, precizia a atins niveluri maxime, dar viteza de procesare a scăzut considerabil. Modelele mari (`yolov10l`, `yolov10x`) au funcționat cu sub 15 FPS pe sistemul testat, motiv pentru care Full HD a fost recomandat doar pentru procesare offline sau pentru sisteme cu GPU-uri avansate.

Pe baza acestor observații, s-a implementat o strategie automată de comutare între modele YOLO și rezoluții, adaptată în funcție de etapa de operare. La inițializare, se activează `yolov10l.pt` în HD pentru precizie maximă, iar după poziționarea camerei, sistemul comută pe `yolov8n.pt` în 480p pentru eficiență în timp real. Această strategie a fost testată cu succes în AdminGUI și exportată în UserGUI sub formă de parametri JSON.

6.5 Concluzii generale ale testării și evaluării

Concluziile testării arată că sistemul AI Restaurant Monitoring, validat exclusiv prin interfața AdminGUI, demonstrează o funcționare stabilă, precisă și adaptabilă, datorită strategiei hibride de comutare între modele YOLO, a preprocesării inteligente a iluminării și a filtrării detecțiilor suprapuse, toate calibrate riguros în funcție de rezoluție și performanța hardware; astfel, sistemul asigură o detecție robustă în condiții reale și poate fi operat eficient chiar și pe platforme medii, oferind o experiență predictibilă și ușor de implementat pentru utilizatorul final prin intermediul UserGUI.

Capitolul 7: Concluzii și perspective viitoare

Lucrarea de față a urmărit proiectarea și implementarea unui sistem inteligent de monitorizare a activității într-un restaurant, bazat pe procesare vizuală în timp real și analiză automată. Rezultatele obținute confirmă fezabilitatea unei soluții complete, care combină algoritmi AI cu control hardware, oferind supraveghere eficientă, detecție precisă a meselor și obiectelor, și generare automată de statistici relevante.

Contribuția principală a constat în realizarea unei arhitecturi modulare software-hardware, formată din două interfețe grafice (AdminGUI și UserGUI), un algoritm personalizat de corecție a iluminării, un sistem de filtrare a detecțiilor suprapuse și un mecanism inteligent de comutare între modele YOLO, în funcție de etapa de funcționare. De asemenea, a fost integrat cu succes un microcontroler ESP32-S3-MINI care controlează poziția camerei video motorizate, oferind flexibilitate în inițializarea unghiului de filmare.

În procesul de dezvoltare au fost identificate provocări precum variațiile de lumină, detecțiile multiple, sau limitările hardware. Acestea au fost soluționate prin corecții adaptative ale contrastului, filtre logice de suprapunere și utilizarea unui model YOLO mare pentru inițializare, urmat de un model rapid pentru monitorizare continuă. Sistemul s-a dovedit robust și adaptabil, chiar și pe o configurație hardware medie.

Testarea funcțională în AdminGUI a permis o calibrare riguroasă a fiecărui parametru implicat în procesul de analiză vizuală. Evaluarea performanței a fost realizată pe videoclipuri reale și imagini sintetice, în diferite rezoluții, demonstrând că sistemul poate funcționa în condiții variate de iluminare și complexitate vizuală, menținând acuratețea detecțiilor și stabilitatea generală.

Pe termen lung, sistemul poate fi extins prin integrarea unor componente suplimentare de analiză predictivă, bazate pe machine learning. Acestea ar permite anticiparea perioadelor aglomerate și optimizarea în timp real a resurselor umane, contribuind la creșterea eficienței operaționale a unităților de alimentație publică.

Perspectivile de dezvoltare includ integrarea unui control wireless al camerei, migrarea către baze de date relaționale, exportul rapoartelor în cloud, vizualizarea în browser și suportul pentru mai multe camere. Totodată, antrenarea unor algoritmi de predicție ar permite transformarea sistemului într-un instrument decizional activ, util în managementul restaurantelor moderne.

În concluzie, sistemul propus este funcțional, extensibil și adaptabil, reprezentând o bază solidă pentru soluții comerciale scalabile în domeniul monitorizării inteligente.

Bibliografie

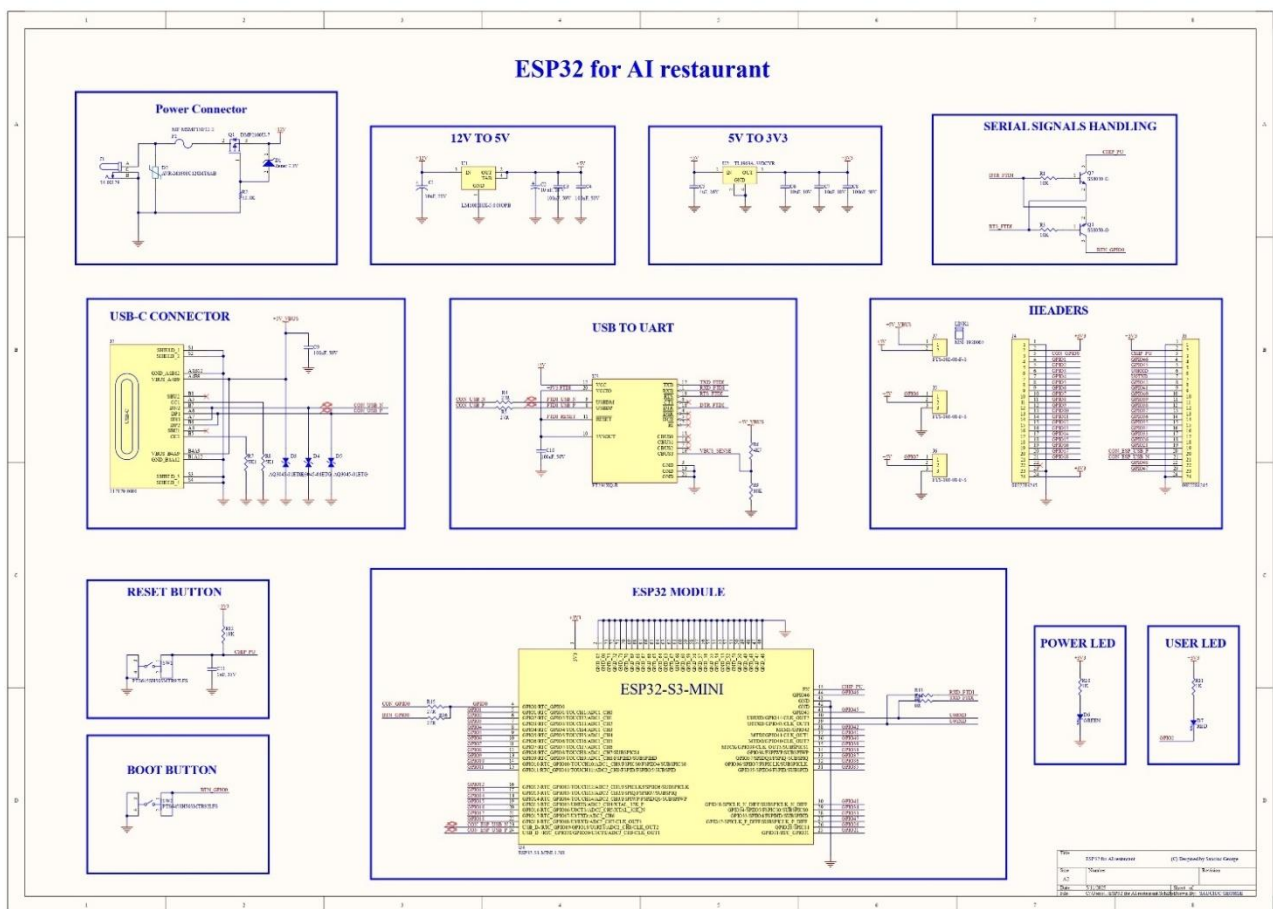
- [1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016. [Online]. Disponibil: <https://www.deeplearningbook.org>
- [2] A. Jenkins, "15 Ways AI is Impacting the Restaurant Industry," *NetSuite*, Oct. 11, 2024. [Online]. Disponibil: <https://www.netsuite.com/portal/resource/articles/business-strategy/ai-in-restaurants.shtml>
- [3] V7 Labs, "YOLO Algorithm for Object Detection Explained [+Examples]," *V7 Labs*, Jan. 2023. [Online]. Disponibil: <https://www.v7labs.com/blog/yolo-object-detection>
- [4] SAS, „Computer Vision: What it is and why it matters,” [Online]. Disponibil: https://www.sas.com/en_us/insights/analytics/computer-vision.html. [Accesat: 6 mai 2025]. [SAS](#)
- [5] Medium, „What is Computer Vision? (History, Applications, Challenges),” [Online]. Disponibil: <https://medium.com/@ambika199820/what-is-computer-vision-history-applications-challenges-13f5759b48a5>. [Accesat: 6 mai 2025]. [Medium](#)
- [6] V7 Labs, „What Is Computer Vision? [Basic Tasks & Techniques],” [Online]. Disponibil: <https://www.v7labs.com/blog/what-is-computer-vision>. [Accesat: 6 mai 2025]. [V7 AI](#)
- [7] Medium, „Top Computer Vision Opportunities and Challenges for 2024,” [Online]. Disponibil: <https://medium.com/sciforce/top-computer-vision-opportunities-and-challenges-for-2024-31a238cb9ff2>. [Accesat: 6 mai 2025]. [Medium](#)
- [8] Wikipedia, „You Only Look Once,” [Online]. Disponibil: https://en.wikipedia.org/wiki/You_Only_Look_Once. [Accesat: 6 mai 2025].
- [9] Encord, „YOLO Object Detection Explained: A Beginner's Guide,” [Online]. Disponibil: <https://encord.com/blog/yolo-object-detection-guide/>. [Accesat: 6 mai 2025]. [Encord+1Learn R, Python & Data Science Online+1](#)
- [10] Medium, „Top Computer Vision Opportunities and Challenges for 2024,” [Online]. Disponibil: <https://medium.com/sciforce/top-computer-vision-opportunities-and-challenges-for-2024-31a238cb9ff2>. [Accesat: 6 mai 2025].
- [11] Kili Technology, „YOLO Algorithm: Real-Time Object Detection from A to Z,” [Online]. Disponibil: <https://kili-technology.com/data-labeling/machine-learning/yolo-algorithm-real-time-object-detection-from-a-to-z>. [Accesat: 6 mai 2025]. [kili-website+1kili-website+1](#)
- [12] J. VanderPlas, *Manualul de știința datelor cu Python: Instrumente esențiale pentru lucrul cu date*, O'Reilly Media, 2016.
- [13] W. McKinney, *Python pentru analiza datelor: Prelucrarea datelor cu Pandas, NumPy și IPython*, ediția a 2-a, O'Reilly Media, 2017.
- [14] E. Charles, *Automatizarea Excel cu Python: Ghid practic pentru openpyxl și xlwings*, Publicare independentă, 2020.
- [15] J.D. Hunter și alții, „Matplotlib: Un mediu grafic 2D,” *Computing in Science & Engineering*, vol. 9, nr. 3, pp. 90-95, 2007.
- [16] D. Crockford, *JSON: Alternativa ușoară la XML*, 2006. [Online]. Disponibil la: <https://www.json.org/json-en.html>. [Accesat: 06-mai-2025].

- [17] Python Software Foundation, „datetime — Tipuri de bază pentru date și ore,” *Documentația Python 3.12.6*, 2024. [Online]. Disponibil la: <https://docs.python.org/3/library/datetime.html>. [Accesat: 06-mai-2025].
- [18] A. Sweigart, *Automatizează sarcinile plictisitoare cu Python: Programare practică pentru începători*, ediția a 2-a, No Starch Press, 2019.
- [19] A. M. Davidson, *Restaurant Operations Management: Principles and Practices*, Routledge, 2020.
- [20] Avigilon Corporation, „Sisteme video inteligente pentru restaurante,” Avigilon, 2023. [Online]. Disponibil la: <https://www.avigilon.com>. [Accesat: 06-mai-2025].
- [21] Energy Wise Solutions, „EnergyManager: Soluții de control energetic pentru industria ospitalității,” 2023. [Online]. Disponibil la: <https://www.energywise.com>. [Accesat: 06-mai-2025].
- [22] Proidea, „Sisteme inteligente de ventilație pentru spații comerciale,” 2023. [Online]. Disponibil la: <https://www.proidea.ro>. [Accesat: 06-mai-2025].
- [23] J. Reda, *Smart Surveillance in Hospitality: Enhancing Safety and Compliance*, Hospitality Tech Publishing, 2022.
- [24] D. Holmes, *Effective Technology Integration in Restaurants*, Springer, 2021.
- [25] J. Devlin et al., „BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *arXiv preprint*, 2018. [Online]. Disponibil la: <https://arxiv.org/abs/1810.04805>. [Accesat: 06-mai-2025].
- [26] OpenAI, „GPT-4 Technical Report,” 2023. [Online]. Disponibil la: <https://openai.com/research/gpt-4>. [Accesat: 06-mai-2025].
- [27] A. Vaswani et al., „Attention is All You Need,” *Advances in Neural Information Processing Systems*, 2017.
- [28] M. Tan și Q.V. Le, „EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” *arXiv preprint*, 2019. [Online]. Disponibil la: <https://arxiv.org/abs/1905.11946>. [Accesat: 06-mai-2025].
- [29] Datategy, „AI-powered virtual assistants: enhancing user interactions,” 2024. [Online]. Disponibil la: <https://www.datategy.net>. [Accesat: 06-mai-2025].
- [30] A. Esteva et al., „Dermatologist-level classification of skin cancer with deep neural networks,” *Nature*, vol. 542, pp. 115-118, 2017.
- [31] H. Zhang și Y. Li, „AI in Financial Fraud Detection: A Review,” *IEEE Access*, vol. 8, pp. 76710-76723, 2020.
- [32] S. M. Lee, „Virtual Fashion Modeling with AI,” *International Journal of Fashion Technology*, vol. 6, no. 1, pp. 10-17, 2021.
- [33] T. Wolf et al., „Transformers: State-of-the-Art Natural Language Processing,” *Proceedings of EMNLP*, 2020.
- [34] S. Ruder, „An Overview of Multi-Task Learning in Deep Neural Networks,” *arXiv preprint*, 2017.
- [35] Hugging Face, „Hugging Face Model Hub,” 2024. [Online]. Disponibil la: <https://huggingface.co/models>. [Accesat: 06-mai-2025].
- [36] M. Shoenberger et al., „Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism,” *arXiv preprint*, 2019.
- [37] B. Friedman și H. Nissenbaum, „Bias in Computer Systems,” *ACM Transactions on Information Systems*, vol. 14, no. 3, pp. 330-347, 1996.
- [38] F. Doshi-Velez și B. Kim, „Towards a Rigorous Science of Interpretable Machine Learning,” *arXiv preprint*, 2017.
- [39] Espressif Systems, *ESP32-S3-MINI-1 & MINI-1U Datasheet*, [Online]. Disponibil la: https://www.espressif.com/sites/default/files/documentation/esp32-s3-mini-1_mini-1u_datasheet_en.pdf [Accesat: 06-mai-2025].
- [40] FRAMOS GmbH, *Intel RealSense Depth Camera D435 – Informații produs*, [Online]. Disponibil la: <https://www.amos.com/en/products/depth-camera-d435-camera-only-20802>. [Accesat: 6-mai-2025].

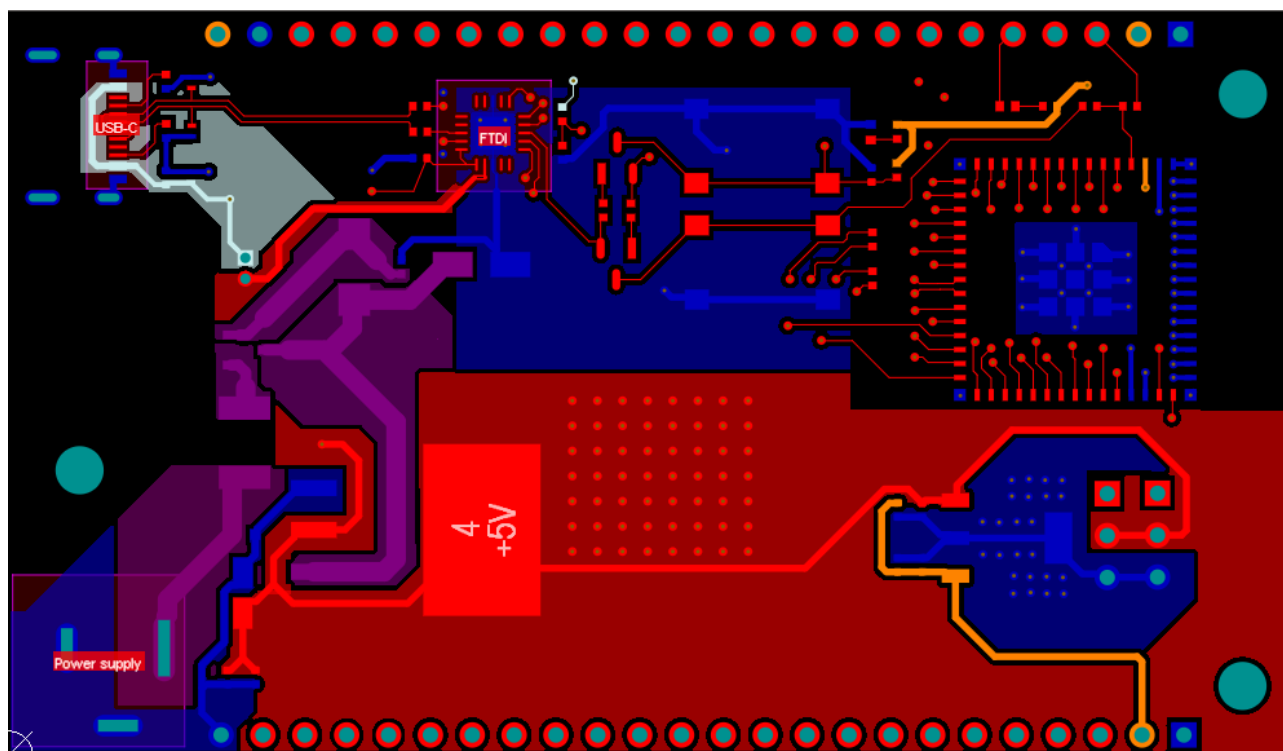
- [41] Intel Corporation, *Intel RealSense Depth Camera D435i – Specificații tehnice*, [Online]. Disponibil la: <https://www.intelrealsense.com/depth-camera-d435i/>. [Accesat: 6-mai-2025].
- [42] Intel Corporation, *Intel RealSense D400 Series Datasheet*, [Online]. Disponibil la: <https://www.intelrealsense.com/d400-series-datasheet/>. [Accesat: 6-mai-2025].
- [43] Intel Corporation, *Intel RealSense SDK 2.0 – Documentație oficială*, [Online]. Disponibil la: <https://dev.intelrealsense.com/docs/sdk2>. [Accesat: 6-mai-2025].
- [44] ETC2, *Fișă tehnică MG996R – Datasheet oficial*, [Online]. Disponibil la: <https://www.alldatasheet.com/datasheet-pdf/view/1131873/ETC2/MG996R.html>. [Accesat: 6-mai-2025].
- [45] TowerPro, *MG996R Servo Motor – Manual utilizator*, [Online]. Disponibil la: <https://www.framos.com/en/products/depth-camera-d435-camera-only-20802>. [Accesat: 6-mai-2025].
- [46] TowerPro, *MG996R Servo Datasheet*, [Online]. Disponibil la: <https://servodatabase.com/servo/towerpro/mg996r>. [Accesat: 6-mai-2025].
- [47] TowerPro, *MG996R Control Specifications*, [Online]. Disponibil la: <https://www.rcmodelreviews.com/mg996rreview.shtml>. [Accesat: 6-mai-2025].
- [48] RS PRO, *Adaptor AC/DC 12V 2A – Fișă tehnică*, [Online]. Disponibil la: <https://docs.rs-online.com/9967/A700000008914878.pdf>. [Accesat: 6-mai-2025].
- [49] Texas Instruments, *LM1085ISX-5.0/NOPB – Fișă tehnică și informații produs*, [Online]. Disponibil la: <https://www.digikey.com/en/products/detail/texas-instruments/LM1085ISX-5.0-NOPB/366711>. [Accesat: 6-mai-2025].
- [50] Texas Instruments, *LM1085 3-A Low Dropout Positive Regulators Datasheet*, [Online]. Disponibil la: <https://www.ti.com/lit/ds/symlink/lm1085.pdf>. [Accesat: 6-mai-2025].
- [51] Texas Instruments, *TL1963A-33DCYR – Fișă tehnică și informații produs*, [Online]. Disponibil la: <https://www.digikey.sg/en/products/detail/texas-instruments/TL1963A-33DCYR/2118656>. [Accesat: 6-mai-2025].
- [52] Texas Instruments, *TL1963A-xx 1.5-A Low-Noise Fast-Transient-Response Low-Dropout Regulator Datasheet*, [Online]. Disponibil la: <https://www.ti.com/lit/ds/symlink/tl1963a.pdf>. [Accesat: 6-mai-2025].
- [53] Future Technology Devices International Ltd., *FT231X – USB to Full Handshake UART IC Datasheet*, [Online]. Disponibil la: https://ftdichip.com/wp-content/uploads/2021/10/DS_FT231X.pdf. [Accesat: 6-mai-2025].
- [54] Molex, *Fișă tehnică pentru conectorul 217179-0001*, [Online]. Disponibil la: https://media.distrelec.com/Web/Downloads/_t/ds/217179-0001_eng_tds.pdf. [Accesat: 6-mai-2025].

Anexe

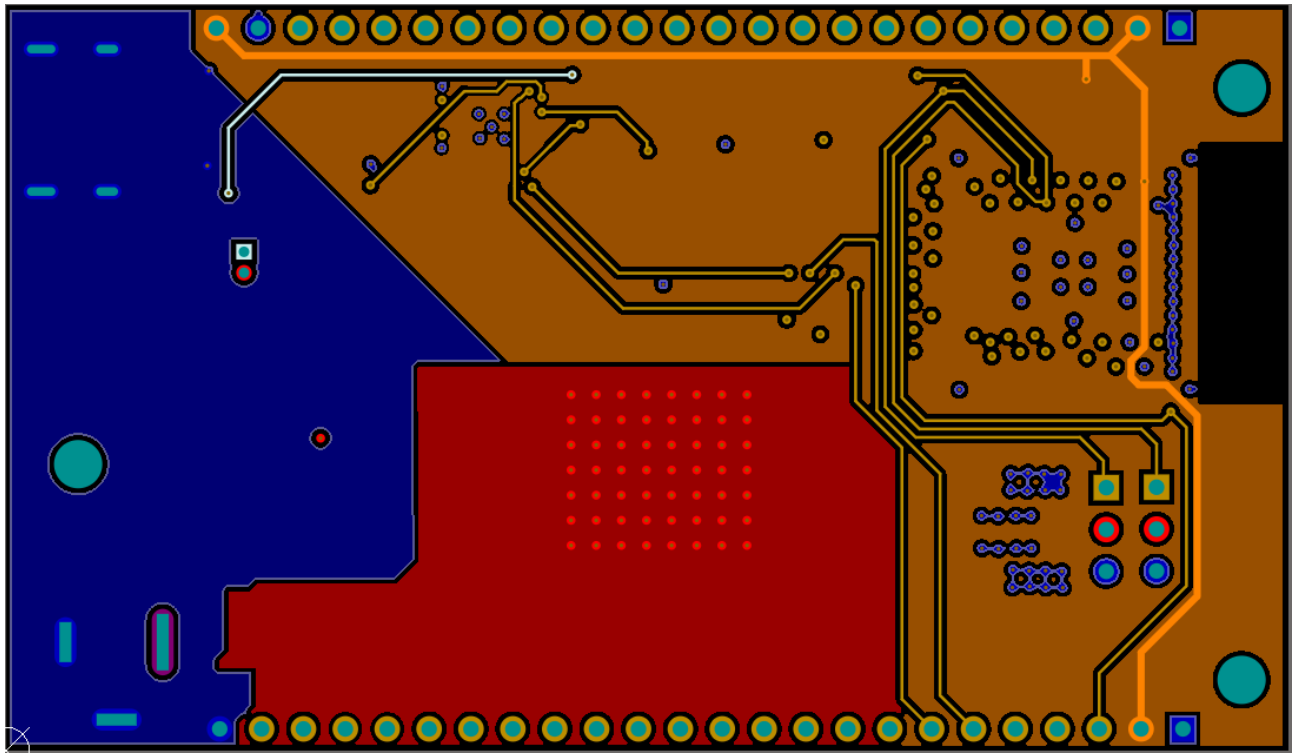
Anexa 1 – Schema Electrica



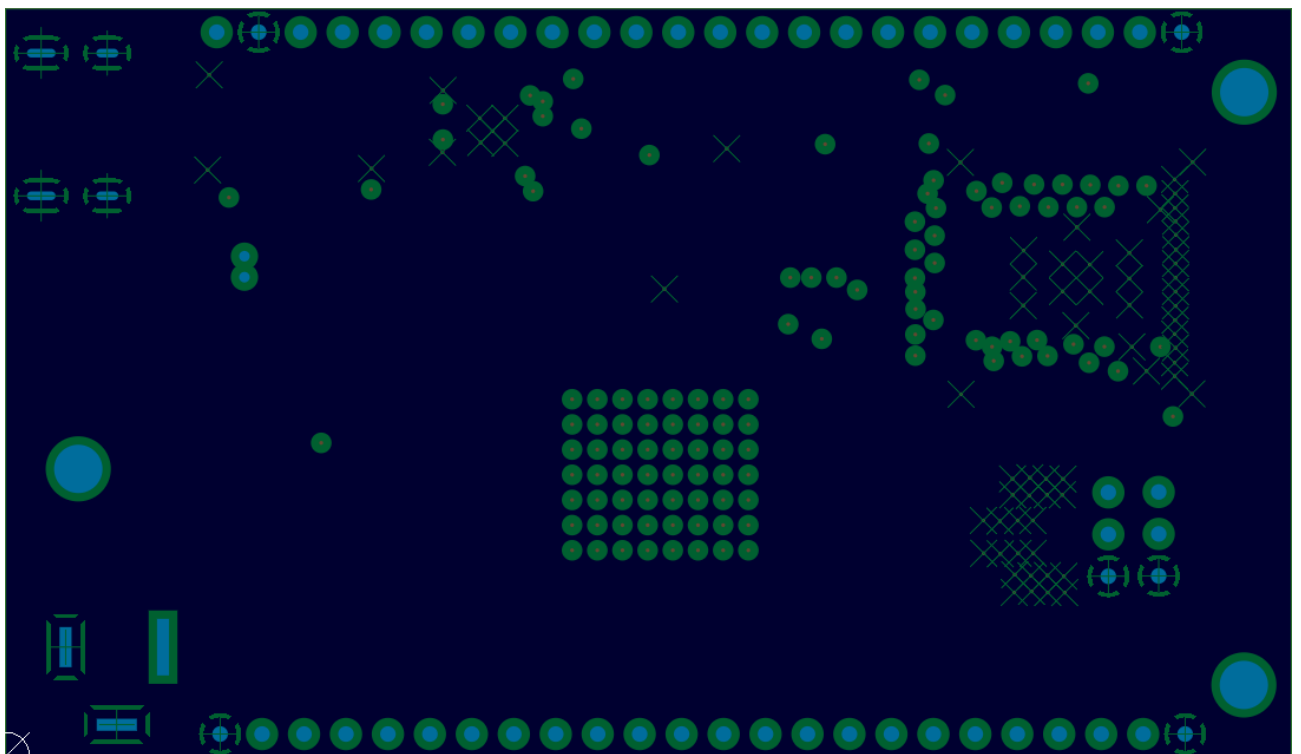
Anexa 2 – Layout-ul PCB-ului



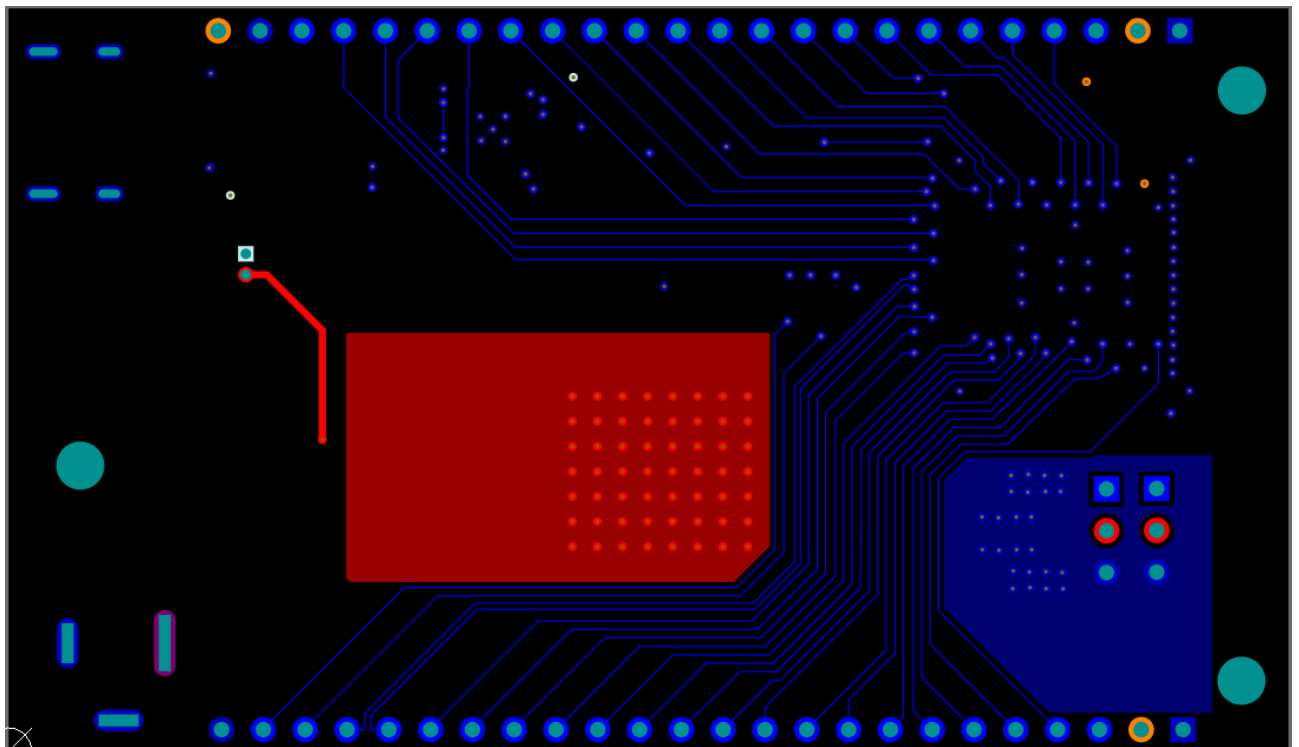
Layer 1



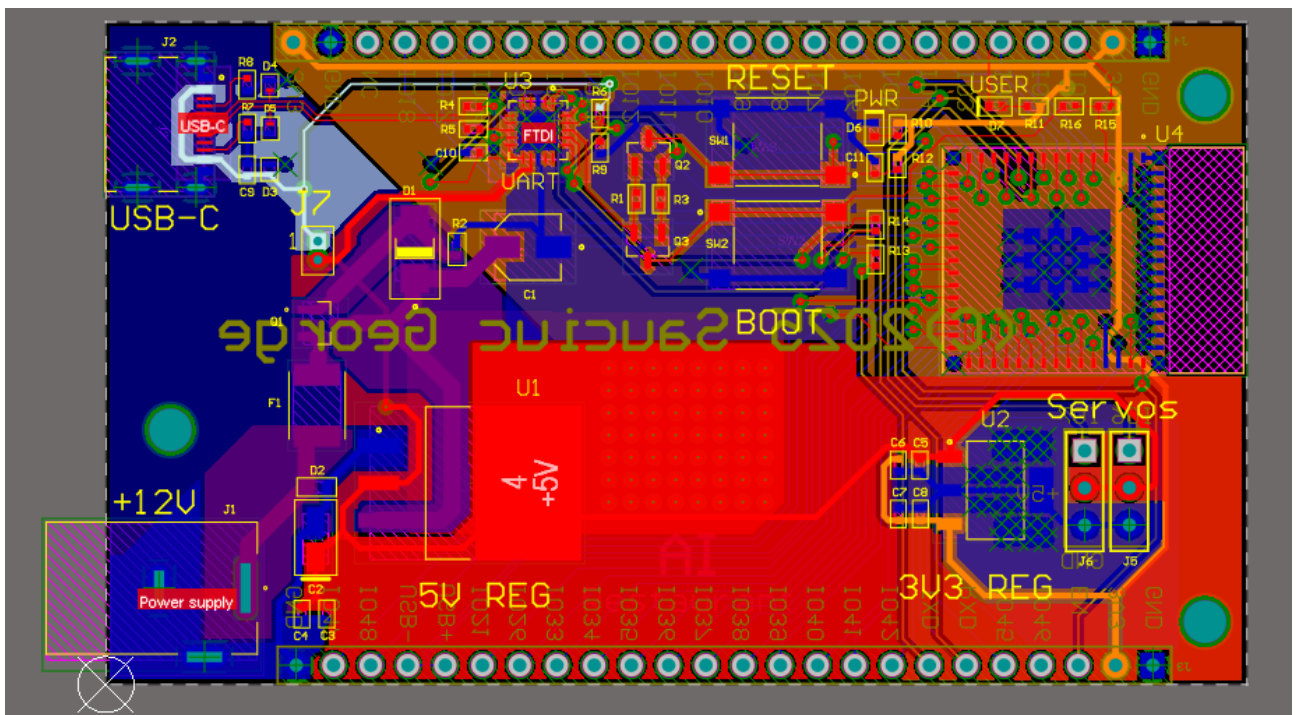
Layer 2



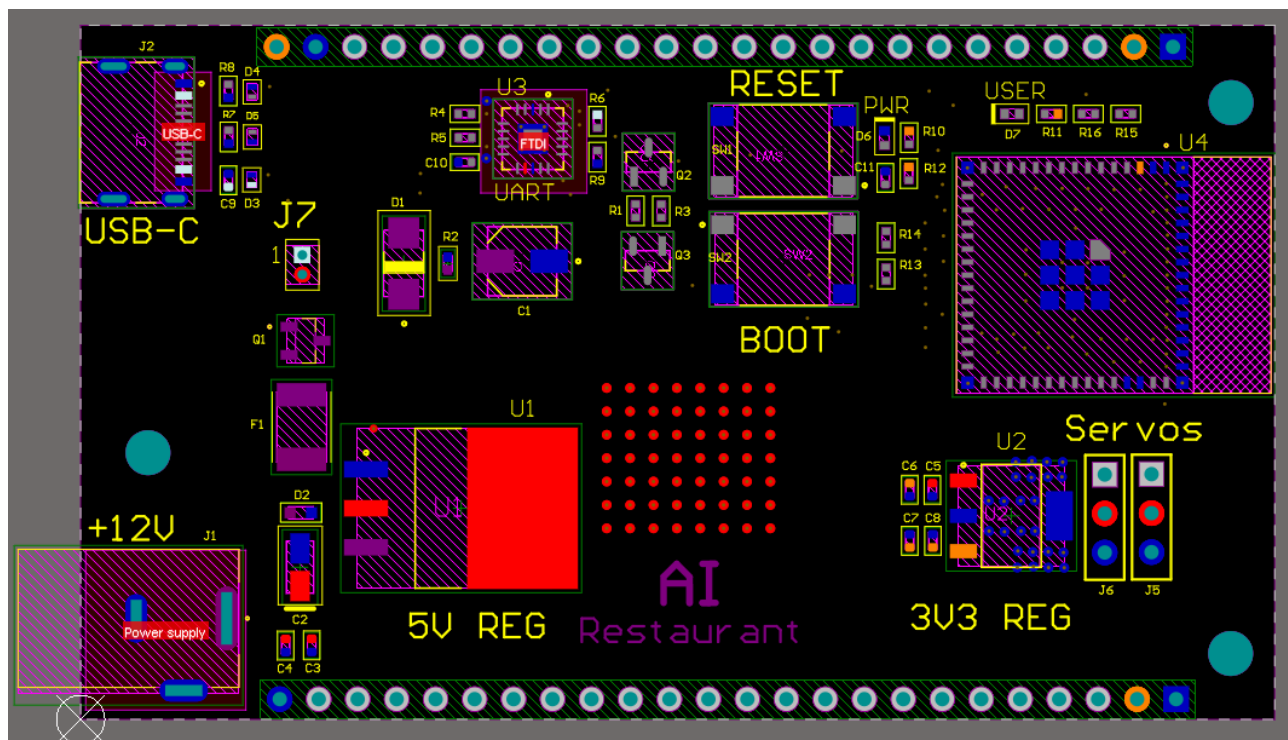
Layer 3



Layer 4

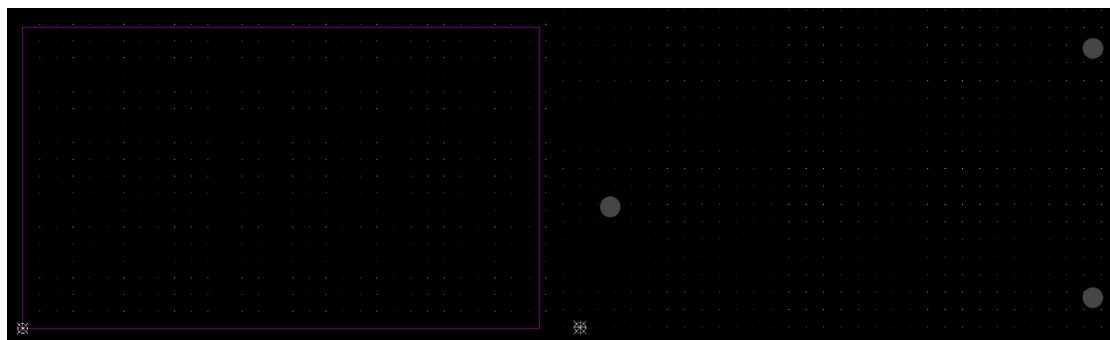


Vizualizarea completă a plăcii PCB



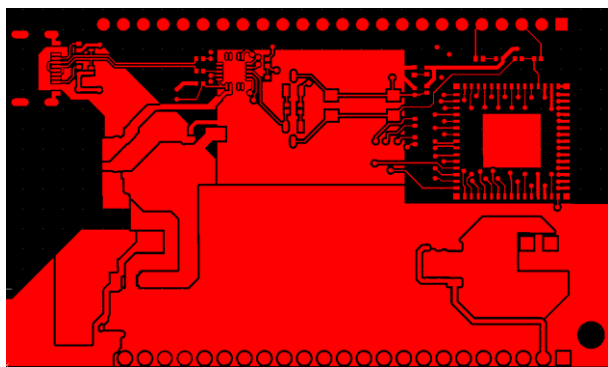
Vizualizarea stratului de componente

Anexa 3 – Fișiere Gerber

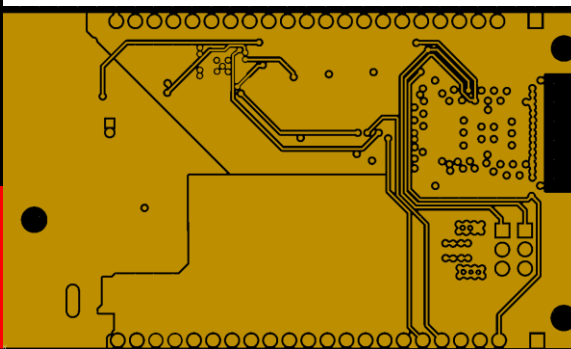


Fișierul de contur al plăcii

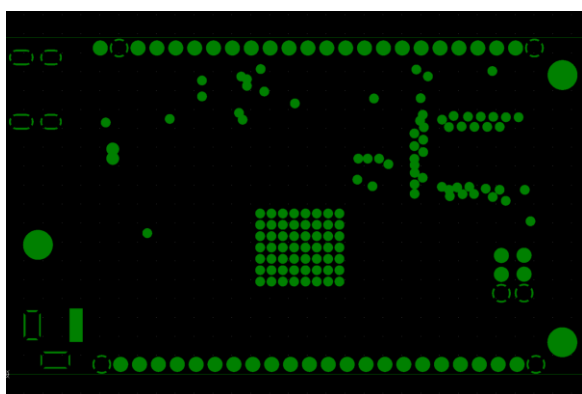
Strat de găuri ne-metalizate



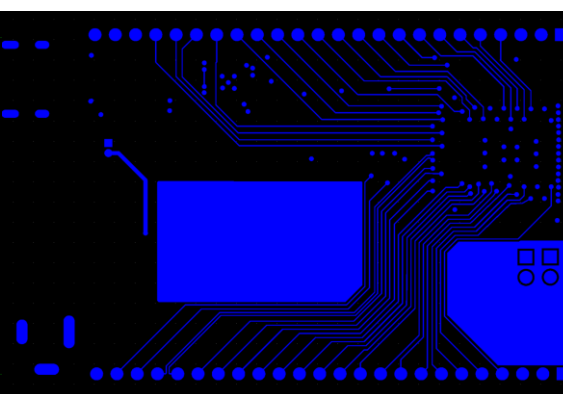
Strat de semnal – parte superioară



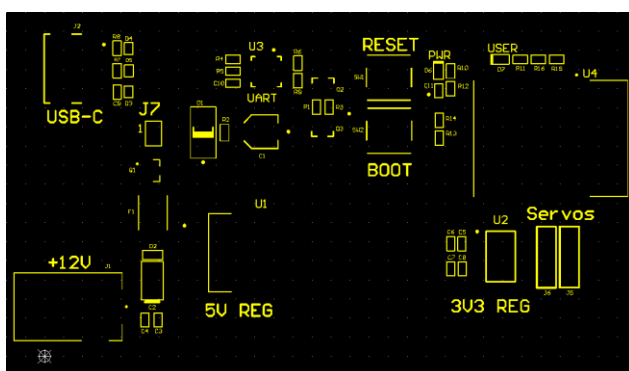
Strat de semnal – intern 1



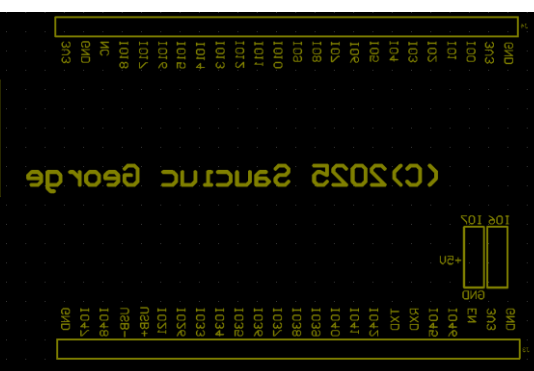
Strat de plan de masă



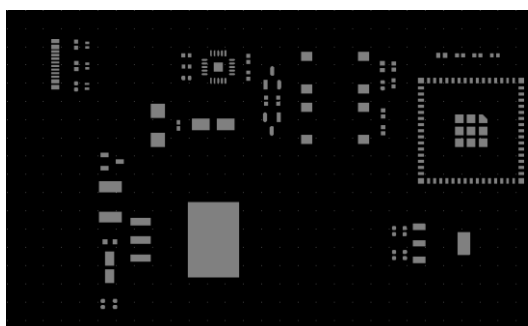
Strat de semnal – parte inferioară



Strat de serigrafie – parte superioară



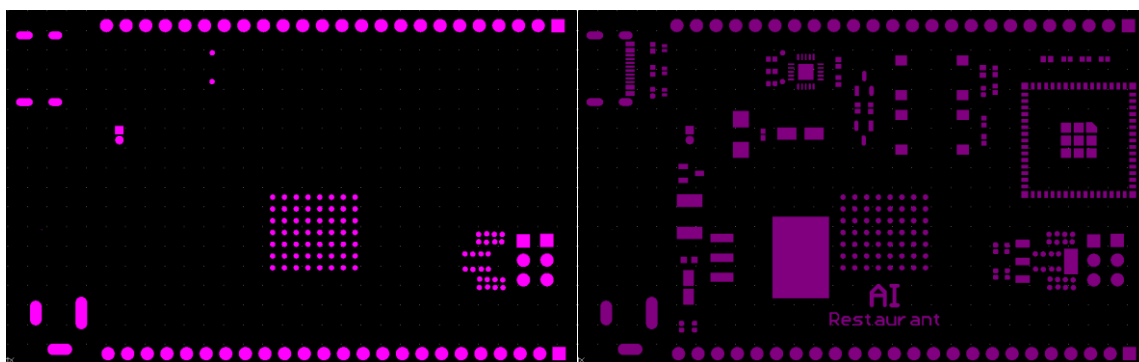
Strat de serigrafie – parte inferioară



Strat de pastă – parte superioară



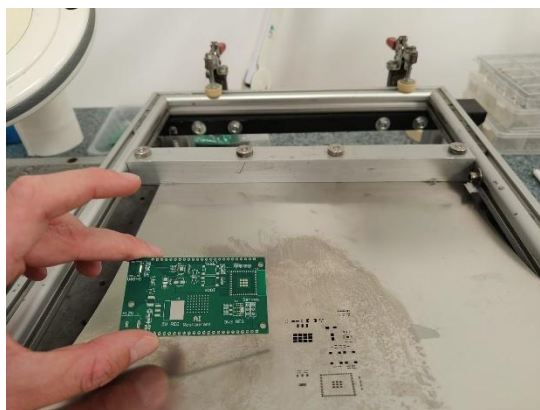
Strat de găuri metalizate (PTH)



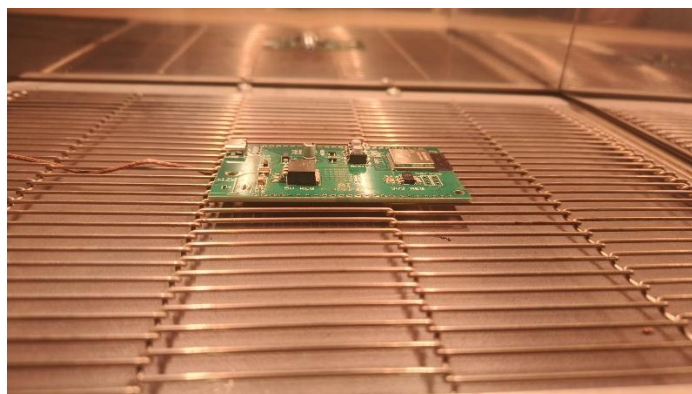
Strat de mască de lipire – parte inferioară

Strat de mască de lipire – parte superioară

Anexa 4 – Realizarea fizică a PCB-ului



Aplicarea pastei de lipit pe PCB



Reflow în cuptorul cu convecție



Verificare și retușare sub microscop

Anexa 5 – Codul sursă al aplicației software și al modulului hardware

Această anexă oferă informații referitoare la disponibilitatea codului sursă utilizat în cadrul proiectului de licență. Atât aplicația principală care rulează pe PC, responsabilă de procesarea imaginilor și de interfața grafică de administrare, cât și codul firmware încărcat pe microcontrolerul plăcii PCB (bazat pe ESP32-S3-MINI), sunt puse la dispoziție într-un repository public pe platforma GitHub.

Accesul complet la întregul cod sursă, organizat pe module, poate fi realizat prin următorul link:

<https://github.com/CptGeo02/licenta.git>

Fișierul *admin_gui.py*

```
from src.libs import *
from src.gui.modes.run_camera import run_camera
from src.gui.modes.run_video import run_video
from src.detectors.yolo_detector import YoloDetector
from src.gui.frames.display_frame import display_frame
```

```

from src.gui.frames.schedule_frame import ScheduleFrame
from src.gui.frames.calendar_selector_frame import CalendarSelectorFrame
from src.gui.frames.move_camera_frame import MoveCameraFrame

class AdminGUI(tk.Toplevel):
    def __init__(self, main_gui):
        super().__init__()
        self.name="admin"
        self.main_gui = main_gui
        self.title("AI Restaurant Monitoring System")
        self.geometry(f"{self.winfo_screenwidth()}x{self.winfo_screenheight()}")
        self.protocol("WM_DELETE_WINDOW", self.on_close)

        self.prev_time = time.time()
        self.last_time = time.time() # Momentul în care a fost actualizat ultima
data # Inițializează YOLO Detector cu un model implicit
        self.current_model = "models/yolov8n.pt"
        self.detector = YoloDetector()
        self.selected_camera = 0
        self.video_resolution = (640, 480)
        # Creează un frame pentru butoane
        self.button_frame = ttk.Frame(self)
        self.button_frame.pack(pady=10)

        self.button_back = ttk.Button(self.button_frame, text="Back",
command=self.go_back)
        self.button_back.grid(row=0, column=0, padx=5, pady=5)

        # Button: Move Camera
        self.move_camera_btn = ttk.Button(self.button_frame, text="Move Camera",
command=self.open_move_camera)
        self.move_camera_btn.grid(row=0, column=1, padx=5, pady=5)

        # Adăugarea elementelor în button_frame utilizând grid
        self.start_camera_btn = ttk.Button(self.button_frame, text="Start Live
Camera", command=self.start_camera)
        self.start_camera_btn.grid(row=0, column=2, padx=5, pady=5)

        self.select_video_btn = ttk.Button(self.button_frame, text="Select Video",
command=self.select_video)
        self.select_video_btn.grid(row=0, column=3, padx=5, pady=5)

        self.images_btn = ttk.Button(self.button_frame, text="Show Images",
command=self.show_images)
        self.images_btn.grid(row=0, column=4, padx=5, pady=5)

        # Selector de model YOLO
        self.model_var = StringVar(value=self.current_model)
        self.model_selector = ttk.OptionMenu(
            self.button_frame,
            self.model_var,
            "Select Model", # Valoarea implicită afișată
            *self.get_model_files(),
            command=self.change_model
        )
        self.model_selector.config(width=15)
        self.model_selector.grid(row=0, column=5, padx=5, pady=5)

        # Butoane de navigare, detectare și resetare mese
        self.previous_btn = ttk.Button(self.button_frame, text="Previous",
command=self.previous_image, state="disabled")
        self.previous_btn.grid(row=0, column=6, padx=5, pady=5)

```

```

        self.next_btn = ttk.Button(self.button_frame, text="Next",
command=self.next_image, state="disabled")
        self.next_btn.grid(row=0, column=7, padx=5, pady=5)

        self.detect_all_btn = ttk.Button(self.button_frame, text="Detect all",
command=self.detect_all, state="disabled")
        self.detect_all_btn.grid(row=0, column=8, padx=5, pady=5)

        self.auto_detect_enabled = BooleanVar(value=False)
        self.auto_detect_switch = Checkbutton(
            self.button_frame,
            text="Auto-Detect",
            variable=self.auto_detect_enabled,
            onvalue=True,
            offvalue=False,
            state="disabled"
        )
        self.auto_detect_switch.grid(row=0, column=9, padx=5, pady=5)

        self.illumination_enabled = BooleanVar(value=False)
        self.illumination_switch = Checkbutton(
            self.button_frame,
            text="Light Correction",
            variable=self.illumination_enabled,
            onvalue=True,
            offvalue=False
        )
        self.illumination_switch.grid(row=0, column=10, padx=5, pady=5)

        # Crearea butonului în interfața Tkinter
        self.export_button = ttk.Button(self.button_frame, text="Export
Parameters", command=self.export_parameters)
        self.export_button.grid(row=1, column=0, padx=5, pady=5)

        self.detect_tables_btn = ttk.Button(self.button_frame, text="Detect
Tables", command=self.detect_tables, state="disabled")
        self.detect_tables_btn.grid(row=1, column=1, padx=5, pady=5)

        self.set_tables_btn = ttk.Button(self.button_frame, text="Set Tables",
command=self.set_tables, state="disabled")
        self.set_tables_btn.grid(row=1, column=2, padx=5, pady=5)

        self.reset_tables_btn = ttk.Button(self.button_frame, text="Reset Tables",
command=self.reset_tables, state="disabled")
        self.reset_tables_btn.grid(row=1, column=3, padx=5, pady=5)

        self.stop_detection_btn = ttk.Button(self.button_frame, text="Stop
Detection", command=self.stop_detection, state="disabled")
        self.stop_detection_btn.grid(row=1, column=4, padx=5, pady=5)

        # Add a button to open the schedule window
        self.open_schedule_button = ttk.Button(self.button_frame, text="Set Weekly
Schedule", command=self.open_schedule_window)
        self.open_schedule_button.grid(row=1, column=5, padx=5, pady=5)
        # Add Generate Statistics Button
        self.btn_generate_statistics = ttk.Button(self.button_frame,
text="Generate Statistics", command=self.open_statistics_calendar)
        self.btn_generate_statistics.grid(row=1, column=6, padx=5, pady=5)

        # Obține camerele disponibile
        self.camera_list = self.get_available_cameras()
        self.move_camera_frame = None
        # Variabila pentru selectorul de camere
        self.camera_var = tk.StringVar(value="Select Camera") # Valoare implicită

```

```

# Selectorul pentru camere
self.camera_selector = ttk.OptionMenu(
    self.button_frame,
    self.camera_var,
    "Select Camera", # Opțiunea implicită
    *self.camera_list, # Opțiuni de camere
    command=self.update_camera
)
self.camera_selector.config(width=20)
self.camera_selector.grid(row=2, column=0, padx=5, pady=5)

self.video_format_var = StringVar(value="480p") # Valoare implicită
self.video_format_selector = ttk.OptionMenu(
    self.button_frame,
    self.video_format_var,
    "Select Format", # Valoare implicită
    "480p", "720p", "1080p", # Opțiuni de formate
    command=self.update_video_format
)

self.video_format_selector.config(width=15)
self.video_format_selector.grid(row=2, column=1, padx=5, pady=5)

# Creează un frame pentru butoane
self.max_frame = ttk.Frame(self)
self.max_frame.pack(pady=10)

# Variabile pentru timpii selectați pentru fiecare status
self.time_available = tk.StringVar(value='Select Max Time Available')
self.time_ready = tk.StringVar(value='Select Max Time Waiting')
self.time_eating = tk.StringVar(value='Select Max Time Eating')
self.time_clean = tk.StringVar(value='Select Max Time Clean')
self.max_people_var = tk.StringVar(value='')

# Crearea selectoarelor
self.create_time_selectors()

# Crearea input-ului pentru numărul maxim de oameni
self.create_people_number_input()

# Frame pentru afișarea informațiilor de performanță
self.performance_frame = ttk.Frame(self)
self.performance_frame.pack(pady=10)

# Adăugarea etichetelor în performance_frame, utilizând grid
self.fps_label = ttk.Label(self.performance_frame, text="FPS:
Calculating...")
self.fps_label.grid(row=0, column=0, padx=10, pady=5, sticky=tk.W)

self.cpu_label = ttk.Label(self.performance_frame, text="CPU Usage:
Calculating...")
self.cpu_label.grid(row=0, column=1, padx=10, pady=5, sticky=tk.W)

self.gpu_label = ttk.Label(self.performance_frame, text="GPU Usage: Not
Available")
self.gpu_label.grid(row=0, column=2, padx=10, pady=5, sticky=tk.W)

self.ram_label = ttk.Label(self.performance_frame, text="RAM Usage:
Calculating...")
self.ram_label.grid(row=0, column=3, padx=10, pady=5, sticky=tk.W)

# Frame pentru toate slidererele
sliders_frame = ttk.Frame(self)
sliders_frame.pack(pady=10)

```

```

# Variabile pentru slidere
self.overlap_threshold_var = tk.DoubleVar(value=0.2)
self.red_threshold_var = tk.DoubleVar(value=0.1)
self.blue_threshold_var = tk.DoubleVar(value=0.1)

# Slider pentru Overlap Threshold
self.create_slider(
    parent=sliders_frame,
    label_text="Overlap Threshold:",
    variable=self.overlap_threshold_var,
    command=self.update_overlap_threshold,
    default_value=0.2,
    column=0,
)

# Slider pentru Red Threshold
self.create_slider(
    parent=sliders_frame,
    label_text="Red Threshold:",
    variable=self.red_threshold_var,
    command=self.update_red_threshold,
    default_value=0.1,
    column=1,
)

# Slider pentru Blue Threshold
self.create_slider(
    parent=sliders_frame,
    label_text="Blue Threshold:",
    variable=self.blue_threshold_var,
    command=self.update_blue_threshold,
    default_value=0.1,
    column=2,
)

self.clahe_clip_var = tk.DoubleVar(value=2.0)
self.clahe_tile_var = tk.IntVar(value=8)
self.create_slider_special(
    parent=sliders_frame,
    label_text="CLAHE Clip Limit:",
    variable=self.clahe_clip_var,
    command=self.update_clahe_clip,
    default_value=2.0,
    column=3
)

self.create_slider_special(
    parent=sliders_frame,
    label_text="CLAHE Tile Size:",
    variable=self.clahe_tile_var,
    command=self.update_clahe_tile,
    default_value=8,
    column=4
)

self.info_label = tk.Label(self, text="Tables: 0 | People: 0")
self.info_label.pack(side=tk.TOP, fill=tk.X)

# Etichetă pentru scorul de frame
self.score_label = tk.Label(self, text="Frame Score: 0.00")
self.score_label.pack(side=tk.TOP, fill=tk.X)
self.start_info_update()

# Atribute pentru calculul FPS

```

```

self.prev_time = time.time()
self.fps = 0

# Actualizare periodică a informațiilor de performanță
self.update_performance()
# Alți parametri de inițializare
self.canvas = Canvas(self, width=1920, height=1080)
self.canvas.pack(side=tk.TOP, fill=tk.BOTH, expand=True)

self.label = ttk.Label(self.canvas)
self.label.pack()

self.status_label = ttk.Label(self, text="", wraplength=1000)
self.status_label.pack(side=tk.TOP, pady=10)

self.video_source = None
self.current_frame = None
self.running = False

self.camera_thread = None
self.video_thread = None
self.image_thread = None

self.images = []
self.image_index = 0
self.stop_event = threading.Event()

self.update_frame()
apply_modern_style(self)

def create_slider_special(self, parent, label_text, variable, command,
default_value, column):
    """
    Creează un slider generic cu etichetă, tooltip și afișarea valorii curente.
    Asigură intervale sigure pentru CLAHE (contrast și tile size).
    """
    frame = ttk.Frame(parent)
    frame.grid(row=0, column=column, padx=10)

    label = ttk.Label(frame, text=label_text)
    label.pack(pady=5)

    # Determinăm limitele în funcție de tipul variabilei și label
    if isinstance(variable, tk.DoubleVar) and "Clip" in label_text:
        from_, to_ = 1.0, 5.0 # CLAHE Clip Limit recomandat
        tooltip_text = "Clip Limit controlează cât de puternic se aplică
contrastul local. Valori mai mari evidențiază detaliile, dar pot genera zgomot
vizual."
    elif isinstance(variable, tk.IntVar) and "Tile" in label_text:
        from_, to_ = 4, 32 # CLAHE Tile Size (X=Y) recomandat
        tooltip_text = "Dimensiunea Tile controlează granularitatea corecției
locale. Tile mai mic = contrast local fin, Tile mai mare = efect global."
    else:
        from_, to_ = 0.0, 1.0
        tooltip_text = "Slider generic."

    slider = ttk.Scale(
        frame,
        from_=from_,
        to=to_,
        orient="horizontal",
        length=150,
        variable=variable,
        command=command,

```

```

)
slider.pack(pady=5)

        value_label = ttk.Label(frame, text=f"{default_value:.2f}" if
isinstance(default_value, float) else str(default_value))
        value_label.pack(pady=5)

# Tooltip informativ
Hovertip(label, tooltip_text, hover_delay=300)

# Salvează eticheta pentru actualizare dinamică
variable.value_label = value_label

def update_performance(self):
# Exemplu de calcul FPS
# Afișare FPS
self.fps_label.config(text=f"FPS: {self.fps:.2f}")

# Utilizare CPU și RAM folosind psutil
cpu_usage = psutil.cpu_percent()
ram_usage = psutil.virtual_memory().percent
self.cpu_label.config(text=f"CPU Usage: {cpu_usage}%")
self.ram_label.config(text=f"RAM Usage: {ram_usage}%")
# Verificare și afișare utilizare GPU dacă este disponibil
if torch.cuda.is_available():
    gpu_usage, gpu_memory_used, gpu_memory_free = self.get_gpu_usage()
    if gpu_usage is not None:
        self.gpu_label.config(
                                text=f"GPU Usage: {gpu_usage}% (Used:
{gpu_memory_used}/{gpu_memory_free} MB)"
        )

# Programare actualizare periodică a informațiilor (la fiecare 1 secundă)
self.after(1000, self.update_performance)
def open_move_camera(self):
    """
    Deschide frame-ul Move Camera pe un thread separat, fără a bloca
    execuția principală.
    """
    move_camera_window = tk.Toplevel(self)
    move_camera_window.title("Move Camera")
    self.move_camera_frame = MoveCameraFrame(self, move_camera_window)
    self.move_camera_frame.pack(fill="both", expand=True)
def get_gpu_usage(self):
    try:
        # Apelăm nvidia-smi pentru a obține date despre GPU
        result = subprocess.run(
                                ['nvidia-smi', '--query-
gpu=memory.used,memory.free,utilization.gpu', '--format=csv,noheader,nounits'],
                                stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True
        )
        # Extragem valorile din rezultatul comenzii
        output = result.stdout.strip().split('\n')[0].split(', ')
        memory_used = int(output[0]) # MB
        memory_free = int(output[1]) # MB
        gpu_usage = int(output[2]) # procentaj utilizare GPU
        return gpu_usage, memory_used, memory_free
    except Exception as e:
        print(f"Error getting GPU usage: {e}")
        return None, None, None

def get_model_files(self):
    """Returnează o listă de fișiere YOLO disponibile în folderul models."""
    model_files = [f for f in os.listdir("models") if f.endswith(".pt")]

```

```

return model_files

def change_model(self, model_name):
    """Schimbă modelul YOLO utilizat."""
    self.current_model = model_name
    self.detector.load_model(os.path.join("models", self.current_model))
    self.status_label.config(text=f"Modelul a fost schimbat la {model_name}")

def update_button_states(self, selected_btn):
    if selected_btn == "camera":
        self.auto_detect_switch.config(state="normal")
        self.detect_all_btn.config(state="normal")
        self.start_camera_btn.config(state="disabled")
        self.select_video_btn.config(state="normal")
        self.images_btn.config(state="normal")
        self.previous_btn.config(state="disabled")
        self.next_btn.config(state="disabled")

    if selected_btn == "video":
        self.auto_detect_switch.config(state="normal")
        self.detect_all_btn.config(state="normal")
        self.start_camera_btn.config(state="normal")
        self.select_video_btn.config(state="disabled")
        self.images_btn.config(state="normal")
        self.previous_btn.config(state="disabled")
        self.next_btn.config(state="disabled")

    if selected_btn == "show_images":
        self.auto_detect_switch.config(state="normal")
        self.detect_all_btn.config(state="normal")
        self.start_camera_btn.config(state="normal")
        self.select_video_btn.config(state="normal")
        self.images_btn.config(state="disabled")
        self.previous_btn.config(state="normal")
        self.next_btn.config(state="normal")

    if selected_btn == "detect_all":
        self.detect_tables_btn.config(state="normal")
        self.set_tables_btn.config(state="disabled")
        self.reset_tables_btn.config(state="disabled")
        self.stop_detection_btn.config(state="normal")

    if selected_btn == "detect_tables":
        self.detect_tables_btn.config(state="disabled")
        self.set_tables_btn.config(state="normal")
        self.reset_tables_btn.config(state="disabled")
        self.stop_detection_btn.config(state="normal")

    if selected_btn == "set_tables":
        self.detect_tables_btn.config(state="disabled")
        self.set_tables_btn.config(state="disabled")
        self.reset_tables_btn.config(state="normal")
        self.stop_detection_btn.config(state="normal")

    if selected_btn == "reset_tables":
        self.detect_tables_btn.config(state="disabled")
        self.set_tables_btn.config(state="normal")
        self.reset_tables_btn.config(state="disabled")
        self.stop_detection_btn.config(state="normal")

    if selected_btn == "stop_detection":
        self.detect_tables_btn.config(state="normal")
        self.set_tables_btn.config(state="disabled")
        self.reset_tables_btn.config(state="disabled")

```

```

        self.stop_detection_btn.config(state="disabled")

def start_camera(self):
    self.stop_running_thread()
    self.stop_event.clear()
    self.running = True
    self.camera_thread = threading.Thread(target=run_camera, args=(self,))
    self.camera_thread.start()

    self.update_button_states('camera')

def select_video(self):
    video_path = filedialog.askopenfilename(filetypes=[("Video Files",
"*.*.mp4;*.avi;*.mov")])

    if video_path:
        self.stop_running_thread()
        self.stop_event.clear()
        self.running = True
        self.video_source = video_path
        self.video_thread = threading.Thread(target=run_video, args=(self,))
        self.video_thread.start()

    self.update_button_states('video')

def show_images(self):
    """Pornește afișarea unei singure imagini într-un thread separat."""
    self.stop_running_thread()
    self.stop_event.clear() # Resetare stop_event

    self.images = load_images('data/images/')
    self.image_index = 0

    if self.images:
        img_path = os.path.join('data/images/', self.images[self.image_index])
        self.current_frame = cv2.imread(img_path)

    self.update_button_states('show_images')

def next_image(self):
    if self.images:
        self.image_index = (self.image_index + 1) % len(self.images)
        img_path = os.path.join('data/images/', self.images[self.image_index])
        self.current_frame = cv2.imread(img_path)

def previous_image(self):
    if self.images:
        self.image_index = (self.image_index - 1) % len(self.images)
        img_path = os.path.join('data/images/', self.images[self.image_index])
        self.current_frame = cv2.imread(img_path)

def detect_all(self):
    self.detector.detecting_tables_only = False
    self.detector.done_setting_tables = False
    self.detector.detecting_all = True
    self.update_button_states('detect_all')

def detect_tables(self):
    self.detector.detecting_tables_only = True
    self.detector.done_setting_tables = False
    self.detector.detecting_all = False
    self.update_button_states('detect_tables')

```

```

def set_tables(self):
    self.detector.detecting_tables_only = False
    self.detector.done_setting_tables = True
    self.detector.detecting_all = False
    self.current_frame = self.detector.set_table_ids()
    self.detector.table_manager.create_new_files()
    self.detector.table_manager.start_auto_save()
    self.detector.people_manager.create_new_files()
    self.detector.people_manager.start_auto_save()
    self.update_button_states('set_tables')

def reset_tables(self):
    self.detector.detecting_tables_only = True
    self.detector.done_setting_tables = False
    self.detector.detecting_all = False
    self.detector.reset_table_manager()
    self.update_button_states('reset_tables')

def stop_detection(self):
    self.detector.detecting_tables_only = False
    self.detector.done_setting_tables = False
    self.detector.detecting_all = False
    self.detector.reset_table_manager()
    self.update_button_states('stop_detection')

def update_status_label(self):
    status_report = self.detector.get_tables_status_report()
    self.status_label.config(text=status_report)

def update_frame(self):
    """Actualizează și afișează cadrul la fiecare 17 ms."""
    display_frame(self, self.current_frame)
    self.after(17, self.update_frame) # Reapelează funcția la fiecare 17 ms

def stop_running_thread(self):
    """Oprește orice thread activ înainte de a porni unul nou."""
    self.stop_detection()
    self.stop_event.set() # Semnalăm oprirea thread-urilor
    self.running = False
    # Oprește thread camera
    if self.camera_thread and self.camera_thread.is_alive():
        self.camera_thread.join()
        self.camera_thread = None
        print("[INFO] camera_thread has stopped in Admin mode")
    # Oprește thread video
    if self.video_thread and self.video_thread.is_alive():
        self.video_thread.join()
        self.video_thread = None
        print("[INFO] video_thread has stopped in Admin mode")

def on_close(self):
    """Curăță toate apelurile 'after' și închide aplicația."""
    self.stop_running_thread()
    self.after_cancel(self.update_frame)
    self.after_cancel(self.start_info_update)
    self.destroy() # Închide aplicația

def generate_time_options(self, start_time, end_time, step_minutes):
    """
    Generăm intervalele de timp între start_time și end_time cu pasul
    step_minutes.
    """

```

```

start = datetime.strptime(start_time, "%H:%M:%S")
end = datetime.strptime(end_time, "%H:%M:%S")
step = timedelta(minutes=step_minutes)

times = []
current_time = start
while current_time <= end:
    times.append(current_time.strftime("%H:%M:%S"))
    current_time += step

return times

def get_time_options(self, status_type):
    """
    Funcție care returnează timpii disponibili pentru un anumit tip de status.
    """
    if status_type == 'available':
        return self.generate_time_options('00:00:00', '10:00:00', 30)
    elif status_type == 'ready to order':
        return self.generate_time_options('00:00:00', '01:00:00', 5)
    elif status_type == 'eating':
        return self.generate_time_options('00:00:00', '06:00:00', 30)
    elif status_type == 'need to clean':
        return self.generate_time_options('00:00:00', '02:00:00', 5)
    else:
        return []

def create_time_selectors(self):
    """
    Crează selectoarele de timp pentru fiecare status al mesei.
    """
    # Selector pentru "available"
    self.available_selector = self.create_time_selector('available',
self.time_available)

    # Selector pentru "ready to order"
    self.ready_selector = self.create_time_selector('ready to order',
self.time_ready)

    # Selector pentru "eating"
    self.eating_selector = self.create_time_selector('eating',
self.time_eating)

    # Selector pentru "need to clean"
    self.clean_selector = self.create_time_selector('need to clean',
self.time_clean)

def create_time_selector(self, status_type, time_var):
    """
    Creează un selector de timp pentru un anumit tip de status.
    """
    time_options = self.get_time_options(status_type)
    selector = ttk.OptionMenu(
        self.max_frame,
        time_var,
        f"Select max {status_type} time",
        *time_options,
        command=lambda _: self.update_time_status(status_type, time_var)
    )
    # Configurarea selectoarelor cu lățime și plasare
    selector.config(width=35)
    selector.pack(side=tk.LEFT, padx=5, pady=5)
    return selector

```

```

def update_time_status(self, status_type, time_var):
    """
    Funcție de comandă care salvează timpul ales pentru un anumit status.
    """
    time = time_var.get()
    print(f"Selected time for {status_type}: {time}")
    self.detector.table_manager.set_max_time(status_type, time)
    # Aici poți salva valorile în variabilele corespunzătoare sau le poți
    folosi în aplicație.

def create_people_number_input(self):
    # Setează valoarea inițială a câmpului de input din PeopleManager
    self.max_people_var =
tk.StringVar(value=str(self.detector.people_manager.get_max_people_number()))
    people_entry = ttk.Entry(self.max_frame, textvariable=self.max_people_var,
width=10, justify='center')

    # Validare: doar cifre
    validate_command = (self.register(self.validate_unsigned_int), '%P')
    people_entry.config(validate='key', validatecommand=validate_command)

    # Event pentru a actualiza valoarea când utilizatorul apasă Enter
    label = ttk.Label(self.max_frame, text="Max People:")
    label.pack(side=tk.LEFT, padx=5)

    people_entry.bind('<Return>', self.update_max_people)
    people_entry.pack(side=tk.LEFT, padx=5)

def validate_unsigned_int(self, value):
    """
    Permite doar valori unsigned int în câmpul de input.
    """
    if value == "":
        return True # Permite câmp gol temporar
    return value.isdigit()

def update_max_people(self, event=None):
    """
    Actualizează valoarea maximă de persoane în PeopleManager.
    """
    try:
        max_people = int(self.max_people_var.get())
        self.detector.people_manager.set_max_people_number(max_people)
        print(f"Max people number updated to: {max_people}")
    except ValueError as e:
        messagebox.showerror("Invalid Input", "Please enter a positive
integer.")
    # Resetează la valoarea actuală
    self.max_people_var.set(str(self.detector.people_manager.get_max_peo
ple_number()))

def start_info_update(self):
    """Inițiază actualizarea periodică a informațiilor."""
    self.update_info() # Actualizează imediat
    self.after(1000, self.start_info_update) # Reapelează peste 1 secundă

def update_info(self):
    """Actualizează textul pentru numărul de mese și persoane detectate."""
    self.info_label.config(
        text=f"Tables: {self.detector.tables_number} | "
        f"People: {self.detector.people_number}"
    )
    self.score_label.config(

```

```

        text=f"Frame Score: {self.detector.frame_score:.2f}" if
self.detector.detecting_tables_only
        or self.detector.done_setting_tables or self.detector.detecting_all
else "Frame Score: 0.00")

def create_slider(self, parent, label_text, variable, command, default_value,
column):
    """
    Creează un slider generic cu etichetă și afișarea valorii curente.
    """
    frame = ttk.Frame(parent) # Frame pentru fiecare slider
    frame.grid(row=0, column=column, padx=10) # Poziționează slider-ul pe
coloană

    label = ttk.Label(frame, text=label_text) # Eticheta slider-ului
    label.pack(pady=5)

    slider = ttk.Scale(
        frame,
        from_=0.0,
        to=1.0,
        orient="horizontal",
        length=150, # Lungimea slider-ului
        variable=variable,
        command=command,
    )
    slider.pack(pady=5)

    value_label = ttk.Label(frame, text=f"{default_value:.2f}") # Valoarea
curentă
    value_label.pack(pady=5)

    # Salvează eticheta pentru actualizare dinamică
    variable.value_label = value_label

def update_overlap_threshold(self, _):
    """
    Actualizează valoarea overlap_threshold din detector.
    """
    new_threshold = self.overlap_threshold_var.get()
    self.detector.overlap_threshold = new_threshold
    self.overlap_threshold_var.value_label.config(text=f"{new_threshold:.2f}")
")

def update_red_threshold(self, _):
    """
    Actualizează valoarea red_threshold pentru toate mesele.
    """
    new_threshold = self.red_threshold_var.get()
    self.detector.table_manager.set_red_threshold_for_all_tables(new_thresho
ld)
    self.red_threshold_var.value_label.config(text=f"{new_threshold:.2f}")

def update_blue_threshold(self, _):
    """
    Actualizează valoarea blue_threshold pentru toate mesele.
    """
    new_threshold = self.blue_threshold_var.get()
    self.detector.table_manager.set_blue_threshold_for_all_tables(new_thresh
old)
    self.blue_threshold_var.value_label.config(text=f"{new_threshold:.2f}")

def open_schedule_window(self):

```

```

        """Function to open the schedule frame in a new window."""
        schedule_window = tk.Toplevel()
        schedule_window.title("Set Weekly Schedule")
        schedule_window.geometry("600x400") # Adjust size as needed
        frame = ScheduleFrame(schedule_window)
        frame.pack(fill="both", expand=True, padx=10, pady=10)

    def open_statistics_calendar(self):
        """Function to open the statistics calendar window in a new window."""
        statistics_window = tk.Toplevel() # Creează o fereastră de tip Toplevel
        statistics_window.title("Select Date Range for Statistics") # Titlul ferestrei
        statistics_window.geometry("600x400") # Setează dimensiunile ferestrei

        # Creează frame-ul pentru calendar și alte componente
        statistics_frame = CalendarSelectorFrame(statistics_window) # Pass the
        Toplevel window to CalendarSelectorFrame
        statistics_frame.pack(fill="both", expand=True, padx=10, pady=10) #
        Adaugă frame-ul în fereastră

    def update_video_format(self, selected_format):
        """Actualizează dimensiunea video și afișează în consolă."""
        if selected_format == "480p":
            self.video_resolution = (640, 480) # Rezoluție 480p
        elif selected_format == "720p":
            self.video_resolution = (1280, 720) # Rezoluție HD
        elif selected_format == "1080p":
            self.video_resolution = (1920, 1080) # Rezoluție FullHD
        # Afișează dimensiunea selectată în consolă
        print(f"Selected video format:
        {self.video_resolution[0]}x{self.video_resolution[1]} pixels")

    def get_available_cameras(self):
        """Detectează camerele video disponibile și le returnează ca o
        listă."""
        camera_list = []
        for i in range(10):
            cap = cv2.VideoCapture(i, cv2.CAP_DSHOW)
            if cap is not None and cap.isOpened():
                camera_list.append(f"Camera {i}")
                cap.release()
        return camera_list

    def update_clahe_clip(self, _):
        value = self.clahe_clip_var.get()
        self.clahe_clip_var.value_label.config(text=f"{value:.2f}")

    def update_clahe_tile(self, _):
        value = self.clahe_tile_var.get()
        self.clahe_tile_var.value_label.config(text=str(value))

    def update_camera(self, selected_camera):
        """Actualizează camera aleasă și afișează în consolă."""
        if selected_camera != "Select Camera":
            print(f"Selected camera: {selected_camera}")
            self.selected_camera = int(selected_camera[-1])
        else:
            print("No camera selected")

    def go_back(self):
        """Închide această fereastră și revine la MainGUI."""
        self.stop_running_thread()
        self.destroy() # Închide fereastra curentă
        self.main_gui.deiconify() # Afișează din nou MainGUI

```

```

def export_parameters(self):
    # Crearea directorului dacă nu există
    config_dir = "data/config"
    os.makedirs(config_dir, exist_ok=True)

    # Crearea numelui de fișier
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    config_filename = f"config_{timestamp}.json"
    config_path = os.path.join(config_dir, config_filename)

    # Structura datelor
    config_data = {
        "selected_camera": self.selected_camera,
        "selected_model": self.model_var.get(),
        "video_resolution": self.video_resolution,
        "max_times": {
            "available": self.time_available.get(),
            "ready_to_order": self.time_ready.get(),
            "eating": self.time_eating.get(),
            "need_to_clean": self.time_clean.get()
        },
        "max_people": int(self.max_people_var.get()),
        "overlap_threshold": round(float(self.overlap_threshold_var.get()), 2),
        "red_threshold": round(float(self.red_threshold_var.get()), 2),
        "blue_threshold": round(float(self.blue_threshold_var.get()), 2),
        "illumination_enabled": bool(self.illumination_enabled),
        "clahe_clip_var": round(float(self.clahe_clip_var.get()), 2),
        "clahe_tile_var": int(self.clahe_tile_var.get())
    }

    # Salvare în JSON
    with open(config_path, "w") as json_file:
        json.dump(config_data, json_file, indent=4)

    print(f"Configurația a fost salvată în {config_path}")

if __name__ == "__main__":
    app = AdminGUI()
    app.mainloop()

```

Fișierul user_gui.py

```

from src.libs import *
from src.gui.modes.run_camera import run_camera
from src.detectors.yolo_detector import YoloDetector
from src.gui.frames.display_frame import display_frame
from src.gui.frames.schedule_frame import ScheduleFrame
from src.gui.frames.calendar_selector_frame import CalendarSelectorFrame

class UserGUI(tk.Toplevel):
    def __init__(self, main_gui):
        super().__init__()
        self.name="user"
        self.main_gui = main_gui
        self.title("AI Restaurant Monitoring System")
        self.geometry(f"{self.winfo_screenwidth()}x{self.winfo_screenheight()}")
        self.protocol("WM_DELETE_WINDOW", self.on_close)

        self.prev_time = time.time()
        self.last_time = time.time() # Momentul în care a fost actualizat ultima
        dată

        # Inițializează YOLO Detector cu un model implicit

```

```

self.current_model = "models/yolov101.pt"
self.detector = YoloDetector()
self.selected_camera = 0
self.video_resolution = (640, 480)
# Creează un frame pentru butoane
self.button_frame = ttk.Frame(self)
self.button_frame.pack(pady=10)

        self.button_back = ttk.Button(self.button_frame, text="Back",
command=self.go_back)
        self.button_back.grid(row=0, column=0, padx=5, pady=5)

        self.import_parameters_btn = ttk.Button(self.button_frame, text="Import
Parameters", command=self.import_parameters)
        self.import_parameters_btn.grid(row=0, column=1, padx=5, pady=5)

        self.detect_tables_btn = ttk.Button(self.button_frame, text="Detect
Tables", command=self.detect_tables)
        self.detect_tables_btn.grid(row=0, column=3, padx=5, pady=5)

        self.set_tables_btn = ttk.Button(self.button_frame, text="Set Tables",
command=self.set_tables, state="disabled")
        self.set_tables_btn.grid(row=0, column=4, padx=5, pady=5)

        self.reset_tables_btn = ttk.Button(self.button_frame, text="Reset Tables",
command=self.reset_tables, state="disabled")
        self.reset_tables_btn.grid(row=0, column=5, padx=5, pady=5)

        self.stop_detection_btn = ttk.Button(self.button_frame, text="Stop
Detection", command=self.stop_detection, state="disabled")
        self.stop_detection_btn.grid(row=0, column=6, padx=5, pady=5)

        # Add a button to open the schedule window
        self.open_schedule_button = ttk.Button(self.button_frame, text="Set Weekly
Schedule", command=self.open_schedule_window)
        self.open_schedule_button.grid(row=0, column=7, padx=5, pady=5)
        # Add Generate Statistics Button
        self.btn_generate_statistics = ttk.Button(self.button_frame,
text="Generate Statistics", command=self.open_statistics_calendar)
        self.btn_generate_statistics.grid(row=0, column=8, padx=5, pady=5)
        # Creează un frame pentru butoane
        self.max_frame = ttk.Frame(self)
        self.max_frame.pack(pady=10)

        # Variabile pentru timpii selectați pentru fiecare status
        self.time_available = tk.StringVar(value='Select Max Time Available')
        self.time_ready = tk.StringVar(value='Select Max Time Waiting')
        self.time_eating = tk.StringVar(value='Select Max Time Eating')
        self.time_clean = tk.StringVar(value='Select Max Time Clean')
        self.max_people_var = tk.StringVar(value='')

        # Crearea selectoarelor
        self.create_time_selectors()

        # Crearea input-ului pentru numărul maxim de oameni
        self.create_people_number_input()

        self.info_label = tk.Label(self, text="Tables: 0 | People: 0")
        self.info_label.pack(side=tk.TOP, fill=tk.X)
        self.start_info_update()

        self.canvas = Canvas(self, width=1920, height=1080)
        self.canvas.pack(side=tk.TOP, fill=tk.BOTH, expand=True)

```

```

self.label = ttk.Label(self.canvas)
self.label.pack()

self.status_label = ttk.Label(self, text="", wraplength=1000)
self.status_label.pack(side=tk.TOP, pady=10)

self.illumination_enabled = BooleanVar(value=False)
self.clahe_clip_var = tk.DoubleVar(value=2.0)
self.clahe_tile_var = tk.IntVar(value=8)

self.video_source = None
self.current_frame = None
self.running = False
self.camera_thread = None
self.images = []
self.image_index = 0
self.stop_event = threading.Event()
self.update_frame()
self.start_camera()
apply_modern_style(self)

def go_back(self):
    """Închide această fereastră și revine la MainGUI."""
    self.stop_running_thread()
    self.destroy() # Închide fereastra curentă
    self.main_gui.deiconify() # Afișează din nou MainGUI

def detect_tables(self):
    self.detector.detecting_tables_only = True
    self.detector.done_setting_tables = False
    self.detector.detecting_all = False
    self.set_tables_btn.config(state="normal")
    self.reset_tables_btn.config(state="disabled")
    self.stop_detection_btn.config(state="normal")

def set_tables(self):
    self.detector.detecting_tables_only = False
    self.detector.done_setting_tables = True
    self.detector.detecting_all = False
    self.change_model(self.current_model)
    self.current_frame = self.detector.set_table_ids()
    self.set_tables_btn.config(state="disabled")
    self.reset_tables_btn.config(state="normal")
    self.stop_detection_btn.config(state="normal")
    self.detector.table_manager.create_new_files()
    self.detector.table_manager.start_auto_save()
    self.detector.people_manager.create_new_files()
    self.detector.people_manager.start_auto_save()

def reset_tables(self):
    self.detector.detecting_tables_only = True
    self.detector.done_setting_tables = False
    self.detector.detecting_all = False
    self.detector.reset_table_manager()
    self.set_tables_btn.config(state="disabled")
    self.reset_tables_btn.config(state="disabled")

def stop_detection(self):
    self.detector.detecting_tables_only = False
    self.detector.done_setting_tables = False
    self.detector.detecting_all = False
    self.detector.reset_table_manager()
    self.set_tables_btn.config(state="disabled")

```

```

        self.reset_tables_btn.config(state="disabled")

def update_status_label(self):
    status_report = self.detector.get_tables_status_report()
    self.status_label.config(text=status_report)

def update_frame(self):
    """Actualizează și afișează cadrul la fiecare 17 ms."""
    display_frame(self, self.current_frame)
    self.after(17, self.update_frame) # Reapelează funcția la fiecare 17 ms

def stop_running_thread(self):
    """Oprește orice thread activ înainte de a porni unul nou."""
    self.stop_detection()
    self.stop_event.set() # Semnalăm oprirea thread-urilor
    self.running = False
    # Oprește thread camera
    if self.camera_thread and self.camera_thread.is_alive():
        self.camera_thread.join()
        self.camera_thread = None
        print("[INFO] camera_thread has stopped in User mode")

def generate_time_options(self, start_time, end_time, step_minutes):
    """
    Generăm intervalele de timp între start_time și end_time cu pasul
    step_minutes.
    """
    start = datetime.strptime(start_time, "%H:%M:%S")
    end = datetime.strptime(end_time, "%H:%M:%S")
    step = timedelta(minutes=step_minutes)

    times = []
    current_time = start
    while current_time <= end:
        times.append(current_time.strftime("%H:%M:%S"))
        current_time += step

    return times

def get_time_options(self, status_type):
    """
    Funcție care returnează timpuri disponibili pentru un anumit tip de status.
    """
    if status_type == 'available':
        return self.generate_time_options('00:30:00', '10:00:00', 30)
    elif status_type == 'ready to order':
        return self.generate_time_options('00:05:00', '01:00:00', 5)
    elif status_type == 'eating':
        return self.generate_time_options('00:30:00', '06:00:00', 30)
    elif status_type == 'need to clean':
        return self.generate_time_options('00:05:00', '02:00:00', 5)
    else:
        return []

def create_time_selectors(self):
    """
    Crează selectoarele de timp pentru fiecare status al mesei.
    """
    # Selector pentru "available"
    self.available_selector = self.create_time_selector('available',
self.time_available)

    # Selector pentru "ready to order"

```

```

        self.ready_selector = self.create_time_selector('ready to order',
self.time_ready)

        # Selector pentru "eating"
        self.eating_selector = self.create_time_selector('eating',
self.time_eating)

        # Selector pentru "need to clean"
        self.clean_selector = self.create_time_selector('need to clean',
self.time_clean)

def create_time_selector(self, status_type, time_var):
    """
    Creează un selector de timp pentru un anumit tip de status.
    """
    time_options = self.get_time_options(status_type)
    selector = ttk.OptionMenu(
        self.max_frame,
        time_var,
        f"Select max {status_type} time",
        *time_options,
        command=lambda _: self.update_time_status(status_type, time_var)
    )
    # Configurarea selectoarelor cu lățime și plasare
    selector.config(width=35)
    selector.pack(side=tk.LEFT, padx=5, pady=5)
    return selector

def update_time_status(self, status_type, time_var):
    """
    Funcție de comandă care salvează timpul ales pentru un anumit status.
    """
    time = time_var.get()
    print(f"Selected time for {status_type}: {time}")
    self.detector.table_manager.set_max_time(status_type, time)
    # Aici poți salva valorile în variabilele corespunzătoare sau le poți
    folosi în aplicație.

def create_people_number_input(self):
    # Setează valoarea inițială a câmpului de input din PeopleManager
    self.max_people_var =
tk.StringVar(value=str(self.detector.people_manager.get_max_people_number()))
    people_entry = ttk.Entry(self.max_frame, textvariable=self.max_people_var,
width=10, justify='center')

    # Validare: doar cifre
    validate_command = (self.register(self.validate_unsigned_int), '%P')
    people_entry.config(validate='key', validatecommand=validate_command)

    # Event pentru a actualiza valoarea când utilizatorul apasă Enter
    label = ttk.Label(self.max_frame, text="Max People:")
    label.pack(side=tk.LEFT, padx=5)

    people_entry.bind('<Return>', self.update_max_people)
    people_entry.pack(side=tk.LEFT, padx=5)

def validate_unsigned_int(self, value):
    """
    Permite doar valori unsigned int în câmpul de input.
    """
    if value == "":
        return True # Permite câmp gol temporar
    return value.isdigit()

```

```

def update_max_people(self, event=None):
    """
    Actualizează valoarea maximă de persoane în PeopleManager.
    """
    try:
        max_people = int(self.max_people_var.get())
        self.detector.people_manager.set_max_people_number(max_people)
        print(f"Max people number updated to: {max_people}")
    except ValueError as e:
        messagebox.showerror("Invalid Input", "Please enter a positive integer.")
    # Resetează la valoarea actuală
    self.max_people_var.set(str(self.detector.people_manager.get_max_people_number()))

def start_info_update(self):
    """Inițiază actualizarea periodică a informațiilor."""
    self.update_info() # Actualizează imediat
    self.after(1000, self.start_info_update) # Reapelează peste 1 secundă

def update_info(self):
    """Actualizează textul pentru numărul de mese și persoane detectate."""
    self.info_label.config(
        text=f"Tables: {self.detector.tables_number} | "
        f"People: {self.detector.people_number}"
    )

def open_schedule_window(self):
    """Function to open the schedule frame in a new window."""
    schedule_window = tk.Toplevel()
    schedule_window.title("Set Weekly Schedule")
    schedule_window.geometry("600x400") # Adjust size as needed
    frame = ScheduleFrame(schedule_window)
    frame.pack(fill="both", expand=True, padx=10, pady=10)

def open_statistics_calendar(self):
    """Function to open the statistics calendar window in a new window."""
    statistics_window = tk.Toplevel() # Creează o fereastră de tip Toplevel
    statistics_window.title("Select Date Range for Statistics") # Titlul ferestrei
    statistics_window.geometry("600x400") # Setează dimensiunile ferestrei

    # Creează frame-ul pentru calendar și alte componente
    statistics_frame = CalendarSelectorFrame(statistics_window) # Pass the Toplevel window to CalendarSelectorFrame
    statistics_frame.pack(fill="both", expand=True, padx=10, pady=10) # Adaugă frame-ul în fereastră

def import_parameters(self):
    # Deschide fereastra de selecție fișier
    file_path = filedialog.askopenfilename(
        initialdir="data/config",
        title="Select Config File",
        filetypes=(("JSON files", "*.json"), ("All files", "*.*"))
    )

    if file_path:
        try:
            # Citește fișierul JSON
            with open(file_path, "r") as json_file:
                config_data = json.load(json_file)

            # Afișează valorile în consolă

```

```

        self.selected_camera = config_data.get("selected_camera", "Not
set")
        print("Selected Camera:", self.selected_camera)
        self.current_model = config_data.get("selected_model", "Not set")
        print("Selected Model:", self.current_model)
        self.video_resolution = config_data.get("video_resolution", "Not
set")
        print("Selected Format:", self.video_resolution)

        max_times = config_data.get("max_times", {})
        self.max_time_available = max_times.get("available", "Not set")
        self.max_time_ready_to_order = max_times.get("ready_to_order",
"Not set")
        self.max_time_eating = max_times.get("eating", "Not set")
        self.max_time_need_to_clean = max_times.get("need_to_clean", "Not
set")

        self.detector.table_manager.set_max_time("available",
self.max_time_available)
        self.detector.table_manager.set_max_time("ready_to_order",
self.max_time_ready_to_order)
        self.detector.table_manager.set_max_time("eating",
self.max_time_eating)
        self.detector.table_manager.set_max_time("need_to_clean",
self.max_time_need_to_clean)

        print("Max Time Available:", self.max_time_available)
        print("Max Time Ready to Order:", self.max_time_ready_to_order)
        print("Max Time Eating:", self.max_time_eating)
        print("Max Time Need to Clean:", self.max_time_need_to_clean)

        self.max_people = config_data.get("max_people", "Not set")
        self.detector.people_manager.set_max_people_number(self.max_peop
le)
        print("Max People:", self.max_people)

        self.overlap_threshold = config_data.get("overlap_threshold", "Not
set")
        self.detector.set_overlap_threshold(self.overlap_threshold)
        print("Overlap Threshold:", self.overlap_threshold)

        self.red_threshold = config_data.get("red_threshold", "Not set")
        self.detector.table_manager.set_red_threshold_for_all_tables(sel
f.red_threshold)
        print("Red Threshold:", self.red_threshold)

        self.blue_threshold = config_data.get("blue_threshold", "Not set")
        self.detector.table_manager.set_blue_threshold_for_all_tables(se
lf.blue_threshold)
        print("Blue Threshold:", self.blue_threshold)

        self.illumination_enabled.set(config_data.get("illumination_enab
led", False))
        print("Illumination Enabled:", self.illumination_enabled.get())

        self.clahe_clip_var.set(float(config_data.get("clahe_clip_var",
2.0)))
        print("CLAHE Clip Limit:", self.clahe_clip_var.get())

        self.clahe_tile_var.set(int(config_data.get("clahe_tile_var", 8)))
        print("CLAHE Tile Size:", self.clahe_tile_var.get())

        # Actualizează UI
        self.time_available.set(str(self.max_time_available))

```

```

        self.time_ready.set(str(self.max_time_ready_to_order))
        self.time_eating.set(str(self.max_time_eating))
        self.time_clean.set(str(self.max_time_need_to_clean))
        self.max_people_var.set(str(self.max_people))

        self.start_camera()
    except Exception as e:
        print(f"Error reading the config file: {e}")

def on_close(self):
    """Curăță toate apelurile 'after' și închide aplicația."""
    self.after_cancel(self.update_frame)
    self.after_cancel(self.start_info_update)
    self.destroy() # Închide aplicația

def start_camera(self):
    self.selected_mode = 'camera'

    self.stop_running_thread()
    self.stop_event.clear()
    self.running = True

    self.camera_thread = threading.Thread(target=run_camera, args=(self,))
    self.camera_thread.start()

def change_model(self, model_name):
    """Schimbă modelul YOLO utilizat."""
    self.current_model = model_name
    self.detector.load_model(os.path.join("models", self.current_model))
    self.status_label.config(text=f"Modelul a fost schimbat la
{model_name}")
if __name__ == "__main__":
    app = UserGUI()
    app.mainloop()

```

Fișierul *yolo_detector.py*

```

from src.libs import *
from src.managers.table_manager import TableManager
from src.managers.people_manager import PeopleManager
from src.managers.alarm_manager import AlarmManager

class YoloDetector:
    def __init__(self, model_path="models/yolov10l.pt"): # Path to the YOLO model
        self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        # Folosește modelul default 'yolov8n.pt' dacă nu este specificat un
model_path
        self.model_path = model_path
        self.load_model(self.model_path)
        self.table_manager = TableManager() # Instanțiază managerul de mese
        self.people_manager = PeopleManager() # Instanțiem PeopleManager
        self.alarm_manager = AlarmManager() # Instanțiază AlarmManager
        self.detecting_tables_only = False
        self.done_setting_tables = False
        self.detecting_all = False
        self.tables_detected = []

```

```

self.tables_number = 0
self.people_number = 0
self.overlap_threshold = 0.2
self.frame_score=0
print("Yolo running on", self.device)

# Aici definim clasele obiectelor speciale
self.special_object_classes = {
    39: 'bottle', 40: 'wine glass', 41: 'cup', 42: 'fork', 43: 'knife',
    44: 'spoon', 45: 'bowl', 46: 'banana', 47: 'apple', 48: 'sandwich',
    49: 'orange', 50: 'broccoli', 51: 'carrot', 52: 'hot dog',
    53: 'pizza', 54: 'donut', 55: 'cake'
}

def load_model(self, model_path):
    """Încarcă modelul YOLO de la calea specificată pe dispozitivul adecvat."""
    self.model_path = model_path
    self.model = YOLO(model_path, verbose=False).to(self.device)
    print(f"Modelul {model_path} a fost încărcat pe {self.device}")

def detect(self, frame):
    frame_normalized = (frame / 255.0).astype("float32") # Normalizează
    cadrul doar pentru model
    frame_tensor = torch.from_numpy(frame_normalized).to(device).permute(2,
0, 1).unsqueeze(0)

    results = self.model(frame_tensor)

    if len(results) == 0 or len(results[0].boxes) == 0:
        return []

    detections = []
    for box in results[0].boxes:
        x1, y1, x2, y2 = box.xyxy[0]
        cls = int(box.cls[0])
        conf = box.conf[0]

        detections.append({
            'box': (x1.item(), y1.item(), x2.item(), y2.item()),
            'class': cls,
            'confidence': conf.item()
        })

    filtered_detections = filter_detections(detections,
self.overlap_threshold)
    self.tables_number = sum(1 for d in filtered_detections if d['class'] ==
60)
    self.people_number = sum(1 for d in filtered_detections if d['class'] ==
0)
    # Numărăm persoanele detectate

    self.people_manager.set_people_number(self.people_number)

    return filtered_detections

def draw_only_tables(self, frame, detections):
    self.tables_detected = [det for det in detections if det['class'] == 60]
    num_tables = len(self.tables_detected)
    frame_height, frame_width, _ = frame.shape
    total_score = 0

    if self.tables_detected:
        for table in self.tables_detected:
            box = table['box']

```

```

        x1, y1, x2, y2 = box
        center_x = (x1 + x2) / 2
        center_y = (y1 + y2) / 2

        # Calculăm abaterea centrului mesei față de centrul cadrului,
        # normalizată la dimensiunile cadrului.
        center_deviation_x = abs(center_x - frame_width / 2) / (frame_width
        / 2)
        center_deviation_y = abs(center_y - frame_height / 2) /
        (frame_height / 2)

        # Cu cât abaterea este mai mică, cu atât scorul local este mai
        # mare (max 1 pentru fiecare axă).
        center_score = (1 - center_deviation_x) * (1 - center_deviation_y)

        color = (0, 255, 0)
        label = "table (score: {:.2f})".format(center_score) # Afișăm
        scorul local al mesei
        frame = cv2.rectangle(frame, (int(x1), int(y1)), (int(x2),
        int(y2)), color, 2)
        frame = cv2.putText(frame, label, (int(x1), int(y1) - 5),
        cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 1)

        total_score += center_score

        # Scorul final este media scorurilor meselor, ponderată cu numărul de
        # mese.
        self.frame_score = (total_score / num_tables) * num_tables if
        num_tables > 0 else 0
        else:
            self.frame_score=0

    return frame

def draw_detection_with_table_id(self, frame, detections):
    if detections:
        for det in detections:
            box = det['box']
            class_id = det['class']
            x1, y1, x2, y2 = box
            if class_id == 0: # Persoană
                label = "people"
                people_diff = self.people_manager.people_count -
                self.people_manager.get_max_people_number()
                if people_diff > 0:
                    color = (0, 0, 255)
                    self.trigger_alarm(people_diff=people_diff)
                else:
                    color = (255, 0, 0)
            elif class_id in self.special_object_classes:
                label = f"{self.special_object_classes[class_id]}"
                color = (0, 0, 255)
            else:
                continue

            frame = cv2.rectangle(frame, (int(x1), int(y1)), (int(x2),
            int(y2)), color, 2)
            frame = cv2.putText(frame, label, (int(x1), int(y1) - 5),
            cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 1)

        if self.tables_detected:
            for table in self.tables_detected:
                box = table['box']
                class_id = table['class']

```

```

        x1, y1, x2, y2 = box
        table_id = self.table_manager.get_table_id_by_overlap(box)
        self.table_manager.assign_table_id(table_id, box)
        if table_id is None:
            table_id = self.table_manager.table_counter
            # Verificăm și actualizăm statusul mesei
            self.table_manager.check_and_update_status(detections)
        status, duration = self.table_manager.get_table_info(table_id)

        formatted_duration = format_time(duration) # Formatează
durata

        label = f"TABLE{table_id} {status} for {formatted_duration}"
        max_time = self.table_manager.get_max_time(status)
        time_diff = duration - convert_duration(max_time)
        if max_time is not None and time_diff > 0:
            color = (0, 0, 255)
            self.trigger_alarm(time_diff=time_diff, table_id=table_id,
status=status)
        else:
            color = (0, 255, 0)
            frame = cv2.rectangle(frame, (int(x1), int(y1)), (int(x2),
int(y2)), color, 2)
            frame = cv2.putText(frame, label, (int(x1), int(y1) - 5),
cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 1)
            return frame

def draw_auto_detections(self, frame, detections):
    if detections:
        for det in detections:
            box = det['box']
            class_id = det['class']
            x1, y1, x2, y2 = box
            if class_id == 0: # Persoană
                label = "people"
                color = (255, 0, 0)
            elif class_id == 60: # Masă
                table_id = self.table_manager.get_table_id_by_overlap(box)
                self.table_manager.assign_table_id(table_id, box)
                if table_id is None:
                    table_id = self.table_manager.table_counter
                    # Verificăm și actualizăm statusul mesei
                    self.table_manager.check_and_update_status(detections)
                status, duration = self.table_manager.get_table_info(table_id)
                formatted_duration = format_time(duration) # Formatează
durata

                label = f"Table {table_id} {status} for {formatted_duration}"
                color = (0, 255, 0)
            elif class_id in self.special_object_classes:
                label = f"{self.special_object_classes[class_id]}"
                color = (0, 0, 255)
            else:
                continue

            frame = cv2.rectangle(frame, (int(x1), int(y1)), (int(x2),
int(y2)), color, 2)
            frame = cv2.putText(frame, label, (int(x1), int(y1) - 5),
cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 1)

            return frame

def draw_detections(self, frame, detections):
    if detections:
        for det in detections:
            box = det['box']

```

```

class_id = det['class']
x1, y1, x2, y2 = box
if class_id == 0: # Persoană
    label = f"people: {det['confidence']*100:.2f}%"
    color = (33, 150, 243) # Albastru modern
elif class_id == 60: # Masă
    label = f"table: {det['confidence']*100:.2f}%"
    color = (76, 175, 80) # Verde modern
elif class_id in self.special_object_classes:
    label = f"{self.special_object_classes[class_id]}:
{det['confidence']*100:.2f}%"
    color = (255, 76, 76) # Roșu modern
else:
    continue

    frame = cv2.rectangle(frame, (int(x1), int(y1)), (int(x2),
int(y2)), color, 2)
    #frame = cv2.putText(frame, label, (int(x1), int(y1) - 5),
cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 1)
    frame = self.draw_advanced_label(frame, y1, x1, label, color)

return frame

def draw_advanced_label(self, frame, y1, x1, label, color):
    # Calculăm dimensiunile label-ului
    font = cv2.FONT_HERSHEY_PLAIN
    font_scale = 1
    font_thickness = 1
    text_size = cv2.getTextSize(label, font, font_scale, font_thickness)[0]

    # Calculăm poziția label-ului
    text_x = int(x1) # Plasăm textul de la colțul dreapta al dreptunghiului
    text_y = int(y1) - 5 # Ajustăm poziția pentru text mai sus

    # Desenăm fundalul label-ului cu culoare solidă, începând de la colțul
drept al dreptunghiului
    label_padding = 5 # Spațiu mic între text și marginea fundalului
    cv2.rectangle(frame,
        (text_x, text_y - text_size[1] - label_padding), # Colțul
stâng al fundalului
        (text_x + text_size[0] + label_padding, text_y +
label_padding), # Colțul drept al fundalului
        color, -1) # Fundal colorat

    # Desenăm textul alb pe fundalul colorat
    frame = cv2.putText(frame, label, (text_x, text_y), font, font_scale,
(255, 255, 255), font_thickness, cv2.LINE_AA)

return frame

def get_tables_status_report(self):
    """
    Apelează funcția din TableManager pentru a obține statusul tuturor meselor
    și returnează șirul rezultat.
    """
    return self.table_manager.get_all_tables_status()

def set_overlap_threshold(self, threshold):
    if 0 <= threshold <= 1:
        self.overlap_threshold = threshold
    else:
        raise ValueError("Threshold must be between 0 and 1")

def set_table_ids(self):

```

```

self.reset_table_manager()
if self.tables_detected:
    for table in self.tables_detected:
        box = table['box']
        self.table_manager.assign_table_id(None, box)
        self.detecting_tables_only = False

def reset_table_manager(self):
    self.table_manager.reset_tables()

def trigger_alarm(self, people_diff = 0, time_diff= 0, table_id = -1,
status="unknown"):
    """Apelăm funcția de alarmă din AlarmManager"""
    self.alarm_manager.play_alarm_sound() # Redă alarmă
    self.alarm_manager.popup(people_diff, time_diff, table_id, status)

def preprocess_frame(self, frame):
    """
    Corectează automat iluminarea imaginii pentru îmbunătățirea detecției în
YOLO.
    Include egalizare histogramă, ajustare gamma și denoising adaptiv.
    """

    # Convertim în spațiu HSV pentru a analiza luminozitatea
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    h, s, v = cv2.split(hsv)

    # Calculăm media luminozității
    brightness = np.mean(v)

    # Stabilim praguri de corecție (poți ajusta după testare)
    low_light_threshold = 60
    high_light_threshold = 200

    # 1. Egalizare histogramă dacă imaginea e slab iluminată
    if brightness < low_light_threshold:
        v = cv2.equalizeHist(v)
        hsv = cv2.merge([h, s, v])
        frame = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)

    # 2. Aplicăm și gamma correction (între 1.2 și 1.8)
    gamma = np.interp(brightness, [0, 80], [1.8, 1.2])
    inv_gamma = 1.0 / gamma
    table = np.array([(i / 255.0) ** inv_gamma * 255 for i in
np.arange(256)]).astype("uint8")
    frame = cv2.LUT(frame, table)

    # 3. Denoising pentru cadrele întunecate
    frame = cv2.fastNlMeansDenoisingColored(frame, None, 10, 10, 7, 21)

    # Supraluminozitate - opțional: reducem puțin contrastul
    elif brightness > high_light_threshold:
        frame = cv2.convertScaleAbs(frame, alpha=0.95, beta=-10)

    return frame

```

Fișierul *main.py*

```

from src.libs import *
from src.gui.user_gui import UserGUI
from src.gui.admin_gui import AdminGUI

ADMIN_PASSWORD = "1234" # Setează parola corectă pentru accesul admin

```

```

class MainGUI(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("AI restaurant")
        self.geometry("400x300")
        self.create_widgets()
        apply_modern_style(self)

    def create_widgets(self):
        label = tk.Label(self, text="Choose Interface", font=("Arial", 16))
        label.pack(pady=20)

        button_user = ttk.Button(self, text="User", command=self.launch_user_gui)
        button_user.pack(pady=10)

        button_admin = ttk.Button(self, text="Admin",
command=self.request_admin_password)
        button_admin.pack(pady=10)

    def launch_user_gui(self):
        """Lansează interfața UserGUI și ascunde fereastra principală."""
        self.withdraw() # Ascunde fereastra MainGUI
        user_gui = UserGUI(self) # Trimite referința către MainGUI
        user_gui.mainloop()

    def launch_admin_gui(self):
        """Lansează interfața AdminGUI și ascunde fereastra principală."""
        self.withdraw() # Ascunde fereastra MainGUI
        admin_gui = AdminGUI(self) # Trimite referința către MainGUI
        admin_gui.mainloop()

    def request_admin_password(self):
        """Solicită parola și verifică dacă este corectă înainte de a lansa
interfața admin."""
        password = simpledialog.askstring("Admin Login", "Enter Admin Password:",
show="*")

        if password == ADMIN_PASSWORD:
            self.launch_admin_gui()
        else:
            messagebox.showerror("Access Denied", "Incorrect password!")

if __name__ == "__main__":
    app = MainGUI()
    app.mainloop()

```

Fisierul *run_camera.py*

```

from src.libs import *

def run_camera(app):
    # Se folosește rezoluția setată în self.video_resolution (ex: (640, 480),
(1920, 1080), etc.)
    if app.selected_camera:
        current_camera = app.selected_camera
    else:
        # Valoare implicită, dacă nu este setată o rezoluție
        current_camera = 0

    # Se folosește rezoluția setată în self.video_resolution (ex: (640, 480),
(1920, 1080), etc.)
    if app.video_resolution:
        resolution = app.video_resolution

```

```

else:
    # Valoare implicită, dacă nu este setată o rezoluție
    resolution = (640, 480)

print("[INFO] Pornire cameră:", current_camera)
print("[INFO] Format cameră:", resolution)

cap = cv2.VideoCapture(int(current_camera))
cap.set(cv2.CAP_PROP_FRAME_WIDTH, resolution[0])
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, resolution[1])

previous_resolution = resolution # Variabilă pentru a verifica modificarea
rezoluției

while app.running and not app.stop_event.is_set(): # Verificăm stop_event
    pentru oprire
        if app.selected_camera != current_camera: # Dacă utilizatorul schimbă
camera
            print(f"[INFO] Schimbare cameră: {current_camera} ->
{app.selected_camera}")
            cap.release() # Eliberăm resursele camerei anterioare
            current_camera = app.selected_camera
            cap = cv2.VideoCapture(current_camera) # Deschidem noua cameră
            cap.set(cv2.CAP_PROP_FRAME_WIDTH, resolution[0]) # Setăm noua
rezoluție
            cap.set(cv2.CAP_PROP_FRAME_HEIGHT, resolution[1]) # Setăm noua
rezoluție

            # Verificăm dacă s-a schimbat rezoluția
            if app.video_resolution != previous_resolution:
                print(f"[INFO] Schimbare rezoluție: {previous_resolution} ->
{app.video_resolution}")
                cap.set(cv2.CAP_PROP_FRAME_WIDTH, app.video_resolution[0])
                cap.set(cv2.CAP_PROP_FRAME_HEIGHT, app.video_resolution[1])
                previous_resolution = app.video_resolution # Actualizăm rezoluția
anterioară

            ret, frame = cap.read()
            if not ret:
                print("[ERROR] Eroare la citirea frame-ului!")
                break

            app.current_frame = frame # Stocăm frame-ul curent

cap.release() # Eliberăm camera la ieșire
print("[INFO] Camera oprită.")

```

Fișierul *arduino_servo_camera.ino*

```

#include <ESP32Servo.h> // Biblioteca pentru controlul servo-urilor pe ESP32

Servo servoOx; // Obiect pentru servo-ul de pe axa Ox
Servo servoOy; // Obiect pentru servo-ul de pe axa Oy

// Variabile pentru a stoca unghiurile actuale și finale
int currentOx = 90; // Unghiul curent al servo-ului Ox (poziția inițială)
int currentOy = 90; // Unghiul curent al servo-ului Oy (poziția inițială)
int targetOx = 90; // Unghiul țintă pentru Ox
int targetOy = 90; // Unghiul țintă pentru Oy

// Variabile pentru a memora ultimele valori primite prin serial
int lastOx = 90;
int lastOy = 90;

```

```

int stepSize = 1; // Dimensiunea pasului de incrementare a unghiului
unsigned long lastUpdateTime = 0; // Timpul ultimei actualizări
unsigned long updateInterval = 20; // Intervalul de actualizare (în ms)

void setup() {
    // Setează pini pentru servo-uri (poți schimba acești pini dacă este necesar)
    servoOx.attach(6); // Servo pentru axa Ox pe pinul GPIO15
    servoOy.attach(7); // Servo pentru axa Oy pe pinul GPIO13

    // Setează pozițiile inițiale ale servo-urilor
    servoOx.write(currentOx);
    servoOy.write(currentOy);

    // Inițializează comunicarea serială
    Serial.begin(921600); // Setează viteza de comunicare la 921600 bauds
}

void loop() {
    // Verifică dacă există date disponibile pe portul serial
    if (Serial.available() > 0) {
        // Citește linia de date (până la un caracter de linie nouă '\n')
        String input = Serial.readStringUntil('\n');

        // Caută poziția caracterului '_', care separă cele două valori
        int underscoreIndex = input.indexOf('_');

        if (underscoreIndex != -1) {
            // Extrage valoarea pentru Ox (înainte de '_')
            String oxValue = input.substring(0, underscoreIndex);
            // Extrage valoarea pentru Oy (după '_')
            String oyValue = input.substring(underscoreIndex + 1);

            // Convertește valorile în numere întregi
            int ox = oxValue.toInt();
            int oy = oyValue.toInt();

            // Verifică dacă valorile sunt în intervalul valid (0 - 180)
            if (ox >= 0 && ox <= 180 && oy >= 0 && oy <= 180) {
                // Actualizează valorile țintă și memorează ultimele valori valide
                targetOx = ox;
                targetOy = oy;
                lastOx = ox;
                lastOy = oy;
            }
        }
        // În cazul în care conexiunea este pierdută, folosește ultimele valori
        // memorate
        targetOx = lastOx;
        targetOy = lastOy;
    }

    // Incrementare treptată către valorile țintă
    unsigned long currentTime = millis();
    if (currentTime - lastUpdateTime >= updateInterval) {
        lastUpdateTime = currentTime;

        // Mișcă servo-ul pe axa Ox către țintă
        if (currentOx < targetOx) {
            currentOx += stepSize;
            if (currentOx > targetOx) currentOx = targetOx;
        } else if (currentOx > targetOx) {
            currentOx -= stepSize;

```

```

    if (currentOx < targetOx) currentOx = targetOx;
}

// Mișcă servo-ul pe axa Oy către țintă
if (currentOy < targetOy) {
    currentOy += stepSize;
    if (currentOy > targetOy) currentOy = targetOy;
} else if (currentOy > targetOy) {
    currentOy -= stepSize;
    if (currentOy < targetOy) currentOy = targetOy;
}

// Trimite noile poziții către servo-uri
servoOx.write(currentOx);
servoOy.write(currentOy);
}
}

```